The PAN simulator book

First printing, August 2019

Contents

1. Preface

This book represents a “hands on” introduction to the PAN circuit simulator. Since the goal is to provide a fast track to potential users, it is formatted as a “walk-through” that starts from the simplest concepts and evolves towards more complex methods and analyses. This book may be used to start using PAN in a simple and direct way.

PAN is an academic simulator, it has many powerful features, it is free, but, obviously, most certainly has bugs of which we are not aware. We are quite eager to correct them, and also to implement new user requested features, but users may not expect the same kind of (excellent) support that comes with commercial simulators.

2. A quick PAN walk through

2.1 Overview

The PAN simulator is mainly a circuit simulator but can be easily extended and controlled in order to perform several different types of system simulations.
PAN is capable of analog, digital or AMS simulation, electro-thermal simulation, power distribution and generation systems and can be extended by linking external modules, written through the VERILOGA language.
A complete list of analyses, devices and directives that are available natively (i.e. without *user extensions*) can be listed by following the simple instruction given in Section 2.2.

This chapter represents a sort of tutorial showing in the most direct way and through a limited number of examples, how to use the PAN circuit simulator to perform simple simulation tasks. More complex tasks and a detailed definition of all parameters proper to each analysis method will be given in following chapters.

This is the first version of the book and we are fully aware that is may be incomplete and also there may be some parts that can be improved.

2.1.1 Where and How: downloading and installing

The PAN binary requires a standard linux installation. PAN developers use Debian based systems (most recently Debian, Mint and Ubuntu), but PAN has been used without any pain with other Linux flavours, including RedHat, Gentoo, Sabayon and several others.

Using PAN is relatively simple and only requires knowledge of basic Linux terminal commands and the use of any ASCII editor (such as vi). Netlists are entered in ASCII format, but, as it will be shown in the sequel, thanks to the high compatibility of PAN with well known simulator languages, almost any standard netlister can be used, as long as some adjustments to the command language are made.

PAN can be used as a simulation engine in the MATLAB environment; a procedural interface allows the user to run simulations and collect results in the MATLAB environment. Circuit parameters can be changed (altered) directly by MATLAB.

2.2 Reference manual

PAN implements a command line quick reference manual. It is available with

```
pan -s
```

A list of keywords appears, and further help on a specific topic can be obtained with the command

```
pan -s topic
```

Each topic may have several sub-topics. When a topic is entered, all the sub-topics are listed, too. The meaning of a specific sub-topic can be obtained through the command

```
pan -s topic -si subtopic
```

2.3 Netlist files

PAN automatically understands different netlist formats of SPICE, ELDO and SPECTRE. However there is not 100% compatibility and some statements are not fully recognised. This means that the circuit netlist can be written using standard schematic capture software. PAN specific commands, analysis and options must be added manually. In the following examples, PAN "native" format is used; it is very simple and (almost) free format.

PAN netlist and control files are usually written using extension *.pan*, and simulation execution is, in the most simple case, started with

```
prompt> pan netlist.pan
```

The file suffix can be omitted if the Linux PAN_SUFFIX environment variable is given and consists of a space separated list of admissible suffices. For example `setenv PAN_SUFFIX ''pan spi cir''` tells that a file name can be followed by the "pan", "spi" or "cir" suffix.

PAN is case sensitive, upper case and lower case letters are different. Since other simulators such as for example SPICE are not case sensitive, when PAN automatically enters the compatibility mode in reading "foreign" netlists it also becomes carefully case insensitive. Some more information is given in Section 11.2.

In the following the few basic rules that any user needs to know in order to write simple netlist files are reported and some of them are exemplified in Netlist 2.3.1. More complex rules, such as those needed to define subcircuits, models, or control structures, will be shown later.

Comments Comments start from character ";" and continue till the end of line. The "//" character set is equivalent to ";". Recall that SPICE comments are supported.

Devices Each device instance is described by a single line, starting with the *instance name* (that must, obviously, be unique), then the list of nodes to which the instance is connected, the device type (e.g. "resistor", "capacitor" etc.), and, finally, instance specific parameters and options. More details are given in the following. Instance names can be alphanumeric or quoted strings.

Analysis Analysis are in general written on a single line. The first token of the line must be a unique identifier, the second is the analysis type (e.g. dc, tran etc.), finally all analysis parameters and options.

Long lines For formatting or aesthetic purposes it is possible to break long lines using character "\" not followed by any character. The '+' character at the first position of the following line is supported as continuation character as in SPICE netlists.

Nodes Nodes are implicitly defined in the instance definition lines. Node names can be integer positive numbers, alphanumeric strings or quoted strings. The only exception is the reference node that must be explicitly defined using the netlist command (or "card" as in the old `spice` tradition).

```
ground electrical name
```

where *name* can be any unique identifier. The `electrical` keyword must be used since as shown in the following there can be different "reference nodes", such as for example the ambient temperature in an electro-thermal netlists.

Device and Analysis Parameters device and analysis parameters are written in the form

```
name = value
```

separated by spaces. Boolean values can be entered also using keywords `yes` or `no`, `true` or `false` and `1` or `0`.

Numbers In `PAN` all numbers are double precision numbers, thus for example we have that $1 \equiv 1.0 = 1.0\text{E}+00$. Numbers can be followed by a character that specifies the scaling factor and a possible substring. Supported scaling factors are

$$\begin{array}{cccccc} \text{a} \equiv 10^{-18} & \text{f} \equiv 10^{-15} & \text{p} \equiv 10^{-12} & \text{n} \equiv 10^{-9} & \text{u} \equiv 10^{-6} & \text{m} \equiv 10^{-3} \\ \text{k} \equiv \text{K} \equiv 10^{3} & \text{M} \equiv 10^{6} & \text{G} \equiv 10^{9} & \text{T} \equiv 10^{12} & \_ \equiv 10^{0} = 1 & \text{i} \equiv \text{inch} \end{array}$$

With the SPICE syntax the `meg` suffix is equivalent to 10^6. For example the

```
parameters R1=1_Ohm R2=1kOhm A=10_Kelvin
```

command sets the `R1` parameter equal to 1 and `R2` equal to 1000, here the substring `Ohm` is a mnemonic to the meaning of the parameter. Recall to not forget the `_` underscore in `A` value otherwise the first character of `Kelvin` is interpreted as the 10^3 scale factor.

Netlist 2.3.1 — Basic netlist writing rules. –

```
ground electrical 0    ; This is a comment
; Circuit
r1  n1 n2      resistor  r=1k       ; a resistor instance
InstanceName    Node1 Node2 Node3 Node4 \
    Node5 Node6 Node7  MultiportDevice option1=10 \
    option2=30  ; Instance of MultiportDevice with line breaks
                ; Note that the netlist is incomplete to
                ; run a successfull simulation !
```

2.3.1 Encryption

Any part of a netlist can be encrypted by the

```
prompt> pan -cy encrypted.cy input.pan
```

statement, where the `-cy` command line parameter tells that the "input.pan" input file must be encrypted in the "encryted.cy" output file. The "encryted.cy" file can be later inserted in a another netlist or directly read in it through the include "encrypted.cy" statement. Encryption allows to distribute and use private netlists and models without having access to the netlist itself.

2.4 Models and devices

Any circuit is essentially described by a topology and a set of components. Components can be defined in many different ways. `PAN` includes a large number of predefined standard devices, several customizable devices and the possibility to define completely new devices by writing code in many different languages. Moreover, any predefined or user defined device can be made in to a template using the `model` statement. This is a standard feature of most SPICE like simulators.

Netlist 2.4.1 — An example of definition and use of a diode model. –

```
ground electrical 0
; Circuit
r1  n1  0      resistor  r=1k
d1  n1  0      MYDIODE
d2  n1  0      MYDIODE area=100  // This is an instance parameter,
                    // that must be not confused with model parameters.
; Models
model MYDIODE diode is=3.142f rs=4.2 bv=80  // These are model parameters
```

An example of model definition and use is shown in Netlist 2.4.1, note how diode instance `d1` is defined by the model template "as is" while diode `d2` is based on model `MYDIODE` but has the additional `area` instance defined. Instance parameters apply to the specific instance and override model parameters. The `MYDIODE` model is the master that is instantiated and customised in the netlist. In our case there are 2 diode instances of the same diode master; `d2` instance is customised for what concerns the diode area. The `model` card specifying a model can be put anywhere in the netlist. Models defined outside the body of sub-circuits are considered as global. Those inside a sub-circuit are local to the sub-circuit itself.
Definition of new components will be discussed in a following section.

2.4.1 Types of sub-netlists

Netlists can be composed of sub-netlist of the power, electrical and thermal types. A sub-netlist of a type can contain sub-netlists of other types. For example an electrical sub-netlist can contain a power subnetlist and vice versa. The body of a sub-netlist starts with the `'begin name'` statement where `'name'` is the sub-netlist type and ends wirh the `'end'` statement. The default type of a netlist is electrical. A sub-netlist can be composed exclusively by instances that admit the same type. For example a thermal sub-netlist can not contain istances of any transistor since they are not thermal elements. It can contain (thermal) resistor and (thermal) capacitor, but once more no inductors.
In the analysis instance stamenent the type of sub-netlists is selected through the `'nettype=integer'` parameter. For example the instance

```
Dc dc nettype=1
```

tells the `Dc` analysis to consider the default electrical sub-networks together with the power sub-network.

2.5 Global options

Global simulation options, such as temperature, convergence aid parameter values (gmin, gground...), topology checker options, global verbosity level and many more are defined with line:

```
options option1=value1 option2=value2 ...
```

as many space separated options as desired can be given after the `options` keyword.

2.5.1 The `mindevres` option

The `mindevres` parameter sets the minimum value of resistance of resistors both as built-in elements available in `PAN` and as parasitic resistors of semiconductor devices such for example diodes, mosfets, jfets. If the `mindevres` option is not specified by the user and a resistance value

is less than `mindevres` an error is given and `PAN` stops. If the `mindevres` options is given, the value of the resistance is clipped to the `mindevres` value, a warning message is displayed and `PAN` continues execution. Note that this clipping alter the original value of the resistance and thus alter the results of simulations.

2.6 Parameters

2.6.1 Global parameters

A simple definition of parameter can be an algebraic expression. A parameter is defined by

```
parameter param_name=expression
```

where the `parameter` keyword introduces the parameter definition; *param_name* is the name of the parameter and *expression* is an algebraic expression that defines the value of the parameter. For example the netlist 2.6.1

Netlist 2.6.1 — An example of parameter definition and use. –

```
r1  n1  n2  resistor r=R
r2  n2  n3  resistor r=X+Y
parameters X=3 Y=4 R=sqrt(X**2+Y**2)
```

defines the `X`, `Y` and `R` parameters. The value of $\mathtt{R} = \sqrt{3^2+4^2} = 5$ is derived from the values of `X` and `Y`. Note the ordering of the parameter definitions is important. The `r1` instance is a resistor of `R` value. The `r2` instance is a resistor of $\mathtt{X}+\mathtt{Y} = 7\,\Omega$. We have voluntary defined the parameters after the resistor instances. Global parameter definitions can be put anywhere in the netlist (not in subcircuit definitions). Values of global parameters specified in the netlist can be modified at command level. For example

```
prompt> pan netlist.pan -p X 1k
```

modifies the value of the `X` parameter by setting it equal to 10^3.

2.6.2 Predefined parameters

`PAN` has some predefined global parameters, the complete list is:

- pi: the π constant.
- rho_copper_20_celsius: the resistivity of copper at 20 °C.
- electron_charge: the electron charge.
- vacuum_permittivity: the vacuum permittivity.
- vacuum_permeability: the vacuum permeability.
- boltzmann: the Boltzmann constant.
- zero_celsius: the conversion of 0 °C to K.
- alu_resolution: the resolution of the arithmetic-logic unit
- mile: the mile length expressed in meter.
- foot: the foot length expressed in meter.
- inch: the inch length expressed in meter.
- maxexp: the maximum allowed argument of the exponential function that does not cause overflow.
- minexp: the minimum allowed argument of the exponential function that does not cause underflow.
- temperature: simulation temperature in °C.

- temperatureK: simulation temperature in K.
- gmin: parasitic conductance employed in non-linear devices to help
- gground: parasitic conductance that connects circuit nodes to ground. It is added to help convergence. It is swept by the gground-stepping continuation method of DC analysis and time domain analyses.
- srclevel: magnitude of sources set by the source-stepping continuation method of the DC analysis.

2.6.3 Funtion parameters

The user can define parameters that are functions. For example the statement

```
parameters fnc(x)=(x+2*x) gf(x,y)=(x/(3*y))
```

defines the `fnc(x)` and `gf(x,y)` functions those arguments are "x" and "(x,y)", respectively. When used in other expressions the arguments of the calling instance are substituted in the parameter function expression. For examaple consider the instance

```
R1 x y resistor r=fnc(R1)
```

The `R1` resistor has resistance `r=fnc(R1)`. The "R1" argument is substituted to the "x" argument in the `fnc(x)` function. Thus it is just as we have replaced the `R1` instance with its

```
R1 x y resistor r=(R1+2*R1)
```

equivalent.

2.7 Functions

The functions listed in Table 2.1 can be used in expressions.

- The delay(x,d) function can be used in expression describing the characteristic of bevahioral elements. The 'x' argument is an expression containing circuit voltages and/or currents; this waveform is replicated, i.e., returned, by the 'delay' function delayed in time by the 'd' value.
- The int(x) function converts the x double precision number to its corresponding integer representation.
- The sign(x) function returns the sign of its x argument, i.e. $+1$ if $x \geq 0$ and -1 if $x < 0$.
- The table($"m", p, x_1, y_1, x_2, y_2, \ldots, x_n, y_n$), table($"m", p, (x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)$), table($"m", x,$ *"filename"*) and table($"m", p,$ *"cx", "cy"*) functions implement a look-up table in three different ways.
  The look-up table is characterised by an input array that we refer to as x and an output array that we refer to as y. The x and y arrays have an equal number of entries; the minimum allowed number is 2. The entries of the x array must be strictly monotonically increasing. The p input argument is compared with the entries stored in x and two $l < h$ indices are found, if any, such that $x_l \leq p \leq x_h$. The z output of the table is computed as

$$z = \frac{y_h - y_l}{x_h - x_l}(p - x_l) + y_l$$

  The *"m"* argument is a string composed of a single character that determines how the table is visited. The allowed characters are: *"c"* and *"e"*. If the *"c"* character is specified and $p > x_M$, where x_M is the last entry of x, or $p < x_1$, where x_1 is the first entry of x, the output of the table is set at y_M or y_1, respectively, where y_1 is the first entry of y and y_M the

ln	$z = \log(x)$	db20	$z = 20\log_{10}(x)$
dmin	synonymous of 'min'	min	$z = \min(x,y)$
max	$z = \max(x,y)$	sin	$z = \sin(x)$
cos	$z = \cos(x)$	tan	$z = \tan(x)$
acos	$z = \text{acos}(x)$	asin	$z = \text{asin}(x)$
exp	$z = e^x$	log	$z = \log(x)$
log10	$z = \log_{10}(x)$	abs	$z = \lvert x \rvert$
int	$z = \text{int}(x)$	pwr	$z = \lvert x \rvert^y$
pwrs	$z = \text{sign}(x)\,\lvert x \rvert^y$	pow	$z = x^y$
db10	$z = 10\log_{10}(x)$	sqrt	$z = \sqrt{x}$
atan	$z = \text{atan}(x)$	arctan	synonymous of 'atan'
sinh	$z = \sinh(x)$	cosh	$z = \cosh(x)$
asinh	$z = \text{asinh}(x)$	acosh	$z = \text{acosh}(x)$
sign	$z = \text{sign}(x)$	sgn	synonymous of 'sign'
tanh		atan2	
agauss		dtrunc	
trunc		ceil	
floor		error	
rand		sprintf	
randn	$z = \text{randn}()$		
limit		atoi	
atof		ddt	$z = \dfrac{dx}{dt}$
idt	$z = \int x(\tau)d\tau$	dbm	$z = 30 + 20\log_{10}(x)$
table		delay	$z = \text{delay}(x,d)$
tolower		u	synonymous of 'ustep'
ustep	$z = \text{ustep}(x)$	uramp	$z = \text{uramp}(x)$

Table 2.1: function list

last one. In this way the content of the table is "continued" above and below its extremes. If the *"e"* character is specified any time $p > x_M$ or $p < x_1$ an error is issued. Note that this may have a relevant impact, since any time $p > x_M$ or $p < x_1$ for example during the Newton iterations performed to compute the DC solution of a circuit, an error is issued.
The entries of the x and y arrays can be given in three different ways as specified by the different syntax of the table function. In table(*"m"*, $p, x_1, y_1, x_2, y_2, \ldots, p_n, y_n$) the entries are given as arguments to the table function. Note that they form pairs of x and y entries, i.e., (x_1, y_1), (x_2, y_2) and so on. In table(*"m"*,*x*, *"filename"*), the pairs are stored in the *"filename"*. There must be a pair of x and y entry per row. In table(*"m"*, *p*, *"cx"*, *"cy"*), the *"cx"* name specifies the column vector defined in a `control` analysis, whose entries form the x vector of the table. Similarly the *"cy"* name specifies the column vector defined in a `control` analysis, whose entries form the y vector of the table. Obviously these two vectors must have the same number of entries.

- The uramp(x) returns 0 if $x \leq 0$ and returns x, i.e. its argument, value if $x > 0$.
- The ustep(x) implements the step function; it returns 1 if $x > 0$ and 0 if $x \leq 0$.
- The randn() returns a random number, the mean of these random numbers is null and variance is 1. Their distribution is Gaussian. Note that, if randn() is used in different expresions, the *same* random number generator generator is in these expresions, thus the meand and variance must be interpreted in the correct way.

- The pow(), pwr() and pwrs() functions are implemented to easy the computation of the derivative of the z result with respect to the y exponent. Consider $z = x^y$, it can be recast $z = xw$ where $w = x^{y-1}$, the $\frac{dz}{dx}$ derivative is thus $\frac{dz}{dx} = yw$. This avoids to call twice the $pow()$ system function. The possible errors displayed by the pow(), pwr() and pwrs() functions have to be interpreted according to their reported implementation.

2.8 Output format

In general PAN will output to terminal and to a log file a considerable amount of messages. These include error messages, information on the simulator current activity and, in some cases, some information about simulation results. Terminal output can be limited to error messages with option "-q", i.e.

```
prompt> pan -q netlist.pan
```

will result in a log file *netlist.log* and terminal output containing only few notices concerning simulator status and error messages. An intermediate action where some more information about the simulator running is displayed can be set with the "-m" option.
Simulation results for each analysis in *netlist.pan* are stored in a directory named *netlist.raw*. This directory contains an ASCII index file and one or more output files for each analysis. Each file is named with the unique analysis identifier specified in the netlist file.
These files are in standard spice3 binary rawfile format and can thus be viewed using several visualization tools such as the open source tools GWAVE, KJWAVE, or commercial tools such as SYNOPSYS SX or CUSTOMWAVE.
It is also possible to import rawfiles into MATLAB or OCTAVE using one of the several rawfile import filters easily available on the web.

2.9 The dc analysis

The dc analysis computes a stable or unstable equilibrium point of a given circuit. The default behaviour of the simulator is to compute **one single equilibrium point** even if the circuit admits more than one. Nevertheless the analysis also implements a method to look for more than one operating point.
Like most methods implemented in PAN the dc analysis is based on the Modified Nodal Analysis (MNA) formulation [**1084079**; **1406179**]. This means that the default behaviour of the simulator is to compute only node voltages and currents of circuit ports that are not voltage controlled, these are often called "bad branches". An introduction to MNA can be found in many classic books, see e.g. [**Desoer**], [**Vlach**].
Obviously, if we wish to explicitly compute the value of a specific current of the circuit, the simplest solution is to introduce a null independent voltage source in series with the branch whose current we are interested in.

2.9.1 A simple example

As a first example, consider the circuit in Figure 2.1, where node names are in red. The corresponding Netlist is 2.9.1

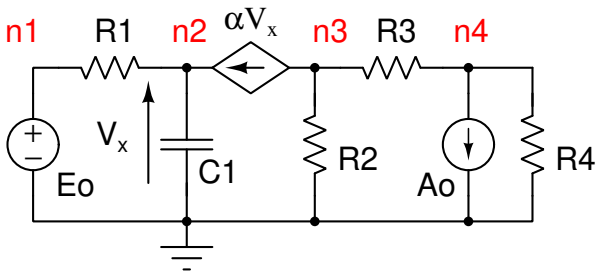

Figure 2.1: A simple example.

Netlist 2.9.1 — Simple netlist. – File: `Simple_001.net` –

```
ground electrical gnd
parameters R1=1k R2=2k R3=100 alpha=0.2
; Analysis
Dc0 dc print=yes
; Circuit
r1  n1   n2      resistor  r=R1
c1  n2   gnd     capacitor c=1u
r2  n3   gnd     resistor  r=R2
r3  n3    n4     resistor  r=R3
r4  n4   gnd     1meg
tc1 n3    n2  n2 gnd vccs  gain1=alpha
e0  n1   gnd     vsource   vdc=2
ao  n4   gnd     isource   idc=1m
```

Each device instance in this circuit is entered in its own line and correspondence of figure to netlist should be self evident. The two-port voltage controlled current source is defined by first giving, respectively, the sink and source terminals, followed by the sense nodes, each pair of sense nodes requires a corresponding `gain` parameter.
The `dc` analysis command is on a single line:

```
Dc0 dc print=yes
```

The `Dc0` is a unique identifier, more than one `dc` analysis with different options can be issued in the same file, each must have its unique name. Option `print=yes` instructs the simulator to print the dc solution that has been found to output. Default value of this parameter is `no` and, in this case, the operating point is written only in the output file. Direct terminal output can be useful, but, in general, is practical only for small circuits.
Let us now execute the simulation:

```
prompt> pan Simple_001.net
```

the output of this command consists in terminal output, a log file (that is essentially equivalent to the screen output) and an output directory named `./Simple_001.raw` that contains results of all simulations in the `PAN` file, plus an index file.
In general log files are used only for debugging purposes, since this is the first example that we are going to see let's look at the output with some detail. The logfile is shown in 2.9.2.
`PAN` reads the netlist and, after a number of messages asserting execution status, prints a circuit structure summary. In our case 4 nodes and 5 equations (node count does not consider the reference node). The equations are 5 since the `vsource` is a "bad branch", i.e. not voltage controlled and thus requires the introduction of an additional equation. The device summary section reports the number of device instances found and is quite interesting when large circuits

are simulated. After these preliminary steps, analyses are performed one at a time in the order they are encountered. In our case we have one single `dc` analysis called `Dc0`. After a few messages (about `gground` resistors, and initial guess for the Newton algorithm (the meaning of these messages will be discussed later), the `dc` simulations is performed. Total power absorbed by independent sources is always output to log and screen, current solution, i.e. the 4 node voltages and the current in the voltage source, is output only if parameter `print` is set to true. Finally some execution time statistics are reported. Note that the `R4` instance of the 2.9.1 netlist is defined through the SPICE syntax. This is automatically detected by `PAN` that gives the ‘‘`Interpreting the netlist through the SPICE syntax.`’’ in the log file.

Logfile 2.9.2 — Logfile of Simple netlist. – Simple_001.log –

```
Reading circuit from file <Simple_001.pan> ...
Notice given by pan during circuit read-in.
    Interpreting the netlist through the SPICE syntax.
Netlist successfully read ...
Building global model declarations ...
Building instance and node declarations ...
Mapping node names ...
Building analysis and option declarations ...
Checking for undefined nets ...

Node and Branch Summary:
      4 Nodes

      5 Total equations

Device Summary:
      1 capacitor
      1 isource
      4 resistor
      1 vccs
      1 vsource

Reference node is <gnd>.

**********************
* Dc analysis: <Dc0>. *
**********************
Checking circuit topology for DC analysis...
Installing <gground> resistors ...
Finding the initial guess for the Newton algorithm ...
Start of DC analysis ...
..

Total power dissipated by independent sources is <2.1000 mW>.
Analysis <Dc0> successfully terminated in <2> iterations.
Current solution of <Dc0> DC analysis:

  V(n1)   =  2.0000  V
  V(n2)   = -10.050 mV
  V(n3)   =  2.0201  V
  V(n4)   =  1.9201  V
  I(v1:i) = -2.0101 mA

CPU time for this analysis was 0.00 seconds.
Elapsed time was 0.00.
Peak memory usage was  29.4 kBytes.
```

2.9.2 dc sweep

A common use of the `dc` analysis is not limited to the computation of one single operating point, but to the computation of operating points as the value of one circuit parameter is varied. This analysis mode is known as *sweep* and most simulators provide a `dc` sweep analysis. In `PAN` dc sweep can be performed through a number of different options that control the parameter or option to be swept, and the sampling of the values to be used for the sweep (e.g. linear, logarithmic,

etc.). Instance parameters (e.g. resistance value of a resistor, value of an independent source, etc.), model parameters (e.g. threshold voltage of MOS) or simulator options (e.g. temperature) may be swept.

Sweep, among other applications, can be used to plot the dc characteristic of a nonlinear component. In this case we wish to plot the characteristic of an instance of a diode described by some of its process parameters. We thus add the model line

```
model mydiode diode is=3.142f rs=3.536 n=1 xti=3 eg=1.11 \
          m=0.4186 vj=0.75 fc=0.5 bv=80 ibv=24.573m imax=100
```

where the model of a device of type diode and named mydiode is created. A quick reference to the meaning of diode parameters in the line above is given by command pan -s diode. All these parameters are compatible with those found in standard commercial component libraries. Now mydiode can be used as any other diode component. More than one diode model can be specified in each netlist file.

Netlist 2.9.3 — Simple netlist. – File: Simple_002.net –

```
ground electrical 0
; Analysis
Dc0 dc instance=v1 param="vdc" start=-85.0 step=0.01 stop=10.0
; Circuit
v1  n1  0      vsource   vdc=2
d1  n1  0      mydiode
; Model
model mydiode diode is=3.142f rs=3.536 n=1 xti=3 eg=1.11 \
    m=0.4186 vj=0.75 fc=0.5 bv=80 ibv=24.573m imax=100
```

The Dc0 analysis card instructs the simulator to perform a (large) number of independent dc analysis, sweeping parameter "vdc" of instance v1 from -85.0V to 10.0V with steps of 100mV. The resulting output (imported and plotted using Matlab) is shown in Fig. 2.2.

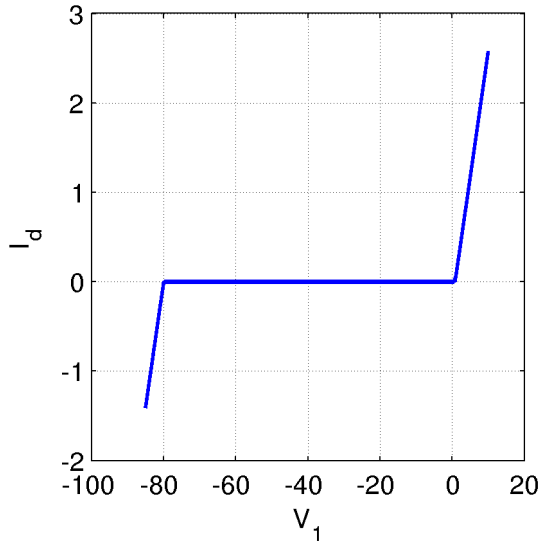

Figure 2.2: Diode current vs. generator voltage, generated by Netlist 2.9.3.

2.9.3 The `ic` parameter of the DC analysis

Consider the circuit in Figure 2.3 and the Netlist 2.9.4 modeling it.

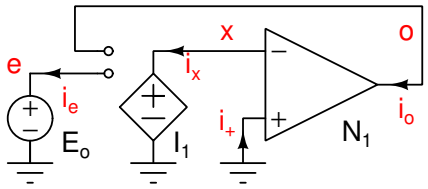

Figure 2.3: The schematic of a simple circuit that causes convergence problems to the DC analysis.

Netlist 2.9.4 — The netlist modeling the simple circuit. – File: `fixsm.net` –

```
ground electrical gnd
Dc dc print=1 fixsm=1
N1   x    gnd    o   gnd   nullor
I1   x    gnd    o   e   vcvs func=v(o)^3 + v(e)
Eo   e    gnd    vsource vdc=0.5
```

Despite its simplicity it causes convergence problems to PAN (and other commercial simulators) when a DC analysis is performed. The `N1` operational amplifier is modeled by a nullor, this means that $i_1 = 0$ and $v_x = 0$. The `I1` nonlinear voltage controlled voltage generator is driven by the v_o and v_e voltages respectively, more precisely $v_x = v_o^3 + v_e$. The `I1` nonlinear voltage controlled voltage generator is described by the

```
I1   x    gnd    o   e   vcvs func=v(o)^3 + v(e)
```

instance. Since $v_x = 0$ due to the characteristic of the nullor, we must have $v_o = (-0.5)^{1/3}$. When the `Dc` DC analysis is performed, PAN prints the message

```
Fatal error  detected during DC analysis "Dc".
    Matrix is singular.
    Singularity is at row <5> and column <2> corresponding to equations <N1:i>
    and <o> respectively. Singularity occured during the computation of the
    initial guess of the Newton method.
```

and the analysis fails, not converging to a solution. To understand why this happens, we write the expression of the n-th iteration of the Newton method used by PAN to compute the solution

$$\begin{bmatrix} v_e^{n+1} \\ v_o^{n+1} \\ v_x^{n+1} \\ i_e^{n+1} \\ i_x^{n+1} \\ i_o^{n+1} \end{bmatrix} = \begin{bmatrix} v_e^{n} \\ v_o^{n} \\ v_x^{n} \\ i_e^{n} \\ i_x^{n} \\ i_o^{n} \end{bmatrix} - \overbrace{\begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ -1 & -3\left(v_o^n\right)^2 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}}^{\mathbf{J^n}} \begin{bmatrix} i_e^n \\ i_o^n \\ i_x^n \\ v_e^n - E_o \\ v_x^n - v_e^n - \left(v_o^n\right)^3 \\ v_x^n \end{bmatrix}$$

The Newton method starts from null initial conditions. This implies that $v_o^0 = 0$ and the $\mathbf{J}^0$ Jacobian matrix is singular (the entries of the second column are null).

A way to help PAN to determine a solution is to suggest a suitable initial condition such that $v_o^0 \neq 0$. This can be done through the statement

```
Dc dc print=yes ic=["o",10m]
```

The ic parameter is a list of pairs each composed of a string and a real value. The string identifies the symbolic name of a circuit equation (or bad branch) and the value *suggests* its initial guess to the Newton method. By using this feature PAN finds the correct solution $v_o = (-0.5)^{1/3}$.

The use of the ic parameter may be not straightforward. In fact, if the circuit shown in Figure 2.3 were a sub-circuit of a more complex one, i.e., it represented a small portion of a large and complex circuit, it would be very difficult to isolate it as the problematic sub-circuit, to understand what happens and to suggest a suitable initial guess.

2.9.4 The fixsm parameter of the DC analysis

PAN implements the fixsm parameter. When set "true", the simulator automatically inserts parasitic elements that try to solve the singularity problems of the Jacobian matrix of the Newton method. The statement that does this is

```
Dc dc print=yes fixsm=yes
```

By considering the circuit shown in Figure 2.3 and the error message printed by PAN and reported in Section 2.9.3, a linear controlled current source, that injects current into the N1:i node of the circuit and that is driven by the voltage at node o, is automatically inserted by the simulator to try to solve the problem. The transconductance value of this linear voltage controlled current source is $g_{\min}$. By using the fixsm parameter, PAN finds a correct solution.

2.10 Directives

PAN supports directives in a way similar to the C language. Directives are defined through the statement

```
#define name
```

Directive can be used to conditionally execute some sections of the netlist. For example the commands

```
#ifdef boolean_expression
Body_1
#else
Body_2
#endif
```

tells the simulator to consider "Body_1" if the *boolean_expression* is true else to consider "Body_2". The #else statement and obviously "Body_2" can be omitted. The *boolean_expression* is composed of directive names, if a name was previously defined it is considered "true" otherwise it is considered "false". The PAN directive is automatically defined by the simulator when it runs standalone. Note that PAN can run as a function of MATLAB; this will be better introduced in the Section 4.1 and in this case the PAN directive is not defined.
A more complex example is

```
#ifdef ! exist( "Dc1.dc") || FORCE
Dc1 dc print=yes save="Dc1.dc"
#else
Dc2 dc print=yes iabstol=1n load="Dc1.dc"
#endif
```

where the Dc1 analysis is executed if the "Dc1.dc" file does not exist or if the FORCE directive was given. If the Dc1 analysis is executed it saves results in the "Dc1.dc" file. Thus at

second run the Dc2 analysis will be executed. A way to force execution of Dc1 even though the "Dc1.dc" file exists is to define FORCE. This can be done at command line

```
prompt> pan netlist.pan -ad FORCE
```

The -ad options means "analog define" and is equivalent to #define FORCE.
Linux commands can be executed though the statement

```
#system "command"
```

where "command" is a string.
PAN search files according to a "path" mechanism. First it looks for the file where the netlist resides then in the current directory. The statement

```
#path "path1 path2 pathN"
```

allows the definition of the search paths for files. Paths can be added also through the "-I" command line options

```
prompt> pan netlist.pan -I pathC
```

where pathC is the new path.
The statements

```
#caseon
#caseoff
```

turn on and off respectively the case-sensitivity of PAN. These statements are effective only if the "-i" command line option of PAN was not given. When PAN is run with the command line

```
prompt> pan netlist.pan -i
```

it becomes case insensitive during its entire run.

2.11 The alter pseudo-analysis

Even if alter is listed among the analysis and follows the same syntax, it may be actually considered as a directive, instructing the simulator to modify a parameter or option value. Consider once more the diode dc sweep example, but assume we are interested in comparing its characteristic at different temperatures. A simple way to do this is shown in Netlist 2.11.1.

Netlist 2.11.1 — Diode dc sweep with alter statements. – File: Simple_003.net –

```
ground electrical 0
; Analyses
Dc0 dc print=yes instance=v1 param="vdc" start=-85 step=0.01 stop=10
Alter1 alter opt=yes param="temp" value=-10
Dc1 dc print=yes instance=v1 param="vdc" start=-85 step=0.01 stop=10
Alter2 alter opt=yes param="temp" value=120
Dc2 dc print=yes instance=v1 param="vdc" start=-85 step=0.01 stop=10
; Circuit
v1  n1  0      vsource   vdc=2
d1  n1  0      mydiode
; Models
model mydiode diode is=3.142f rs=3.536 n=1 xti=3 eg=1.11 \
m=0.4186 vj=0.75 fc=0.5 bv=80 ibv=24.573m imax=100
```

Three subsequent dc analysis are performed at three different temperatures: the first at nominal temperature (i.e. 27 °C), the second at $T = -10$ °C and the third at $T = 120$ °C. Results, zoomed in to show the right knee area, are shown in Fig. 2.4. The alter statement in netlist 2.11.1 modifies the value of a global simulation option:

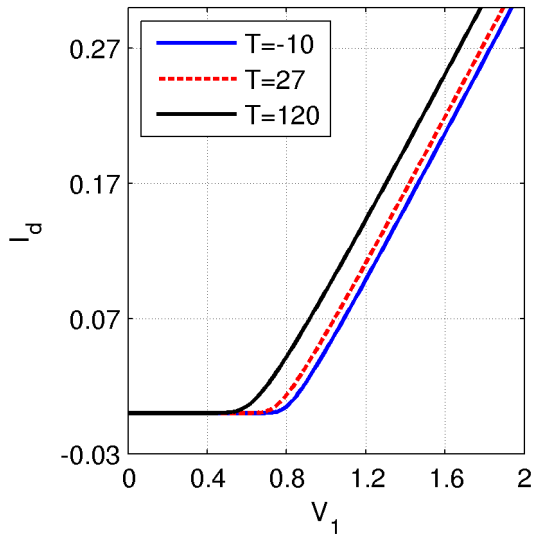

Figure 2.4: Diode current vs. generator voltage, at three different temperatures, generated by Netlist 2.11.1.

```
Alter1 alter opt=yes param="temp" value=-10
```

Device and model parameters are modified using a similar syntax; to modify resistor instance `r1` a possible `alter` card is shown below:

```
⋮

r1  n1  n2  resistor    r=1k

⋮

Alter_r1 alter instance=r1 param="r" value=10k

⋮
```

The `alter` analysis can be used also to modify the value of circuit parameters, i.e. those defined inside the netlist for example by means of the

```
parameters X=2 Y=2*X+1
```

statement. Assume that we want to change the value of the `X` parameter from 2 to 3, in this case the alter statement is

```
AlP alter param="X" value=3
```

Note that since the value of the `Y` parameter depends on that of `X`, also the value of `Y` is updated. This means that the dependency tree of the algebraic expressions involving parameters are stored by `PAN` and when the value of a parameter is changed all the expressions of depending parameters are *re-evaluated*. If in the circuit there are instances of elements whose arguments are expressions containing parameters, also these expressions are re-evaluated and the values of the instance arguments are updated. For example consider the instance

```
R1 pn nn resistor r=2+Y
```

the value of the `r` argument of the `R1` resistor instance is updated to $9\,\Omega$ since `Y` $= 7$ after having set `X` $= 3$.
Updating of an argument value does not apply to analyses, that is, consider the analysis statement

```
Dc dc vmax=X
```

before altering, the value of `X` was 2 and thus `vmax=2`. After altering, the new value of `X` is 3 but still we have `vmax=2`, since the argument of analyses, that in our specific case is `Dc`, is not updated.
The `alter` can be used inside the body of `control` analyses. During the parsing phase of the netlist `PAN` checks for `alter` statements and determines if the `alter` modifies a parameter. When this check succeeds the parse tree of all the dependent expressions are store for future possible reevaluations. The user can explicitly declare that the parse tree of dependent expressions must be stored through the parameter `rt` of the `alter` analysis. A use example is

```
AlP alter param="X" rt=true
```

where it is declared that the `"X"` parameter will be altered in a subsequent alter statement and that the parse trees of dependent expressions must be kept for subsequent reevaluations. This is mandatory if the `alter` analysis is invoked by the MATLAB interface (see Section 4). In this case during the parsing phase of the netlist `PAN` can not determine that a parameter will later be altered by the MATLAB interface. The user has thus to explicitly set this through the `rt` parameter of an `alter` as the above one, inserted in the netlist.
Note that parameters can be used also in *verilog* modules. The value of these parameters are determined during the compilation of the verilog modules when `PAN` reads the netlists and builds the analog/digital circuit. If these parameters are subsequently altered by the `alter` analysis, the values of these previously compiled parameters in the verilog modules are not modified.

2.12 The sweep pseudo-analysis

This pseudo-analysis sweeps parameters of a model, an instance or a simulator option and executes the actions specified in the sweep body, i.e. between the `begin` and `end` keywords. Sweeps can be applied also to parameters defined through the `parameter` keyword in the circuit netlist. More that one sweep can be defined in the same analysis card. Here we anticipate that the statements of the sweep analysis body are those of another peculiar pseudo-analysis, i.e., the `control` analysis, which is better described in Section 2.19.
For example consider the card and sweep body

```
Sweep sweep start=0 stop=1 step=0.1 instance=r1 mioparam="r" begin
    Tran tran tstop=1m
    Shoot shooting autonomous=1 fund=1k restart=no
end
```

A sweep named `Sweep` acts on the `r` parameter of the `r1` instance. The parameter value is linearly swept in the $[0, 1]$ interval by steps of 0.1 value. At each discrete value of the sweep the `Tran` transient analysis is executed till the `stop=1m` time stop. After the `Tran` analysis the `Shoot` shooting analysis is executed starting from the results computed by the previous `Tran` analysis (`restart=no`). In this example the sweep acts on an instance parameter. Consider the example

```
parameters p1=0 p2=0
Sweep sweep start=0 stop=1 step=0.1 param="p1"
+           start=0 stop=1 step=0.1 param="p2" begin
    Tran tran tstop=1m
    Shoot shooting autonomous=1 fund=1k restart=no
end
```

the p1 and p2 parameters (defined through the parameters keyword) are varied and the same analyses described above are executed for each value assumed by each parameter. Note that this sweep is "two-dimensional", i.e. the sweep generates a square grid of samples. There is no upper limit to the number of sweeps in the same sweep card. As last example consider

```
Sweep sweep start=0 stop=1 step=0.1 variable="x" begin
    printf("X: %e\n", x);
    Alter alter instance=r1 param="r" value=x
    Tran tran tstop=1m
    Shoot shooting autonomous=1 fund=1k restart=no
end
```

In this sweep the sweeping variable is x and it can be accessed in the sweep body. In this example, the x variable is printed and used to alter the value of the r1 resistor.

2.13 The time domain analysis tran

The tran analysis computes the dynamic evolution in the time domain of a circuit. Since it is based on MNA, it actually computes the time evolution of all node voltages and currents in "bad-branches".
When the annotate parameter of the tran analysis is suitably set, run statistics are displayed at the end of the analysis. The meaning of the listed items is:

- 'Newton cpu time': the (total) CPU time spent by the Newton method to find a solution at each time point of the analysis.
- 'Solve cpu time': the (total) CPU time spent by the Newton to LU-factorise the MNA matrix and to solve the linear problem originated at each iteration of the Newton method. It is a fraction of the 'Newton cpu time'.
- 'Load cpu time': the (total) CPU time spent by the Newton to load the MNA matrix and the RHS vector of the linear problem originated at each iteration of the Newton method. It is a fraction of the 'Newton cpu time'.
- 'Accepted time pts': the number of accepted time point, i.e., those where the Newton method converged to a solution.
- 'Total Newton iters': the total number of Newton iterations. It is greater than 'Accepted time pts' since several iterations are in general needed to get convergence at a time point.
- 'Rejected Newton iters': the total number of rejected Newton iterations, i.e., the Newton method iterated an possibly converged at a time point of the tran analysis but the solution was rejected.
- 'Rejected lte time pts': the Newton method converged to a solution but it did not satisfy the accuracy set by the local truncation error and thus it was recomputed with a different integration time step length.
- 'Load Device Count': how many time devices were loaded, i.e., how may time the MNA matrix and/or the RHS were loaded.
- 'Matrix element number': the total number of entries of the MNA matrix.

- 'Sparsity percentage': the percentage of sparsity of the MNA matrix.

2.13.1 The tmax parameter

The tmax parameter sets the maximum allowed time step length. This parameter has meaning only when PAN uses a variable time step integration schema. PAN varies the length on the integration step but not above the tmax value. tmax can be a list with format '[ts,tm,te,...]', i.e., the user specifies triplets, where 'ts' is the starting time instant at which the 'tm' maximum integration time step is applied till the ending 'te' time instant. Outside each 'ts'-'te' time interval the default maximum integration time step is used. The default value is automatically determined by PAN. If another tmax parameter is given as a number in addition to a triplet list, this single value is used in the time intervals that are not covered in the list.

2.13.2 A Van der Pol oscillator, parameters, expressions, and the poly device

Consider schematic of the Van der Pol oscillator shown in Fig. 2.5

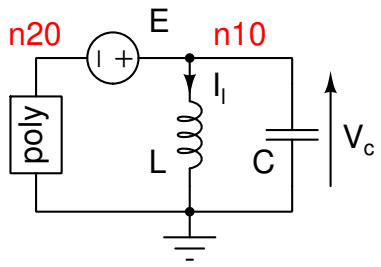

Figure 2.5: The schematic of the Van der Pol oscillator.

Netlist 2.13.1 — The Van der Pol oscillator. – File: vanderpolPoly.net –

```
ground electrical 0
parameters R=10 ALPHA=1/3
; Analyses
Tran0 tran tstop=100 uic=1
Tran1 tran tstop=120 restart=no
; Circuit
r1    n20     0  n20   0   poly n0=0 n1=-1/R n2=0 n3=ALPHA/R d0=1
vr1   n10   n20  vsource   vdc=0
c1    n10     0  capacitor c=1 icon=3
l1    n10     0  inductor  l=1
```

The netlist implementing the oscillator is shown in Netlist 2.13.1. The reference terminal of MNA is 0. There are the definitions of two parameters that are used in the polynomial expression of the nonlinear resistive device.

The nonlinear transconductance device poly

PAN also allows to define rational trans-conductances. These are defined using the poly built-in device. It is a nonlinear voltage controlled current source two ports. The first port, i.e. the first two node names, represents the current source, while the second pair the "sense" nodes. In our case, since we want to realize a simple nonlinear one port, the two ports are connected to the same circuit nodes. The coefficients of the numerator and denominator of the rational function implementing the electrical characteristic are given through the n0, n1 ... n8 and d0, d1 ... d8 parameters respectively. This means that polynomials up to degree 8 can be defined.

Not specifying any coefficient, the denominator is equivalent to `d0=1`, on the other hand, not specifying any coefficient, the numerator results in a null polynomial. In our case the netlist card

```
r1 n20 0 n20 0 poly n0=0 n1=-1/R n2=0 n3=ALPHA/R d0=1
```

implements the $i = \frac{\alpha}{R}v^3 - \frac{1}{R}v$ function.

Voltage sources as current measure devices

An independent voltage source of null magnitude is connected in series to the nonlinear element, this forces MNA to explicitly add the current through the generator as a variable providing its value in the output data file.

Initial Conditions and simulation of the Van der Pol oscillator

Let us finally consider the *first* analysis card (we will consider the second one later)

```
Tran0 tran tstop=100 uic=1
```

As always, the first word `Tran0` represent a unique name for the analysis while the second is the analysis type, in our case the keyword `tran`, asserting that this is a *transient* aka *time domain* analysis. Option `tstop` is mandatory, it represents the time at which simulation must stop. In our case, since this is a rather "slow" oscillator, the large `tstop = 100` value is used.
The second option, `uic=1`, stands for "use mode 1 for Initial Conditions (ICs)". Three different IC modes are allowed:

`uic=0` is equivalent to "no ICs", i.e. perform a `dc` analysis to set ICs,

`uic=1` use ICs given for all capacitors and inductors and set to 0 those that are not defined (i.e. 0 voltage on capacitors and 0 current in inductors)

`uic=2` use ICs given for all capacitors and inductors and assume that all capacitors and inductors whose initial conditions have not been specified are initially considered equivalent to open and short circuits respectively.

Note that internal capacitors of devices such as for example MOS, BJT, JFET can not be initialised can not be initialised; thus independently from the value of the `uic` parameter, internal capacitors are always considered as open circuits and inductors as short circuits.
Initial conditions are set on each capacitor and inductor line with option `icon=`*`value`*. In our case the IC has been set for the capacitor, while the inductor has been left unspecified; since mode 1 is set, this means that the initial current value in the inductor is set to 0.
After simulation (the command is `pan vanderpol.pan`, from now on we will specify the simulation command only when non-standard), we find the folder `vanderpol.raw` containing the usual index file and files `Tran0` and `Tran1` containing simulation results.
Plotting the inductor current versus the capacitor voltage yields the trajectory shown in Fig. 2.13.2.

Restarting from previous results

As desired simulation starts from initial conditions set at $V_c = 3V$ and $I_L = 0$. It is sometimes useful to start a time domain simulation from the last point of a previous simulation. This can be done using option `restart`, as in the second analysis line,

```
Tran1 tran tstop=120 restart=no,
```

whose default value is `yes`. Setting `restart=no` instructs the simulator to perform this time domain simulation using as initial point the last point computed in the immediately previous for example `tran`, `shooting` or `envelope` simulation. Note that, when restarting from a previous

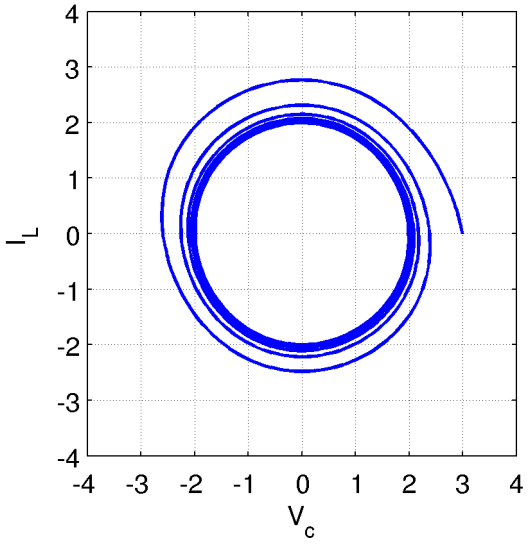

Figure 2.6: Inductor current vs. capacitor voltage for the Van der Pol oscillator (Netlist 2.13.1).

simulation, option `tstop` must take into account the stop time of the previous simulation, in our case setting `tstop = 120` means that, since `Tran1` starts from time `t=100`, total simulation time is 20.

Solutions, i.e. "last points" of time domain analysis can also be saved and loaded using commands `load <filename>` and `save <filename>` respectively. Results are shown in Fig. 2.7, as it can be seen, since the previous analysis ends up very close to the oscillator "steady state" trajectory the circuit limit cycle is evidenced.

2.13.3 The Van der Pol oscillator with a behavioural nonlinear element

`PAN` implements behavioural controlled devices and n-ports. The polynomial resistor can be easily implemented through a behavioural voltage controlled current generator. The nelist of this version of the Van der Pol oscillator is reported in Netlist 2.13.2

Netlist 2.13.2 — The behavioural Van der Pol oscillator. – File: `vanderpolBhv.net` –

```
ground electrical 0
parameters R=10 ALPHA=1/3
; Analyses
Tran0 tran tstop=100 uic=1
Tran1 tran tstop=120 restart=no
; Circuit
r1    n20     0  vccs func=1/R*(ALPHA*v(n20)^3-v(n20))
vr1   n10   n20  vsource   vdc=0
c1    n10     0  capacitor c=1 icon=3
l1    n10     0  inductor  l=1
```

The `poly` element is substituted by the `vccs` controlled source and its behavioural characteristic is specified through the `func` parameter. In the algebraic expression the voltage of the terminals of the `vccs` can be used as arguments of the `v(pos,neg)` functions. If the `neg` terminal is omitted the `v(pos)` voltage is referred to the reference node.

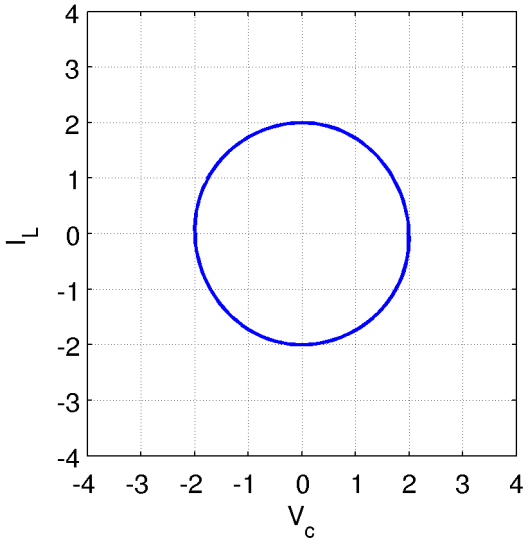

Figure 2.7: Inductor current vs. capacitor voltage for the Van der Pol oscillator (Netlist 2.13.1).

2.13.4 The veriloga version of the Van der Pol oscillator

PAN implements the VERILOGA module compiler. It generates a shared library that is automatically linked by the simulator at run-time. The execution of the VERILOGA code is thus adequately efficient even though the generated and compiled code is not optimised as in the built-in models. Therefore the behavioural element of the Van der Pol oscillator can be implemented as a `nport` through the Verilogа language. The Netlist 2.13.3 is an implementation example.

Netlist 2.13.3 — The veriloga Van der Pol oscillator. – File: `vanderpolVa.net` –

```
ground electrical 0
parameters R=10 ALPHA=1/3
; Analyses
Tran0 tran tstop=100 uic=1
Tran1 tran tstop=120 restart=no
; Circuit
r1    n20     0  POLY R=R ALPHA=ALPHA
vr1   n10   n20  vsource   vdc=0
c1    n10     0  capacitor c=1 icon=3
l1    n10     0  inductor  l=1
model POLY nport veriloga="poly.va" verilogamodname="POLY_VA"
```

The nonlinear polynomial element is of type `POLY`; its model is a `nport` described in the `poly.va` file. The name of the Verilogа module in the `poly.va` file that implements the polynomial characteristic is `POLY_VA`. The VERILOGA code that implements the module is shown in Netlist 2.13.4 [**VeriaRef**].

Netlist 2.13.4 — The VERILOGA code. – File: poly.va –

```
module POLY_VA(pos,neg);
inout pos, neg;
electrical pos, neg;
parameter real R = 10;
parameter real ALPHA = 1/3;
analog begin
  i(pos,neg) <+ (ALPHA*pow(v(pos,neg),3.0) - v(pos,neg))/R;
end
endmodule
```

Note that PAN implements the ELECTRICAL discipline as default, thus the file that describes disciplines to the VERILOGA compiler can be omitted. Moreover default parameters listed in Section 2.6.2 can be directly used in VERILOGA modules.
The pos and neg terminals of the POLY_VA module are the input/output pins. The current flowing from pos to neg is assigned in the module by the algebraic expression. Currents and voltages can be accessed through the v() and i() functions whose arguments must be defined as electrical. The

```
r1 n20 0 POLY R=R ALPHA=ALPHA
```

card passes the values of the R and ALPHA parameters to the corresponding parameters in the VERILOGA module access functions. This netlist can be simulated through the command

```
prompt> pan vanderpolVa.net -p R 1k
```

that sets the value of the R parameter at 1 kΩ.

2.13.5 Time domain noise analysis

PAN performs time domain noise analyses. Each noisy device such as for example a resistor, generates, thermal, shot and flicker noise, this in accordance to the noise model of each element. A time domain realisation of these noise sources are built for each time domain analysis (tran analysis) and noise samples are injected in the circuit. There are two types of noise injections: large signal and small signal. The former superimpose to the large signal solution and may be affected by the *numerical noise* introduced by the solving algorthms of the simulator. The latter is injecnted in the equivalent variational model of the circuit that is derived at each time point of the time domain analysis. This injection suffers of much less numerical noise and thus allows to compute contributions that normally fall below the noise floor of the time domain noise-less analysis. To give a use example, we refer once more to the van der Pol oscillator in Figure 2.5. The only noisy element of this cuircuit is the non-linear polynomial resistor. The netlist for the time domain noise analysis is shown in Netlist 2.13.5.

Netlist 2.13.5 — Time domain noise analysis. – File: vanderpolTranNoise.net –

```
ground electrical 0
parameters R=10 ALPHA=1/3 NFS=1k
; Analyses
TranI tran tstop=100 uic=1
TranN tran tstop=1000 noisefmax=NFS restart=0
Sh shooting autonomous=1 restart=no period=2*pi
Pn pnoise start=1m stop=10 dec=50 sweeptype=0 onodes=["n20"]
Ns control begin
    V20 = get("TranN.n20:noise");
    [f,x] = mtpsd(V20,null,null,2*NFS,null,10u);
    save("raw","psd","freq",f,"psd",x);
endcontrol
; Circuit
r1    n20     0  n20   0   poly n0=0 n1=-1/R n2=0 n3=ALPHA/R d0=1
vr1   n10   n20  vsource   vdc=0
c1    n10     0  capacitor c=1 icon=3
l1    n10     0  inductor  l=1
```

The target of the

```
TranI tran tstop=100 uic=1
```

statement is to bring the oscillator to work at steady state. This is not the better way to have the oscillator working in a steady state condition. The `shooting` analysis does this much better. Anyway the

```
tran tstop=1000 noisefmax=NFS restart=0
```

analysis starts from the last time point of `TranI` and performs a time domain noise analysis for a sufficiently long time interval (1000s). The time domain noise analysis is activated by the `noisefmax=NFS` argument, where `2*NFS` is the frequency at which the noise samples are injected. It defines the bandwidth of the noise that in our case is `NFS`, that is assumed autocorrelated. The noise band-with should be much larger that the bandwidth of the circuit. This sentence is generic but we hope it gives the right idea about the value of `noisefmax`. The vector of noisy voltage samples at the circuit node **n20** is retrived through the

```
V20 = get("TranN.n20:noise");
```

statement of the `Ns control` analysis. The noisy results can be retrieved by postponing the ":noise" suffix. The power spectral density is computed by the `mtpsd` control function. The power spectral density is saved in the `"psd"` file by the

```
save("raw","psd","freq",f,"psd",x);
```

statement. A `shooting` analysis was also performed to compute the accurate periodic solution of the oscillator and then the `Pn pnoise` analysis was performed to compute the periodic noise analysis of the oscillator at the same **n20** output node.
The spectra obtained by the realisation of the `TranN` time domain noise analysis and by the `Pn` periodic noise analysis are shown in Fig. 2.8. A good matching can be noted, better agrement and less oscillations can be obtained by performing several time domain noise analyses and averaging results.

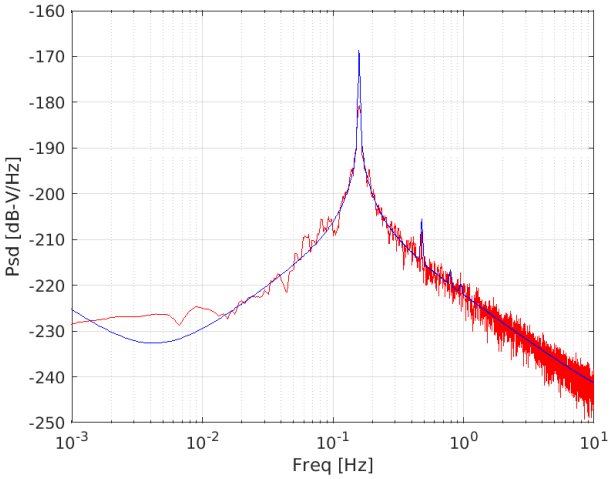

Figure 2.8: The spectra of the noise at the **n20** node of the van der Pol oscillator in Figure 2.5. `TranN` (red) and `Pn` (blue) analyses.

2.14 The envelope analysis

By referring to the schematic of the Van der Pol oscillator in Figure 2.5, if we set the `R` parameter in the Netlist 2.13.4 at 1 kΩ, this relatively high value increases the quality factor of the resonator and the oscillator takes a long time to reach a steady state working condition. The time domain analysis can be accelerated by using the `envelope` analysis. A fully description of the envelope analysis implemented in `PAN` can be found in [**855455**]. The envelope analysis is useful when the circuit has an almost periodic behaviour enveloped by a relatively slowly varying waveform. For example the $y(t) = \left(1 - e^{-\lambda t}\right)\cos(\omega t + \phi)$ functions with $\lambda \in \mathbb{R}^+$, represents an almost periodic solution of $T = {}^{2\pi}/_{\omega}$ period whose amplitude grows governed by the $\left(1 - e^{-\lambda t}\right)$ envelope function. If $T \ll \lambda$ the variation of $y(t)$ from kT to $(k+1)T$ (k integer) is very small, i.e $y(kT + t) \simeq y((k+1)T + t)$. The $\cos(\omega t + \phi)$ component is estimated by the envelope analysis and instead of computing it along a large number of periods corresponding to a small variation of $e^{-\lambda t}$, this computation is skipped and the solution at the beginning and at the end of this skipped time interval are extrapolated. In other words the full and exact solution (assuming the conventional `tran` analysis is the reference) is under-sampled and samples are single T working periods.
The netlist of the Van der Pol oscillator with also the `envelope` analysis card is shown in Netlist 2.14.1.

Netlist 2.14.1 — Envelope analysis. – File: `vanderpolEnv.net` –

```
ground electrical 0
parameters R=1k ALPHA=1/3
; Analyses
Tran0 tran tstop=100 uic=1
Tran1 tran tstop=120 restart=no
Env envelope period=2*pi autonomous=yes stop=2k restart=no
; Circuit
r1    n20     0  POLY R=R ALPHA=ALPHA
vr1   n10   n20  vsource   vdc=0
c1    n10     0  capacitor c=1 icon=3
l1    n10     0  inductor  l=1
model POLY nport veriloga="poly.va" verilogamodname="POLY_VA"
```

With respect to the Netlist 2.13.4 the new card

```
Env envelope period=2*pi autonomous=yes stop=2k restart=no
```

introduces the envelope analysis labelled as `Env` that starts from the results of `Tran1` and ends at `stop=10k` seconds. The suggested period is `period=2*pi` and it is declared that the circuit is autonomous (`autonomous=yes`). By observing Figure 2.9 we see that the waveform of voltage

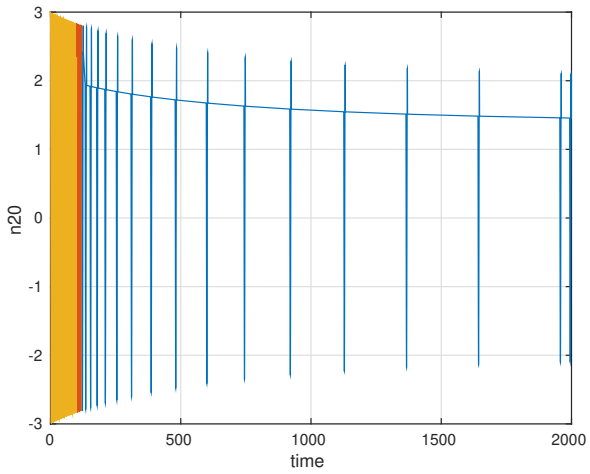

Figure 2.9: The voltage of the `n20` node computed by `Tran0` (yellow), `Tran1` (red) and `Env` (blue) analyses.

at node `n20` is composed of dicrete periods (they look as "spikes"), that is, only these discrete periods were computed. The envelope analysis assumed that in the time intervals between two adjacent computed periods the full non-computed waveform can be adequately approximated as a repetition of these computed periods. From the figure we clearly see the envelope that modulates the magnitudes of the computed periods. The time intervals among the computed periods increase with respect to simulation time (x-axis) since the envelope goes towards the steady state solution of the Van der Pol oscillator and thus the shape of the computed periods also stabilises. Acceleration by the envelope analysis consists exactly in skipping the computation of most of the working periods.

2.14.1 Parameters governing accuracy of the envelope waveform and the jump

When the envelope analysis makes a jump of length H, i.e., it skips H periods, the accuracy of the solution after the jump is estimated by using the *local truncation error* (*lte*) introduced in computing the enveloping solution. The enveloping solution is obtained by integrating the complete solution on a time mesh where samples are separated by the H jumps. Details on the *lte* can be found in [**855455**].

The parameters that govern the length of the H jump and thus the efficiency and accuracy on the envelope analysis in determining the solution are:

eabstol The absolute tolerance to check acceptability of the solution found by the envelope analysis after the H jump. the circuit.

ereltol The relative tolerance to check acceptability of the solution found by the envelope analysis after the H jump. the circuit.

etoltype This option specifies the type of check used to check if the envelope analysis has reached convergence inside a simulation window, i.e one T working period of the circuit. Checks

are done on the variation of variables computed by the time domain analysis along one time window.
Define as $\delta x_n = x_n(t_m + T) - x(t_m)$ the variation of the n-th electrical variable between the beginning and the end of one time window, i.e., at t_m and $t_m + T$ of the T working period simulated in detail, as α_r and α_a the relative and absolute maximum allowed variations o the n-th electrical variable, respectively. We assume that the n-th variable has converged if $|\delta x_n| < \alpha_r y_n + \alpha_a$. The etoltype parameter defines three different ways to compute y_n:

1 global type. y_n is defined as the maximum absolute value along all time windows, till the current one, assumed by any electrical variable, that is $y_n = \max_{k=1:N} |x_k(t)|$, where N is the number of circuit variables. Note that in this case y_n is a unique value adopted for all the δx_n variable variations to check their convergence.

2 local type. y_n is defined as the maximum absolute value of $|x_n(t)|$ for t varying in the current T time window. Note that in this case y_n is different for each electrical variable.

3 point local. $y_n = \max(|x_n(t_m + T)|, |x(t_m)|)$.

The convergence check tightens as the value of the etoltype parameter is increased from 1 to 3.

ltesc An empirical factor that scales the *lte* value in computing the H jump. Higher is this ltesc, larger is H with the same value of the estimate *lte*. Too large values can lead to inaccurate results.

hgfactor Set the growing factor of the envelope jump H. At each iteration if the *lte* is below the threshold set through the `ereltol` and `eabstol` parameters the new envelope jump H is estimated. The `hgfactor` parameter limits the relative growth of H, that is, if `hgfactor` $= 2$ the H jump can not increase with a rate higher that 2.

maxh Upper limit the maximum value that can be reached by H. For example assume that the circuit is quiescent at its steady state solution, the H jump will increase from time window to time window. When its value reaches or goes above the value of `maxh` it is upper limited.

corrector This parameter defines the type of corrector algorithm that is used to compute the envelope; it must have a value from 0.5 (pure trapezoidal method) to 1.0 (pure Implicit Euler method).

2.15 The harmonic analysis

`PAN` implements some methods to compute the steady state solution of a nonlinear circuit. Among those there is the harmonic balance method that assumes that the solution can be expressed through a Fourier series with a finite number of terms (dc component, fundamental and harmonics). The original differential-algebraic problem in the time domain is transformed in an algebraic nonlinear problem in the frequency domain. The unknowns are the complex coefficients of the Fourier series representing the components of the solution.
The Van der Pol oscillator, which we are using as example, is an autonomous nonlinear circuit that admits a periodic solution. The harmonic balance analysis is used to compute this periodic solution. The netlist of this example is

Netlist 2.15.1 — Harmonic analysis. – File: vanderpolHb.net –

```
ground electrical 0
parameters R=1k ALPHA=1/3
; Analyses
Tran0 tran tstop=100 uic=1
Tran1 tran tstop=120 restart=no
Env envelope period=2*pi autonomous=yes stop=2k restart=no \
             fft=yes fftharms=32
Probe control begin
    Pb = abs( (get("Env.n20:spectrum"))(2) );
    Hb harmonic fund=1/(2*pi) vprobemag1=Pb autonomous=4 \
                harms=32 twaves=yes
endcontrol
; Circuit
r1    n20     0  POLY R=R ALPHA=ALPHA
vr1   n10   n20  vsource   vdc=0
c1    n10     0  capacitor c=1 icon=3
l1    n10     0  inductor  l=1
model POLY nport veriloga="poly.va" verilogamodname="POLY_VA"
```

The hb harmonic analysis implements several solvers for autonomous circuits, we chose the autonomous=4 solver that implements the *voltage probe insertion* technique [**Boianapally**; **SCS09**; **brambilla2010robust**; **Duan3**]. In short the voltage probe forces the Van der Pol oscillator by setting the magnitude of the fundamental of a voltage node of the circuit. In this example the node is automatically chosen as n10 (see the circuit schematic in Figure 2.5). The magnitude of the probe is adjusted by the harmonic balance solver so that the current through the probe is null. This means that the probe *can be removed* without altering the solution of the circuit and mainly that a periodic solution of the original circuit without the probe is found.
The chosen version of the solver needs an initial tentative value of the probe magnitude. We decided that this value is given by the previous Env envelope analysis. To this end the envelope analysis computes the spectrum of the solution through the Fast Fourier transform (fft=yes) limiting the spectrum to fftharms=32 harmonics. Note that the circuit does not operate in its steady state condition at the end of the Env analysis.

The body of the control analysis supports a language that is a subset of that of MATLAB plus specific statements to execute the simulator analyses and retrieve results. To have more details of the contro analysis, see Section 2.19. The get() function used in the statement

```
Pb = abs( (get("Env.n20:spectrum"))(2) );
```

allows the user to retrieve results of analyses and values of parameters of circuit elements and models. The user must pay a lot attention since results are store in memory and thus they may allocate a large quantity of memory. In this example the Env keyword is the name of the analysis, n20 is the circuit node and the :spectrum suffix specifies that the spectrum of the voltage at node n20 is needed. In this case the get() function returns a complex vector, we take its second entry that is the Fourier coefficient of the fundamental; the DC component is stored in the first entry. The abs() function computes the modulus of the complex number and assigns it to the Pd real variable.
The statement

```
Hb harmonic fund=1/(2*pi) vprobemag1=Pd autonomous=4
            harms=32 time=yes
```

defines the `Hb harmonic` analysis that uses `harms=32` harmonics, computes also the `time=yes` time domain waveform. The `vprobemag1` initial magnitude of the probe is `Pd`.

2.16 The shooting analysis

There are many situation where the designer is interested in finding the periodic steady state solution of a circuit. While a first and naive approach could be performing a long time domain simulation, waiting for the solution to settle in a stable situation, this could lead to very long simulation times for certain classes of circuits, such as high-Q oscillators. The shooting method overcomes this problem by solving a boundary value problem by finding an initial condition x_0 such that the circuit, after one period, is back at x_0. The circuit can be *autonomous* or *non-autonomous*, simply meaning, in the second case, that the state equations of the circuit have explicit dependence on time, or, from a practical point of view, that there are independent periodic voltage or current generators in the circuit. Since we are looking for a periodic steady state, in the *non-autonomous* case the circuit is *periodically* forced with known period T; on the other hand, if the circuit is autonomous, period T must also be determined. The shooting algorithm requires computing the sensitivity of the final solution with respect to the initial conditions[1]. This is quite straightforward if all equations are continuous, but requires some work otherwise. `PAN` implements different types of analyses that efficiently determine the steady state solution of a nonlinear circuit. The shooting analysis works in the time domain and can be applied also to mixed analog-digital circuits, i.e. to circuits that are modeled by a time-continuos analog circuit and by a combinatorial and/or state machine digital circuit. We use the shooting method to compute the steady state solution of the Van der Pol oscillator shown in Figure 2.5. Furthermore we ask to compute the Floquet multipliers to see if the oscillator orbit in stable. The Netlist 2.15.1 is extended by adding the analysis card

```
Sht shooting period=2*pi autonomous=yes restart=no floquet=yes
```

that uses as initial condition a point of the time domain trajectory computed by the previous `Hb harmonic` analysis.

[1]The method is called *shooting* for this reason: it is analogous to the adjustments that a gunman makes to his initial cannon tilt and yaw (i.e. the initial guess) in order to reach the target. In our case the gunman is actually "firing" a boomerang, expecting it to come back in the same point where it was thrown.

Netlist 2.16.1 — Shooting analysis. – File: vanderpolSh.net –

```
ground electrical 0
parameters R=1k ALPHA=1/3
; Analyses
Tran0 tran tstop=100 uic=1
Tran1 tran tstop=120 restart=no
Env envelope period=2*pi autonomous=yes stop=2k restart=no \
             fft=yes fftharms=32
Probe control begin
    Pb = abs( (get("Env.n20:spectrum"))(2) );
    Hb harmonic fund=1/(2*pi) vprobemag1=Pb autonomous=4 \
                harms=32 twaves=yes
    Sh shooting restart=no autonomous=yes floquet=yes
endcontrol
; Circuit
r1    n20     0  POLY R=R ALPHA=ALPHA
vr1   n10   n20  vsource   vdc=0
c1    n10     0  capacitor c=1 icon=3
l1    n10     0  inductor  l=1
model POLY nport veriloga="poly.va" verilogamodname="POLY_VA"
```

2.16.1 The restart argument

The restart argument merits a particular attention. The shooting analysis can be started "from scratch", i.e. from an operating point of the circuit as is done for the tran analysis, from some initial condition given by the user in the netlist or by other analyses performed before the current shooting analysis. For example we can perform a tran analysis to "start-up" the circuit and then perform a shooting analysis to compute its steady state behaviour. This is exactly what happens in the Netlist 2.16.1; in the Probe control analysis, the Sh shooting analysis is started from the results obtained by the previous Hb analysis. When a shooting analysis, that we refer to as ShZ is started from the results of a previous shooting analysis, i.e., we set restart=no, that we refer to as ShA, the first time point of ShZ is equal or larger that the last time point of ShA. If several shooting analyses are subsequently started, each from the previous one, the first time point of shooting analyses monotonically increases and become larger and larger. This may cause numerical issues, particularly when the simulation time step may become very short to follow very fast variations of electrical variables. In this case the simulator can suffer of round-off errors since the very short integration time step can not be accurately summed to the current integration time. To avoid this problem the restart=2 option forces the following shooting analyses to start from the first time instant of the previous shooting analysis and not from the last one (ending period). Thus during the execution of the sequence of the shooting analyses, the time interval along which each shooting analysis is performed should not increase or increase moderately according to the circuit to be simulated and to the behaviour of the shooting analyses[2].

2.17 The pac analysis

The periodic small signal analysis (pac)

2.18 The pnoise analysis

PAN can compute the small-signal effects of cyclo-stationary noise sources such as for example the amplitude and phase noise in oscillators. The Van der Pol oscillator described by the 2.13.4

[2] It may happen that a shooting analysis simulates a circuit for more than a given or estimated working period.

VERILOGA code is noiseless, but that described in the Netlist 2.5 that uses a polynomial element is noisy since the `poly` element can be noisy. The netlist of the noisy Van der Pol oscillator with the addition of the `pnoise` periodic noise analysis in reported in Netlist 2.18.1.

Netlist 2.18.1 — Pnoise analysis. – File: `vanderpolPnoise.net` –

```
ground electrical 0
parameters R=1k ALPHA=1/3
; Analyses
Tran0 tran tstop=100 uic=1
Tran1 tran tstop=120 restart=no
Env envelope period=2*pi autonomous=yes stop=2k restart=no \
             fft=yes fftharms=32
Probe control begin
    Pb = abs( (get("Env.n20:spectrum"))(2) );
    Hb harmonic fund=1/(2*pi) vprobemag1=Pb autonomous=4 \
                harms=32 twaves=yes
    Sh shooting restart=no autonomous=yes floquet=yes fund=1/(2*pi)
    Pn pnoise start=1m stop=1 lin=1000 onodes=["n10"] sweeptype=0
endcontrol
; Circuit
r1    n20     0  n20   0   poly n0=0 n1=-1/R n2=0 n3=ALPHA/R d0=1
vr1   n10   n20  vsource   vdc=0
c1    n10     0  capacitor c=1 icon=3
l1    n10     0  inductor  l=1
```

The `pnoise` analysis can be executed only after a `shooting` analysis, since it is assumed that noise sources contribute additive noise effects, i.e. effects that can be summed to the large signal periodic solution of the circuit computed with a noiseless circuit. Better said the large signal solution is computed with all the noise sources turned off and then the noise effects are summed to the large signal solutions. The

```
Pn pnoise start=1m stop=1 lin=1000 onodes=["n10"] sweeptype=0
```

analysis computes the noise at the `n10` node by considering a noise spectrum in the $[1\,\text{mHz},\ 1\,\text{Hz}]$ output bandwidth. The analysis considers an evenly spaced mesh of frequencies composed of `lin` $= 1000$ samples.

2.19 Control analysis

The control analysis interprets a language similar to that of "Octave/Matlab". It allows to perform elaborations of the simulation results, to alter instance and model parameters and to perform analyses. In other words it gives the possibility to implement algorithms that "drive" the simulation flow. The interpreted language allows the definition of procedures in the same control body. Supported data are scalars, vectors matrices and array of matrices. Supported types are integer, real and complex.

As a usage example consider the circuit shown in Figure 2.11 that implements a simple MOS amplifier. Assume that the $i_{ds} = k\,(v_{gs} - v_T)^2$ simple characteristic of the MOS working in saturation region is adopted. We would like to determine the value of the R_s resistor such that the i_d current is equal to 1 mA. The value of circuit parameters are reported in the caption of Figure 2.11. The value of R_s can be easily found by "pen and paper". We have

$$R_s k\,(v_{gs} - v_T)^2 + v_{gs} - v_g = 0 \implies R_s k\,(v_g - v_s - v_T)^2 + v_g - v_s - v_g = 0$$

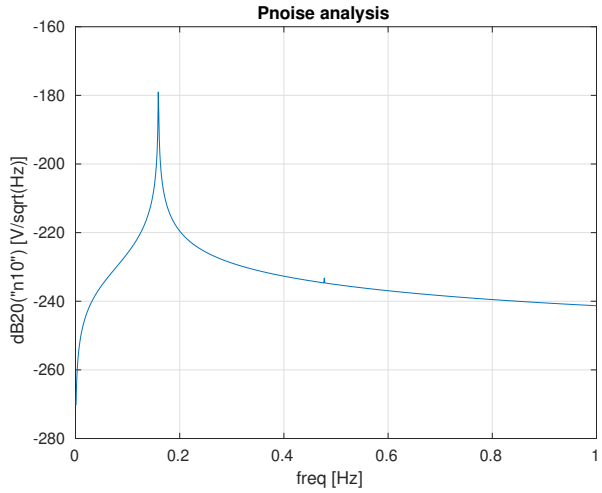

Figure 2.10: The total noise at the n10 node of the noisy Van der Pol oscillator (Netlist 2.13.1).

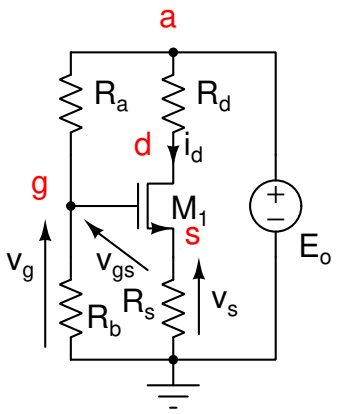

Figure 2.11: A simple MOS amplifier. $R_a = 70\,\mathrm{k\Omega}$, $R_b = 30\,\mathrm{k\Omega}$, $R_d = 4\,\mathrm{k\Omega}$, $E_o = 10\,\mathrm{V}$, $k = 1\,\mathrm{mAV}^{-2}$, $v_T = 1\,\mathrm{V}$

we know that $v_g = \dfrac{r_b}{r_a + r_b} E_o = 3\,\mathrm{V}$, thus obtaining

$$R_s k\,(3 - v_s - 1)^2 - v_s = 0 \implies (2 - v_s)^2 - v_s = 0 \implies v_s^2 - 5v_s + 4 = 0$$

The solution can be tentatively found by considering two numbers whose product equals 4 and whose sum equals 5. These two numbers are $v_s^{'} = 1$ and $v_s^{''} = 4$. The $v_s^{''}$ must be dropped since $v_{gs} = v_g - v_s^{''} < v_t$ and the MOS is off.
We would like that PAN computes the value of R_s. To this end the netlist of the circuit in Figure 2.11 is reported in Netlist 2.19.1.

Netlist 2.19.1 — Simple netlist. – File: Cntrl_001.net –

```
ground electrical gnd
parameter E0=10 RD=4k RS=100 VT=1 K=1m ID=1m

Solve control begin
  function Id=Func(x)
    Al alter instance="Rs" param="r" value=x
    Dc dc
    Id = (E0 - get("Dc.d"))/RD - ID;
  end
  x = multirootf("Func",RS);
  printf("Rs: %23.16e\n", x );
endcontrol

Eo  a   gnd  vsource  vdc=E0
Ra  a     g  resistor r=70k
Rb  g   gnd  resistor r=30k
Rd  a     d  resistor r=RD
I1  d  s  g  vccs func=(v(g,s) > VT) ? K*(v(g,s)-VT)^2 : 0
Rs  s   gnd  resistor r=RS
```

The code that controls the simulator and determines the value of the R_s resistance is the body of the `Solve` control analysis. In the bode there is the definition of the `Func` function, that has as input argument the `x` variable and that returns the `Id` variable. Inside the body the `Al` alter analysis sets the value of the `Rs` resistance at `x`. Then the `Dc` DC analysis is performed and the `Id` value is computed through the expression

```
Id = (E0 - get("Dc.d"))/RD - ID
```

Note that `Id` is the difference between the actual i_d current and the ID target value.
The `multirootf` control function finds a root of the nonlinear function `Func`, starting from the initial guess `RS`. It used an iterative approach that is introduced in Section 2.19.2. At each iteration the `Func` functions may be executed with the `x` value and at convergence the `Id` residue is reduced below a lower threshold.

2.19.1 The get() function

The `get()` control function allows to retrieve results computed by analysis, values of circuit elements and models. The syntax is `get("Key.item")` where "Key" can be an analysis name, an instance name or a model name. When "Key" is an analysis name, "item" is an electrical quantity, such as for example a node voltage or a branch current. For example `get("Dc.d")` gets the value of the electrical variable "d" computed by the "Dc" analysis. When "Key" is an instance name, "item" is a parameter of the instance. For example `get("Ra.r")` returns the value of the "r" parameter of the "Rs" instance. When "Key" is a model name, "item" is a parameter of the model. For example `get("Dio.is")` returns the value of the "is" parameter of the "Dio" model. The `get("string",index)` control function can have the `index` second parameter. If it is not supplied, the entire vector of results is retrieved when the analyses compute an array of results such as for example the time domain ones. If `index` is equal to 0, the current size of the resulting vector is retrieved, i.e., the entire vector of results; this is equivalent to not give the `index` parameter. If it is a positive integer the first `index` elements of the result vector are retrieved. If `index` is a negative number the last `index` elements of the result vector are retrieved. For example the `get("Tr.x",-2)` statement retrieves the last two values of the `x` electrical quantity, computed at the last two time steps of the `Tr` TRAN analysis.

When the get() control functions are used to retrieve results of analyses, the simulator *stores* these results in memory and thus a large amount of memory may be used. Results are kept in memory until explicitly deleted by the user. Deletion can be done through the control command clearwav name, where name is the result to be removed. For example the statement clearwav Tr.x removes the Tr.x result computed by the Tr TRAN analysis from memory.

2.19.2 The multirootf() function

The [x,err,iters]=multirootf("func",x,"deriv",iters,abstol) function finds a solution of a system of nonlinear equations by adopting a hybrid version of the Newton iterative method. The r=func(x) function takes as argument the x vector representing the initial value of the solution and returns the the r residue vector. The j=deriv(x) function takes as input the current value of the solution and returns the Jacobian matrix. If the "deriv" string is not specified (argument is omitted) since multirootf is called with only two arguments or the third argument is equal to null, the Jacobian matrix is automatically estimated by the Newton hybrid method. The iter argument is optional and defines the maximum allowed number of iterations. The abstol argument is optional and defines the absolute tolerance, when the norm of the residue is below this abstol, it is assumed that convergence was reached. The returned values are the x vector that stores the found solution, the real value err that stores the norm of the residue and the integer iters that stores the total number of performed iterations of the Newton algorithm.

2.19.3 The evaluate() function

2.19.4 The fork(), wait() & exit() functions

The pid=fork() function creates a new process that splits the execution of the PAN subsequent control statements and following analyses also outside the current control analysis where the fork() function is executed. The pid positive integer returned value is the process is (pid) of the newly created child process in the child execution context and 0 in the father execution context that invoked the fork() function. This split values allow the identification of the child execution context and of father one.
The rt=wait(pid) function waits for the termination of a generic child process if the pid argument is missed or waits for the termination of the specific child process with a process id equal to the given pid argument. The rt returned value is an array of two integer entries. The first entry is the pid of the terminated child process; if the first entry is negative something went wrong in waiting a child termination, in the majority of the cases this means that there are no more child processes to wait for. The second entry is the value assumed by the argument of the exit() function, if any.
The exit(st) function causes the termination of the child process if executed in this context or of PAN if executed in the father context. The st argument is a small (< 128) integer value that is recorded in the second entry of the array returned by the wait() function. The st argument can be used to monitor the correct or incorrect execution of the code by the child. The st argument can be omitted; in this case when the child process terminates the default exit codes of PAN are used.
The fork(), wait() functions allow the implementation of parallel code/analyses execution. A simple use example is given in Netlist 2.19.2.

Netlist 2.19.2 — The simple parallel code. – File: `fork.net` –

```
ground electrical gnd
parameters MODE = (rand() > 0.5) ? 1 : 0  TAG=8
Fork control begin
    ProcN = 5;
    for k=1:ProcN
        Pid(k) = fork();
        if(0 == Pid(k))
            printf("----> Child <----\n" );
            exit( TAG );
        end
    end

    if( MODE )
        Count = 0;
        while( 1 )
            Cond = wait();
            printf("----> Wait Pid: %d Status: %d <----\n",Cond(1),Cond(2));
            if(Cond(1) < 0)
                break;
            end
            if(Cond(2) != TAG)
                abort();
            end
            Count = Count + 1;
        end
        if(ProcN != Count)
            abort();
        end
    else
        for k=1:ProcN
            Cond = wait(Pid(k));
            printf("----> Wait Pid: %d Status: %d <----\n",Cond(1),Cond(2));
            if(Cond(2) != TAG)
                abort();
            end
        end
    end
endcontrol
```

The `MODE` parameter selects two different ways to wait child processes. The `TAG` value is the argument of the `exit(TAG)` function executed by the child processes. The `ProcN` variable sets the number of child processes running in parallel. The `fork()` function is executed `ProcN` times in the `for` body. If the `if(0 == Pid(k))` if clause is true the code is executed by the child process, that simply write the `"--> Child <--"` message and exits. The unique father waits for the child process terminations. It wait for a generic child process if `MODE` is 1 (true) or waits for a specific child pid if `MODE` is 0 (false). In both cases it is checked that the exit code of each child process is `TAG`

2.20 The pole/zero analysis

The pole/zero analysis is applied to the simple circuit shown in the Figure 2.1. This circuit is described by the Netlist 2.20.1.

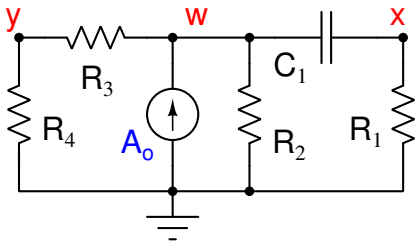

Figure 2.12: The schematic of the simple circuit used in the pole/zero analysis. .

Netlist 2.20.1 — The pole/zero simple circuit. – File: `pz1.net` –

```
Pz pz onodes=[x,y] idev="Ao"
Ao      w     gnd    isource
R2      w     gnd    resistor  r=1
C1      w       x    capacitor c=1
R1      x     gnd    resistor  r=1
R3      w       y    resistor  g=2
R4      y     gnd    resistor  g=3
ground electrical gnd
```

We want to compute the poles of the circuit and the zeros of the transfer function $\frac{v_x(s)-v_y(s)}{A_o}$.
The statement

```
Pz pz onodes=[x,y] idev="Ao"
```

performs a pole/zero analysis. The `onodes=[x,y]` specifies that the output port is identified by the `x` and `y` node pair. The input port is the `A1` independent current source, thus the input port is defined by the `w` and ground nodes.
Note that if the input and output ports are omitted only the pole analysis is performed.

2.20.1 Pole/zero analysis formulation

Consider the

$$\begin{bmatrix} \mathbf{G_{xx}} & \mathbf{G_{xy}} \\ \mathbf{G_{yx}} & \mathbf{G_{yy}} \end{bmatrix} \quad \text{and} \quad \begin{bmatrix} \mathbf{C_{xx}} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} \end{bmatrix}$$

matrices where the left matrix is the conductance matrix of the circuit and the right one is the capacitance matrix. We speak of "conductances" and "capacitances" but a similar reasoning can be applied to inductors. The $_{\mathbf{x}}$ suffix refers to the $\mathbf{x} \in \mathbb{R}^N$ pseudo-state variables[3] and the $_{\mathbf{y}}$ suffix refers to the $\mathbf{y} \in \mathbb{R}^M$ algebraic variables.
Now consider the

$$\det(\mathbf{G} + s\mathbf{C}) = 0 \tag{2.1}$$

equation, where $\mathbf{G} = \mathbf{G_{xx}} - \mathbf{G_{xy}}\mathbf{G_{yy}}^{-1}\mathbf{G_{xy}} \in \mathbb{R}^{N\times N}$ is the conductance matrix of the circuit, $\mathbf{C} = \mathbf{C_{xx}} \in \mathbb{R}^{N\times N}$ is the capacitance/inductance matrix and N is the number of equations/unknowns. The poles are the values of $s \in \mathbb{C}$ that satisfy (2.1). The roots of the (2.1) equation are also

[3] Since the MNA is adopted some variables that we consider as being state variables and thus we group together, are not. We refer to them as pseudo-state variables. Some of those can transform in algebraic one. For example think of a capacitor with its terminals grounded by two distinct resistors. We count 2 pseudo-state variables (one per node) but in reality there is only one. One pseudo-state variable will degenerate in an algebraic variable.

referred to as the generalised eigenvalues.
In a more general way, the generalized eigenvalue problem of two **G** and **C** matrices is to find a scalar p_k and the corresponding vector ϕ_k such that the equation $\mathbf{G}\phi_k = p_k\mathbf{C}\phi_k$ holds. In our case we modify this problem as

$$(\mathbf{G}+\sigma\mathbf{C})\,\phi_k = \widehat{p}_k\mathbf{C}\phi_k \quad k=[1,\ldots,N] \tag{2.2}$$

where $\sigma \in \mathbb{R}^+$. We look for $\widehat{p}_k \in \mathbb{C}$ that satisfies (2.2). We modified the original problem since, for a suitable choice of σ, $\mathbf{G}+\sigma\mathbf{C}$ is non-singular. Note that both **G** and **C** or one of these can be singular. We assume that the circuit is well posed and thus it admits a solution for a suitable $s \in \mathbb{C}$ angular frequency. If the $\mathbf{G}+\sigma\mathbf{C}$ were singular or ill-conditioned, σ would be varied till a well-conditioned matrix is obtained. Note that σ is randomly varied through a random number generator and so is the initial value of σ, if not specified by the user. Therefore the pole/zero analysis may give results that slightly vary from a run to another, due to accumulation of round-off errors in handling matrices.
Equation (2.2) is rewritten as $\phi_k = \widehat{p}_k\,(\mathbf{G}+\sigma\mathbf{C})^{-1}\,\mathbf{C}\phi_k$ and then as

$$(\mathbf{G}+\sigma\mathbf{C})^{-1}\,\mathbf{C}\phi_k - \frac{1}{\widehat{p}_k}\phi_k = 0 \tag{2.3}$$

Equation (2.2) is in canonical form to compute the eigenvalues, i.e. by setting $\mathbf{M} = (\mathbf{G}+\sigma\mathbf{C})^{-1}\,\mathbf{C}$ we have to compute the eigenvalues of **M** by solving the problem $\mathbf{M}\phi_k - \lambda_k\phi_k = 0$ with $\lambda_k = 1/\widehat{p}_k$. The original eigenvalues, that is the poles, can be derived as

$$p_k = \frac{{\lambda_k}^*}{|\lambda_k|^2} + \sigma \quad k \in [1,\ldots,N]$$

where the * symbol is complex conjugation. When the iterative solver is used, it computes a subset of the eigenvalues. The elements of the subset can be chosen through the `which` argument of the pole/zero analysis. When `which=1` the subset is composed of the p_k eigenvalues with the largest real part, when `which=2` the subset is composed of the p_k eigenvalues with the largest modulus.

The poles and zeros computed by the analysis can be stored in memory through the `mem=["poles","zeros"]` argument. These can be also retrieved through the control function

```
Po = get("PzAn.poles");
Zo = get("PzAn.zeros");
```

where `PzAn` is the name of the `pz` analysis.
The $\mathbf{M} = -\,(\mathbf{G}+\sigma\mathbf{C})^{-1}\,\mathbf{C}$ matrix can be retrieved by the `get()` control function through the statement

```
M = get("PzAn.invmtrx");
```

If $\det(\mathbf{M}) \neq 0$, i.e., **M** is not singular, the $\mathbf{C}^{-1}\mathbf{G}$ vector field can be reconstructed as $\mathbf{C}^{-1}\mathbf{G} = -\mathbf{M}^{-1} - \sigma\mathbf{1}$, where **1** is an identity matrix of suitable size.
The $\mathbf{G_{xx}}$, $\mathbf{G_{xy}}$, $\mathbf{G_{yx}}$ and $\mathbf{G_{yy}}$ matrices can be retrived by the `get()` control function through the statements

```
Gyx = get("PzAn.gxx");
Gyy = get("PzAn.gxy");
```

```
Gyx = get("PzAn.gyx");
Gyy = get("PzAn.gyy");
```

The names of the nodes or branch equations corresponding to the pseudo-state variables of the circuit (such as for example capacitors or inductors) can be retrieved through the statement

```
S = get("PzAn.stvars");
```

where `S` is a vector of strings. The names of the algebraic variables can be retrieved through the statement

```
S = get("PzAn.alvars");
```

The value of the σ shifting variable can be retrieved through the statement

```
Shf = get("PzAn.shifting");
```

These control variables can be made available also by explicit using the `mem` argument of the `pz` analysis. For example the statement

```
Pz pz onodes=[x,y] idev="Ao" mem=["invmtrx","stvars","shifting"]
```

makes available for later use the `"invmtrx"` matrix, `"stvars"` names of pseudo-state variables and the `"shifting"` value.

The zeros of the transfer function defined by the output port and the input port can be derived in a similar way. The output port is defined by the (o1, o2) node pair. The input port is an independent voltage or current source. The **G** matrix is bordered by adding a new extra row and a new extra column obtaining

$$\widehat{\mathbf{G}} = \begin{bmatrix} \mathbf{G} & \alpha \\ \beta & 0 \end{bmatrix}$$

where $\beta = [0 \ldots 1 \cdots -1 \ldots 0]$. In case of an independent current source the input port is defined by the (i1,i2) node pair. In case of an independent voltage source the input port is defined by its branch equation. The ± 1 entries in β has an index that correspond to that of the (o1, o2) nodes. If one of these two nodes is ground the corresponding entry is 0.

The α vector has a meaning similar to the β one. Its entries are all null but those corresponding to the nodes or the branch equation of the input port. The **C** matrix is enlarged by adding an extra-row and an extra-column of null entries. The zeros of the transfer function are the z_k values obtained by solving

$$\left(\widehat{\mathbf{G}} + \sigma\widehat{\mathbf{C}}\right)^{-1}\widehat{\mathbf{C}}\phi_k - \frac{1}{\widehat{z}_k}\phi_k = 0$$

and applying the transformation

$$z_k = \widehat{z}_k + \sigma \quad k \in [1, \ldots, N]$$

The $(\mathbf{G} + \sigma\mathbf{C})^{-1}\alpha$ vector can be retrieved by the `get()` control function through the statement

```
B = get("PzAn.bvec");
```

where `PzAn` is the name of the `pz` analysis. The `"bvec"` vector is made available only if the `idev` parameter of the `pz` analysis is specified.

2.20.2 An application example

The pole/zero analysis is applied to the simple circuit shown in the Figure 2.1. In writing the matrices we consider the circuit shown in Figure 2.1 and chose the x, w, y node order. By appling MNA we obtain

$$\mathbf{G} = \begin{bmatrix} g_1 & 0 & 0 \\ 0 & g_2+g_3 & -g_3 \\ 0 & -g_3 & g_3+g_4 \end{bmatrix} \quad \mathbf{C} = \begin{bmatrix} c_1 & -c_1 & 0 \\ -c_1 & c_1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Since $\det(\mathbf{G}) \neq 0$, to obtain the poles we have to solve the problem

$$\mathbf{G}^{-1} \begin{bmatrix} c_1 & -c_1 & 0 \\ -c_1 & c_1 & 0 \\ 0 & 0 & 0 \end{bmatrix} \phi_k - \frac{1}{\widehat{p}_k}\phi_k = 0$$

We do this by exploiting the *control* language. Consider the statements

```
M = get("Pz.invmtrx");
Shf = get("Pz.shifting");
Lmd = eig(M)
Po = (1/Lmd(2) + Shf)/(2*pi)
```

The `eig(M)` command computes the eigenvalues of `M`. One eigenvalue is null since the C_1 capacitor in the circuit of the Figure 2.1 is floating. The reciprocal of the `Lmd(2)` non-null eigenvalue is computed and the result shifted by `Shf`. The `Po` pole is expressed in Hz, this justifies the division by 2π.

By considering A_o as driving port and the y,x node pair as output port we obtain

$$\alpha = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} \quad \beta = \begin{bmatrix} -1 & 0 & 1 \end{bmatrix}$$

By bordering the $\mathbf{G}$ matrix, we have

$$\widehat{\mathbf{G}} = \begin{bmatrix} g_1 & 0 & 0 & 0 \\ 0 & g_2+g_3 & -g_3 & 1 \\ 0 & -g_3 & g_3+g_4 & 0 \\ -1 & 0 & 1 & 0 \end{bmatrix}$$

Since $\det\left(\widehat{\mathbf{G}}\right) \neq 0$, also in this case we chose $\sigma = 0$. To compute the zeros we have to solve the problem

$$\widehat{\mathbf{G}}^{-1} \begin{bmatrix} c_1 & -c_1 & 0 & 0 \\ -c_1 & c_1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \phi_k - \frac{1}{\widehat{z}_k}\phi_k = 0$$

2.21 The MonteCarlo analysis

The MonteCarlo analysis is described by means of a very simple circuit example. Consider a voltage divider made up of the two `r1` upper resistor and of the `r2` lower resistor. The output voltage at the `x` node is $v_x = \dfrac{r_2}{r_1+r_2} v_1$, where v_1 is the voltage across the independent voltage source that drives the voltage divider. This simple circuit example is implemented through the Netlist 2.21.1.

Netlist 2.21.1 — The netlist of the voltage divider. – File: mcdiv.net –

The voltage at the x output node is half that of the independent voltage source v1, being the r1 and r2 identical. The value of these resistors is equal to the rmc parameter. In turn the rmc parameter is equal to the rx=10 one. These values can be easily identified by looking at the parameter statement in the Netlist 2.21.1. Any time a parameter is followed by the "lot/process" or "dev/process" keywords the statistical process of type "process" is assign to the parameter. The "process" can be of type "gaussian" or "uniform". In our case the rmc parameter in the Netlist 2.21.1 is linked to a <dev/gaussian> stochastic process with variance equal to the value of the std=0.5 parameter. The rmc parameter sets the resistance value of both r1 and r2; it is used in their instances.
When the dev keyword follows a parameter definition, this means that each time the parameter is used to set a value in the circuit, in our case two resistor instances, each setting is *uncorrelated* from the other ones. In our case it means that the random values assigned to the r1 resistance are *uncorrelated* from those of the r2 resistance.
When the lot keyword is used it means that each time the random parameter is used to set a value, this setting is *identical* to the other ones. If the lot keyword were used in our case it means that any time a new random number is generated, this very random number is assigned to both r1 and r2 resistances. Thus r1 and r2 resistances randomly vary but always with the same value. If we did this, we would obtain a constant v_x value despite resistance random variations. The lot keyword is used when it is required to keep devices matched.
The control analysis named MonteCarlo has a body that implements an interpreted language similar to that of Matlab/Octave environments. This language is described in Section 2.19. The count=0 statement initialises the count variable and the samples(100)=0 statement initialises to 0 the samples array of 100 entries. The mc MonteCarlo analysis is defined in the body of the control analysis; it runs 100 times the statements inside its body. Also this body, i.e. the body of any MonteCarlo analysis is composed of control statements. Among those there is the Dc dc analysis. The number of runs and thus of the samples generated by the random number generators is set by the samples=100 parameter of the mc MonteCarlo analysis. At each iteration of the MonteCarlo analysis the its body is executed. The count variable is increased by 1 and then the voltage al node x is retrieved and stored in an entry of the samples array. When the MonteCarlo analysis ends the mean value and the standard deviation of the sampled stored in the array are computed and displayed.

3. AMS simulation with PAN

Modern circuit designs are often based on the Analog Mixed Signal (AMS) "paradigm", where various parts of the overall system are modeled using different approaches. The most common situation is where an analog circuit, such as the ones seen in the previous section, are connected with *digital* circuits or with blocks described using behavioural languages.
PAN is capable of simulating AMS circuits seamlessly integrating the analog netlist with a Verilog description of the digital part and a Verilog-A description of the behavioural part.

Simple analysis are available, but time-domain steady state or more complex analyses such as PAC pr PNOISE are also available. This is not common, since "standard" steady state algorithms require continuous vector fields and this is not the case of AMS circuits.

In the following a few simple examples, mostly derived from those published by the authors in [**IEEE_mag_nostro**] are shown from an operative point of view. Please refer to the cited paper and its bibliography for technical details concerning the used algorithms.

3.1 A simple Analog Digital oscillator

The first circuit we consider is a very simple Pulse Energy Restore Oscillator (PERO) [**5730478**; **6043387**]. The schematic of the PERO oscillator is shown in Fig. 3.1. It is composed of a linear

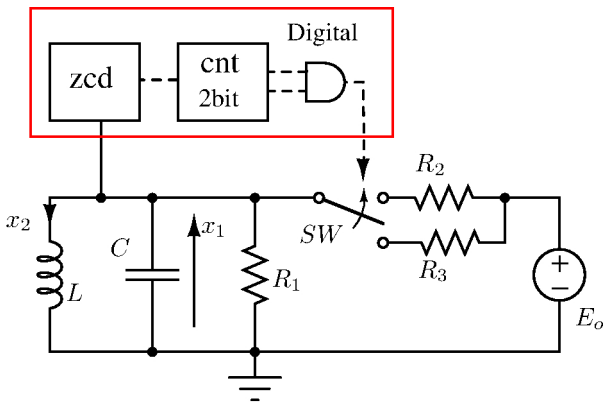

Figure 3.1: The schematic of the PERO oscillator. $C = 1\,\mathrm{F}$, $L = 1\,\mathrm{H}$, $R_1 = 10\,\Omega$, $R_2 = 100\,\mathrm{M}\Omega$, $R_3 = 5\,\Omega$, $E_o = 1\,\mathrm{V}$.

analog part and a digital finite state machine that recharges the energy in the LC tank. A zero crossing detector (zcd) is used as input to a 2 bit digital counter (cnt), that closes switch S when both its bits are high. The analog netlist is written in standard PAN format and is shown in Netlist 3.1.1

Netlist 3.1.1 — The PERO oscillator (analog part). – File: pero.net –

```
ground electrical  gnd
parameters VDD=1
verilog_include comb.v

Tran tran tstop=300 uic=yes
Sh0 shooting period=25 autonomous=yes restart=no \
             method=2 maxord=2 solver=1 minper=19 floquet=yes
Pnoise pnoise start=10u stop=100m dec=4000 onodes=["x"] sweeptype=0

r1    x   gnd   resistor    r=10
c1    x   gnd   capacitor   c=1 ic=1
l1    x   gnd   inductor    l=1
sw    x   vdd   y  gnd  SW
vr  vdd   gnd   vsource     vdc=VDD

A2d1  x   gnd   a2d  dignet="cnt.X" vl=0 vh=0
D2a1  y   gnd   d2a  dignet="cnt.Y" vl=0 vh=VDD
D2a2  z   gnd   d2a  dignet="cnt.Count" vl=0 vh=VDD

model SW vswitch ron=5  roff=100M von=0.6*VDD voff=0.4*VDD
```

The sw voltage controlled switch implements also the R_2 and R_3 resistor of the netlist; in fact

```
model SW vswitch ron=5 roff=100M von=0.6*VDD voff=0.4*VDD
```

sets the values of roff and ron to those chosen, respectively, for R_2 and R_3.
The AMS specific parts of this netlist are two, the first is the single line

```
verilog_include comb.v
```

that instructs the simulator to read also a Verilog file called, in this case, comb.v (more than one file can be read if needed). This file, shown in Netlist 3.1.2, contains the description of the digital part of the circuit. The second AMS specific part is used to connect the analog part and the digital one.

```
A2d1  x   gnd   a2d  dignet="cnt.X" vl=0 vh=0
D2a1  y   gnd   d2a  dignet="cnt.Y" vl=0 vh=VDD
D2a2  z   gnd   d2a  dignet="cnt.Count" vl=0 vh=VDD
```

These three lines define three *pseudo* components that convert analog signals to digital or vice versa (respectively a2d and d2a). Node names are given in standard PAN netlist syntax, while the corresponding digital net by using the dignet keyword. Note that the format for digital net is *module.NET*. Voltage levels must also be stated, representing a threshold level for the a2d components and mapping levels for the d2a ones. More details can be found in Section 6.2
The digital part of the circuit is quite simple and written is standard Verilog language.

Netlist 3.1.2 — The PERO oscillator (digital part). – File: `comb.v` –

```
‘timescale 1ms/10us
module cnt;
reg X, Y;
reg [1:0] Count;
initial begin
    Count = 0;
    Y     = 1;
end
always @(posedge X) begin
    Count[0] = ! Count[0];
    Count[1] = (!Count[0]) ^ Count[1];
end
always @(Count) begin
    if( Count[0] && Count[1] )
        Y = 1;
    else
        Y = 0;
end
endmodule
```

The simulation is executed exactly as any other analog only circuit,

```
prompt> pan pero.pan
```

and the usual directory `pero.raw` containing results for each analysis is created.
Let us first see results from the transient analysis `Tran`. The two analog state variables, i.e. capacitor voltage and inductor current are represented in the netlist file by node voltage `x` and current `l1:i` respectively. The third state variable is digital and corresponds to the 2 bit register `Count` defined in the Verilog part of the netlist. This register is available as a digital net (keyword `dignet`) in the analog part of the netlist as `cnt.Count`, where `cnt` is the Verilog module name. This digital net is remapped to the analog net `z` by the d2a converter `D2a2`. The conversion transforms the two bit value to a single scalar value in the range defined by `vl=0 vh=VDD`, i.e., in our case, to the 4 possible values $0, 1/3, 2/3, 1$. The transient behaviour of these state variables is shown in the three panels of Fig. 3.2.
The shooting analysis follows the transient analysis. This analysis finds the steady state solution for the circuit. Note that, since this circuit is autonomous, also its period must be determined and option `autonomous=yes` must be explicitly specified. Furthermore, the parameter `minper=19` tells the shooting analysis the period of the oscillator must be no less than 19. This allows the shooting analysis to find solution characterised for example by period doubling. In our case the “recharge” of the LC-tank is performed ever four oscillations and thus the working period of the oscillator is longer than that of the LC-tank.
The steady state behaviour of the circuit is shown in Fig. 3.3, where the state variables are shown as a trajectory in state space (on the z axis we have put, in this case, the binary counter value), and Fig. 3.4, where they are shown versus time.

3.2 The pnoise analysis

In the previous example, based on the PERO oscillator, after the shooting analysis a `pnoise` analysis is also performed. The relevant line is

```
Pnoise pnoise start=1m stop=100m lin=1000 onodes=["x"] sweeptype=0
```

This analysis requires a previous shooting analysis and computes the noise at a specified output

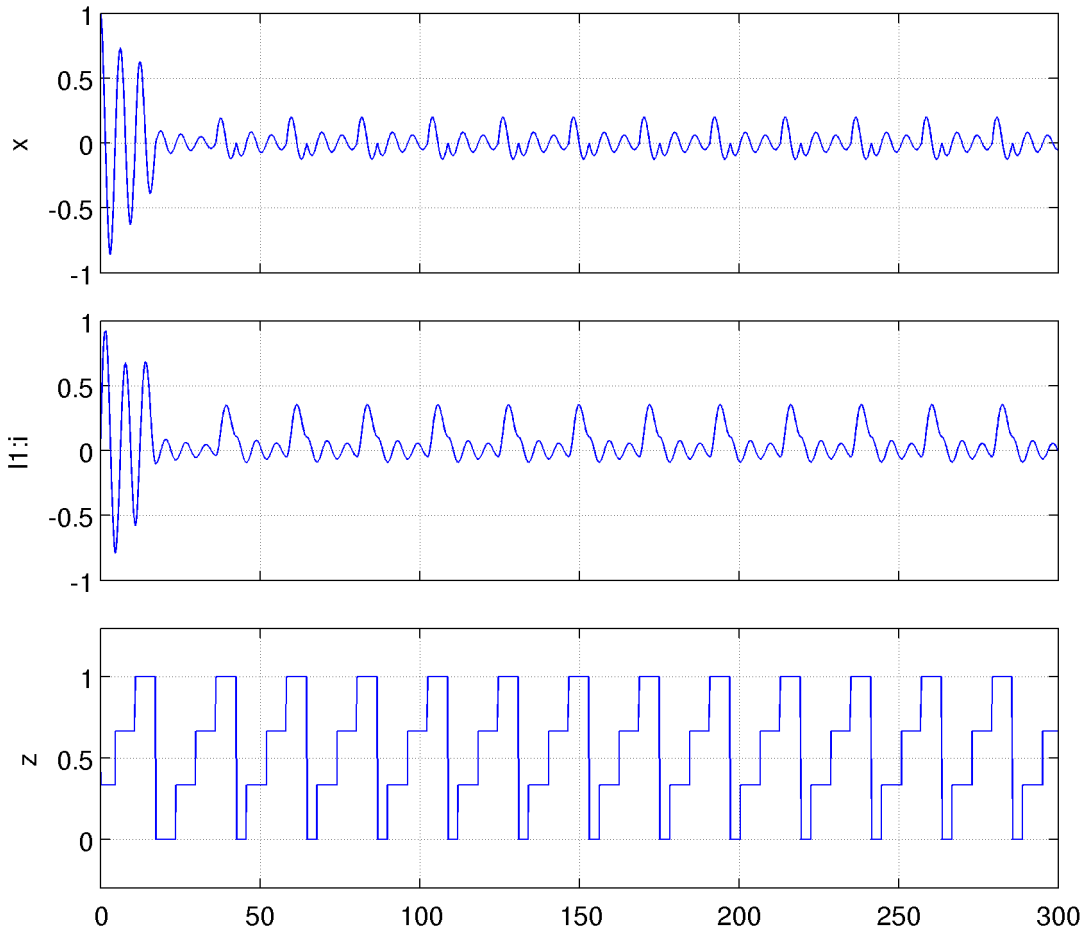

Figure 3.2: The time evolution of the three state variables of the PERO oscillator. Capacitor voltage is `x` in the top panel, inductor current `I1:i` in the centre panel, and the analog signal corresponding to the digital two bit counter value (remapped in the 0 to 1 range) in the bottom one.

(in our case node `x`) of circuits that admit a periodic solution.

3.3 A behavioural analog/digital pll

A more articulated and complex AMS circuit is represented by a PLL. We consider a behavioural PLL where the charge pump is modelled in a simple way by means of two controlled current sources, the low-pass filter is modeled as an analog circuit, the vco (voltage controlled oscillator) is modeled as a behavioural analog block through the VERILOGA language and finally the phase-frequency detector and the frequency divider are modeled as a full digital block through the VERILOG language.
The schematic of the PLL is shown in Figure 3.5. Our target is to show how it can be efficiently and, let us say, quickly modelled in PAN and in the same way simulated.
The netlist of the analog part of the circuit with the instances of the pseudo-a2d and pseudo-d2a converters that link the analog circuit and the digital circuit is reported in Netlist 3.3.1. By observing the Netlist 3.3.1 the instance of the VCO is

```
Vco osc tune VCO FVCO=FVCO-KVCO KVCO=KVCO
```

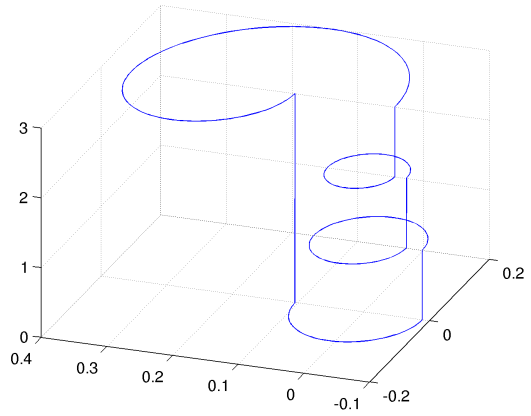

Figure 3.3: Steady state trajectory of the PERO oscillator plotted in state space.

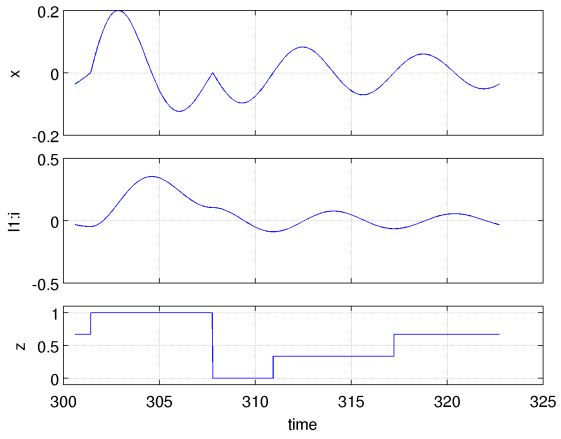

Figure 3.4: Steady state trajectory of the PERO oscillator plotted vs time. Note that, since the shooting analysis is restarted from the previous transient one, the time axis starts at 300.

where we suitably set the KVCO gain and FVCO default frequency in the VCO instance. In this PLL the divider divides by 72. Since the reference frequency is FREF $= 50$ MHz, the VCO will lock at FVCO $= 72 \times 50\,\text{MHz} = 3.6\,\text{GHz}$. Note that we set the default frequency of the VCO instance at FVCO $-$ KVCO $= 3.6\,\text{GHz} - 210\,\text{MHz}$, thus the voltage at the **tune** terminal driving the VCO will be 1 V on average in one working period of the PLL at steady state. The VERILOGA code reported in Netlist 3.3.2 implements the behavioural VCO.

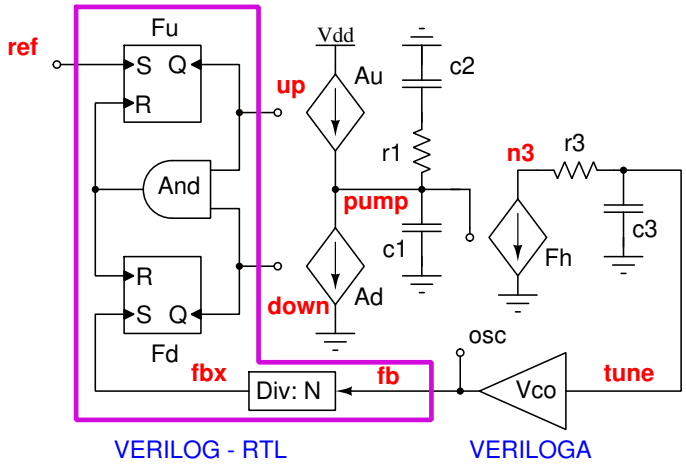

Figure 3.5: The schematic of the behavioural PLL.

Netlist 3.3.2 — The behavioural vco. – File: `vco.va` –

```
module VCO(outmod,tune);
input tune;
output outmod;
electrical outmod, tune;

parameter real FVCO=2G  from [1G:10G];
parameter real KVCO=20M from [1M:500M];
real arg, phasemod;

analog begin
    arg = KVCO*v(tune) + FVCO;
    phasemod = 2.0*pi*idtmod( arg, 0.0, 1.0, 0.0 );
    v(outmod) <+ sin( phasemod );
end
endmodule
```

The default frequency is defined by the `FVCO` parameter. It can be overwritten by the VCO instance. The actual VCO frequency is given by the

```
arg = KVCO*v(tune) + FVCO
```

expression. The `arg` value is integrated by the `idtmod` VERILOGA primitive function

```
phasemod = 2.0*pi*idtmod( arg, 0.0, 1.0, 0.0 );
```

The integral output is reset to 0 when its value crosses the upper value 1 and it it stepped to 1 when in crosses the lower value 0 (see [**VeriaRef**] for more details). The integral output is multiplied by 2π. All this makes the `phasemod` variable vary between 0 and 2π; this variable is the argument of the sin() function that drives the `outmod` output pin of the VCO. Thus if the `tune` voltage is varied the sinusoidal output waveform at pin `outmod` varies consequently. The default frequency/voltage gain of the VCO is `KCO` $= 20\,\mathrm{MHzV}^{-1}$. Recall that this value is overwritten in the instance of the VCO and set at $210\,\mathrm{MHzV}^{-1}$.
By observing the block schematic in Figure 3.5 the output of the VCO drives the integer divider block implemented through the VERILOG language. This VERILOG code is reported in Netlist 3.3.3. In particular the integer divider is implemented by the

Netlist 3.3.1 — The behavioural PLL (analog part). – File: `pll.net` –

```
parameters VDD=1 FREF=50M PERIOD=1/FREF
parameters TRANSIENT_TSTOP=1.2e-04 REGIME_TSTOP=2.21000056e-04 KVCO=2.1e+08 \
    FVCO=3.6e+09 C2=4.85481127e-09 C1=2.00070304e-10 R1= 2.95075887e+02 \
    RPS=1e+03 CPS=6.36619772e-11 ICPu=5.5e-03 ICPd=4.5e-03

; Analyses
Tr tran tstop=TRANSIENT_TSTOP/10+PERIOD/4 tmax=0.01/FVCO tmin=1f uic=2 \
        cmin=0 method=2 maxord=2 trabstol=1a trreltol=1n
Sh shooting restart=no period=PERIOD tmax=0.01/FVCO tmin=1f \
            method=2 maxord=2 trabstol=1a trreltol=1n floquet=1
Pn pnoise start=100 stop=1M dec=10 sweeptype=2 onodes=["osc"] harms=128 \
          refh=72


; Reset phase/frequency detector
vres_n  res_n   gnd   vsource t=0 v=0 t=PERIOD v=0 \
                                       t=PERIOD+10p v=VDD t=REGIME_TSTOP v=VDD
; Reset frequency divider
vres_c  res_c   gnd   vsource t=0 v=0 t=10/FVCO v=0 \
                                       t=PERIOD+10p v=VDD t=REGIME_TSTOP v=VDD

; Reference signal
Vref      ref   gnd   vsource vsin=VDD freq=FREF

; Interface to the digital phase/frequency detector.
x1    ref   gnd   a2d  dignet="pd.ref"   vt=0.0*VDD
x2    osc   gnd   a2d  dignet="pd.fb"    vt=0.0*VDD
x3  res_n   gnd   a2d  dignet="pd.res_n" vt=0.5*VDD
x4  res_c   gnd   a2d  dignet="pd.res_c" vt=0.5*VDD
x5     up   gnd   d2a  dignet="pd.up"    vl=0         vh=VDD
x6   down   gnd   d2a  dignet="pd.down"  vl=0         vh=VDD
verilog_include pdNdiv.v

; Charge pump
Au    ipump   gnd   up  gnd  vccs func=v(up)   > VDD/2 ? -ICPu : 0
Ad    ipump   gnd down  gnd  vccs func=v(down) > VDD/2 ?  ICPd : 0
ipump ipump   pump  vsource dc=0 // Measure pump current

; Low-pass filter
c1  pump   gnd  capacitor c=C1
r1  pump    n2  resistor  r=R1
c2    n2   gnd  capacitor c=C2
Fh    n3   gnd pump gnd vcvs gain=1
r3    n3  tune  resistor  r=RPS
c3  tune   gnd  capacitor c=CPS

; Vco
Vco  osc  tune  VCO FVCO=FVCO-KVCO KVCO=KVCO
model VCO nport veriloga="vco.va"

ground electrical gnd
```

```
always @(posedge fb or !res_c)
    if( !res_c ) Count = 0;
    else begin
Count = Count + 1;
if( Count >= HalfN || HalfN == 0 ) begin
    Count = 0;
    fbx = !fbx;
end
    end
```

Netlist 3.3.3 — The frequency divider and phase detector. – File: pdNdiv.v –

```
‘timescale 1fs/1fs
‘delay_mode_path

module pd (up, down);
output up, down;
reg res_n, res_c, up, down, reset, ref, fb, fbx;
reg [15:0] Count;
reg [15:0] HalfN;

initial begin
    Count = 0; reset = 0; up    = 0; down  = 0; res_c = 0;
    res_n = 0; fb    = 0; fbx   = 0; HalfN = 36;
end
always @(posedge fb or !res_c)
    if( !res_c ) Count = 0;
    else begin
        Count = Count + 1;
        if( Count >= HalfN || HalfN == 0 ) begin
            Count = 0;
            fbx = !fbx;
        end
    end
always @(posedge ref or posedge reset or !res_n)
    if (reset | !res_n) up = 0;
    else                up = 1;
always @(posedge fbx or posedge reset or !res_n)
    if (reset | !res_n) down = 0;
    else                down = 1;
always @(up or down)
    if (up & down) begin
        #2000000 reset = 1;
        #2000000 reset = 0;
    end
always @ (!res_n) reset = !res_n;
endmodule
```

VERILOG code. Each time there is a positive edge of the **fb** digital signal the `Count` variable is incremented by 1. When `Count` equals `HanfN`, that in our case is 36, `Count` is reset to 0 and the **fbx** digital signal is flipped (if it has a 0 logical value it is set at 1 and vice-versa). The **fbx** signal has thus a period that is 76 times longer than that of **fb** and has a duty cycle of 0.5. By observing the block schematic in Figure 3.5 and the code in Netlist 3.3.3, each time **fbx** has a positive edge the corresponding FLIP/FLOP is set, in this case the **down** signal sets at 1 logic level and the `Ad` dependent current source of the charge pump sinks current from the low-pass filter. When the **ref** reference signal has a positive edge the `Fu` FLIP/FLOP is set and it drives the `Au` dependent current source. If both the **up** and **down** signals are at 1 logic level the `And` port resets both FLIP/FLOPs. The `And` port has a delay of 2 ns as shown by the VERILOG code

```
always @(up or down)
    if (up & down) begin
        #2000000 reset = 1;
        #2000000 reset = 0;
    end
```

The `up` and `down` edges are checked and if both `up` and `down` are at 1 logic level the `And` port resets the FLIP/FLOP.
As already partially anticipated the charge pump is composed of the `Au` and `Ad` voltage controlled current sources. Note that the currents injected differ, that is, we are considering a mismatched charge pump. Thus we expect that at steady state the charge pump is still active and that the positive edges of the **up** and **down** signals have a relative delay such that the integral of the injected current per working cycle, i.e., the total injected charge is *null*. This keeps constant the voltage of the **tune** terminal that drives the VCO.
We first simulated this behavioural PLL in the time domain to compute its start-up phase. The card of the tran analysis is

```
Tr tran tstop=TRANSIENT_TSTOP/10+PERIOD/4 tmax=0.01/FVCO tmin=1f uic=2 \
        cmin=0 method=2 maxord=2 trabstol=1a trreltol=1n
```

The meaning of main parameters is: `tmax=0.01/FVCO` limit the maximum integration time step to ensure an adequate number of samples, `tmin=1f` limit the minimum allowed integration time step, this may help convergence and maily speed up simulation. The 1 fs is the time resolution of the pseudo-cpu specified in the VERILOG code implementing the digital circuit model. `cmin=0` turns off the insertion of parasitic capacitors in case of convergence difficulties. `method=2` and `maxord=2` select the Gear integration method of order 2. `trabstol=1a` and `trretol=1n` set the accuracy thresholds that control how *lte* (local truncation error) acts on the integration time step.
The voltage of the **tune** node is shown in Figure 3.6. We immediately see that the **tune** voltage increases till 1 V as expected and that has an overshot at the beginning of the tran analysis.
We then performed the `Sh` shooting analysis by starting from the last result of the tran analysis. The card of this analysis is

```
Sh shooting restart=no period=PERIOD tmax=0.01/FVCO tmin=1f \
            method=2 maxord=2 trabstol=1a trreltol=1n floquet=1
```

Note that the ability to perform a shooting analysis of an analog-mixed signal circuit is a peculiarity of `PAN` that as far as we known is not available in other circuit simulators, even in the commercial ones [**bizzarri2013steady**; **6092511**; **bizzarri2012periodic**; **bizbook**; **CTA:CTA1864**]. `PAN` also computes the Floquet multipliers (`floquet=1`) that are $\lambda_1 = 0.627$, $\lambda_2 = 0.909 \pm 0.0676$ and $\lambda_3 = 0.982$. They are all inside the unit circle in the complex plane since even if the PLL is composed of the VCO it is not an autonomous circuit. The Floquet multipliers show that the

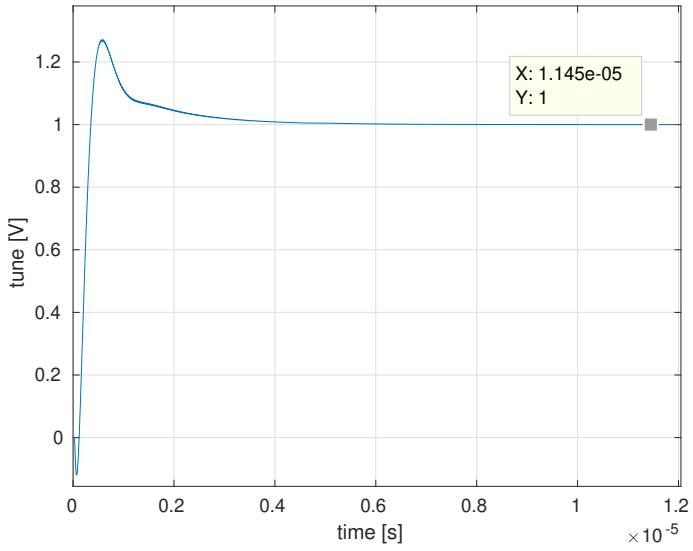

Figure 3.6: The voltage of the **tune** terminal computed by the `Tr` tran analysis.

pll admits a stable steady state solution. The simulation results by the `Sh` analysis are shown in Figure 3.7. We see that the steady state voltage of the **tune** node (upper panel in Figure 3.5) varies around the 1 V value as expected. The average value of this waveform is exactly 1 V. The steady state current of the charge pump (second panel from top in Figure 3.5) instantaneously jumps from 0 to -4.5 mA when the **down** signal (third panel from bottom in Figure 3.5) goes high and steps to 1.5 mA $= (5.5 - 4.5)$ mA, given by the difference of the currents injected by the `Au` and `Ad` controlled current sources, when also the **up** signal goes high (third panel from top in Figure 3.5). The average value of the charge pump current is 0. The charge pump current remains at 1.5 mA for 1 ns that is the delay time of the `And` port in the block schematic of the PLL shown in Figure 3.5.
The last two panels in Figure 3.5 show the waveforms of the reference signal and of the **out** output terminal of the VCO. The working frequency of the VCO is 3.6 GHz as designed.
We complete the walk-through of this example by computing the noise at the output of the VCO. The VCO and the phase/frequency detector and integer divider are noiseless, also the charge pump is noiseless. The resistors of the low-pass filter are noisy (thermal noise). Thus the noise power spectral density at the output of the VCO is exclusively due to the up-conversion of the noise of these resistors. The card that executes the periodic noise analysis is

```
Pn pnoise start=100 stop=1M dec=10 sweeptype=2 onodes=["osc"] harms=128 \
          refh=72
```

The noise analysis computes the power spectral density in the right bandwidth with respect to the fundamental of the VCO that starts from 100 Hz offset till 1 MHz offset. The noise power spectral density is reported in Figure 3.8. The frequency axis reports frequencies from 3.6 GHz + 100 Hz to 3.6 GHz + 1 MHz. This Pn pnoise analysis is CPU time consuming since in order to compute noise at the output of the VCO we have to consider as reference frequency the 72 harmonics of the fundamental (`refh=72`) and a total of at least 128 harmonics (`hamrs=128`). This makes the analysis inefficient. We plan to improve efficiency in a future version of `PAN`.

3.3.1 Alter analysis

The `alter` analysis allows to change values of instance, model parameters and simulation options. For more information see the description of the *alter* analysis. Here we recall that parameters defined in the analog circuit netlist and used in verilog modules are evaluated during the compilation of the analog/digital circuit. If one of these parameters is modified by an `alter` analysis, its new value is not used in the verilog modules, i.e., the original values of these parameters determined when the analog/digital circuit was compiled are unchanged.

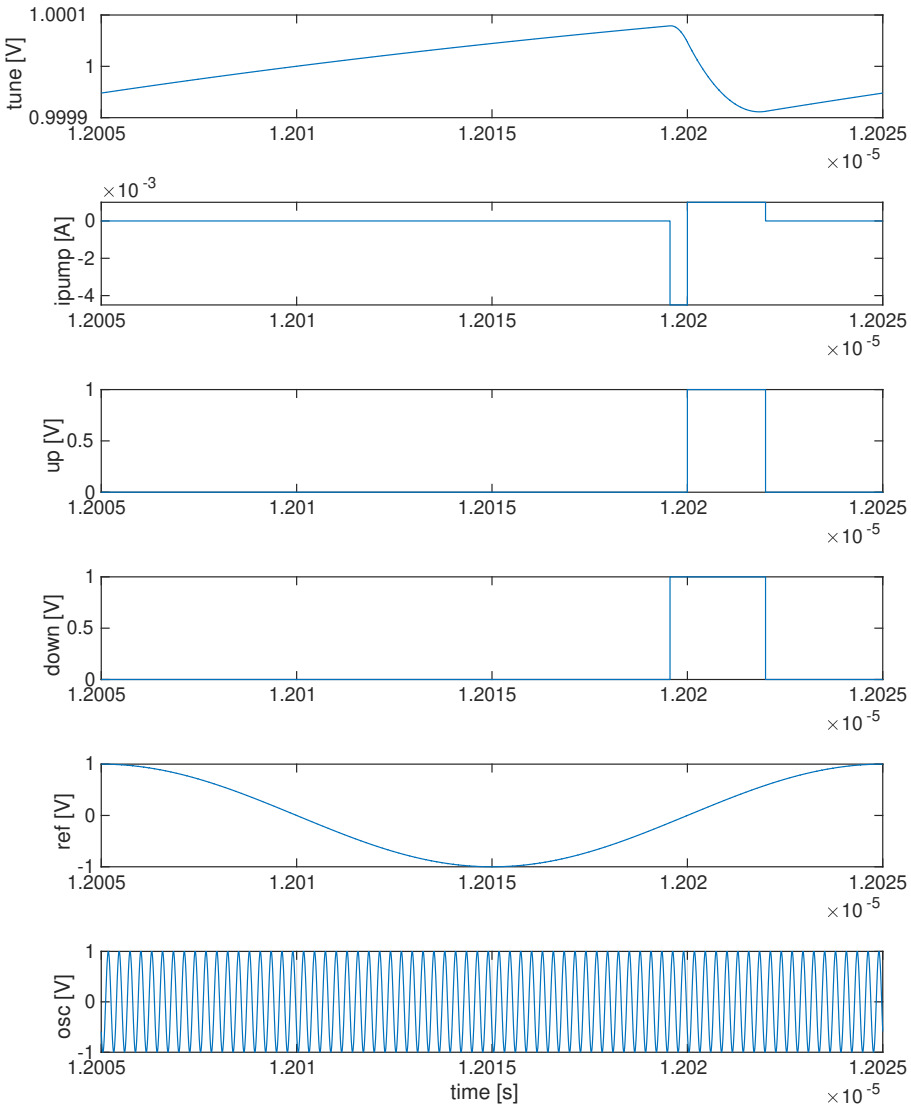

Figure 3.7: The simulation results of the PLL shown in Figure 3.5 computed by the Sh shooting analysis.

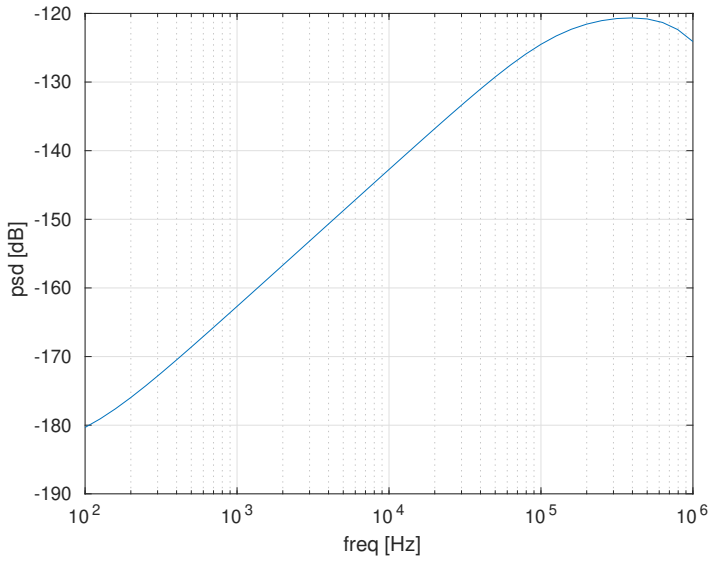

Figure 3.8: The noise power spectral density at the **out** output terminal of the VCO.

4. The Matlab interface

4.1 Introduction

PAN is integrated with MATLAB and it can be seen as a "Toolbox". Analyses can be defined and dynamically executed from MATLAB and results collected. When used in this context PAN comes as a shared library that is linked to MATLAB through auxiliary MEX files. A procedural access to these MEX files and then PAN is available to the MATLAB user that makes easy the interaction. The MATLAB to PAN interface is based on the `pansimc(text)` MATLAB MEX procedure. This

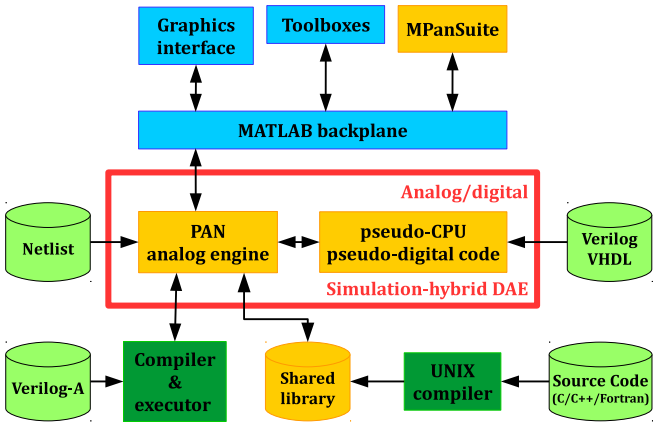

Figure 4.1: Block level structure of the MatlabPAN interface.

procedure has as argument the `text` string that is the replica of the same statement used to execute an analysis in the conventional netlist of PAN. The analyses have the `mem` parameter that allows the user to specify the results that must be stored in memory (memwav data-base). These memory stores results can be accessed by MATLAB.

4.2 How to install the interface

The MatlabPAN interface can be easily installed through a short sequence of simple commands. The first part has to be executed from Linux and the remaining one from MATLAB.

1. From Linux download the "MPanSuite.zip" and "panMat.so" files.

2. Unzip the "MPanSuite.zip" file in a temporary directory.
3. Run MATLAB from the temporary directory.
4. From the MATLAB command interface run the script "MPanSuite/MPanSuiteInstall.m", that is, execute the command

```
run ./MPanSuite/MPanSuiteInstall.m
```

5. Follow the instructions displayed. Mainly it is asked to specify a directory into which the interface will be installed.

After having installed the MatlabPAN interface any time MATLAB is run, the previously installed "MPanSuiteInit.m", which is located in the installation directory, must be executed to make active the interface. An example of what must be run in MATLAB is reported

```
run /home/pan/bin/MPanSuite/MPanSuiteInit.m
```

4.3 How to read the netlist and execute an analysis

The netlist can be read and parsed from MATLAB by executing the `pannet('netlist_file_name')` function, where its argument is the pathname of the file describing the circuit netlist. Several netlist can be loaded but only the last loaded netlist is active and PAN runs only on the last loaded netlist. Any time a new netlist is loaded the old one and all the related data are discharged.
The `pansimc('analysis')` executes the 'analysis' analysis. Its argument is a string and it has exactly the same syntax of the conventional card analysis that can be used in the netlist. Several analyses can be consecutively executed during a simulation session with the same netlist. If in the string defining the analysis some parameter value are expressed in terms of MATLAB variables, the values of these variables are read during the parsing phase of the analysis string. For example consider the MATLAB command

```
pansimc('Tr tran stop=TSTOP+DT');
```

If TSTOP and DT are MATLAB variables defined in the workspace, their values are read when the analysis string is parsed and the simple expression is evaluated.

4.4 Getting results

The MATLAB function `y=panget('argument')` allows the user to retrive results from the analysis and values of parameters of circuit instances, models and parameters. It acts in a way indentical to the `get()` control function, therefore see the `get()` control function description for more details.
The 'argument' string identifies the result in the memory waveform data-base. Note that the simulator saves results in the waveform data-base only if explicitly instructed. Waveform saving in the data-base is RAM memory consuming thus the user must be carefull in handling results.
The syntax is 'NameOfTheAnalysis.Eqn' where 'NameOfTheAnalysis' refers to a simulator analysis and 'Eqn' is the name of a circuit node or branch equation. In the Netlist 4.7.1 there is a usage example.
If a parameter of an instance or model has to be retrived, the syntax is similar to the previous one. The name of the analysis has to be substituted with the name of the instance or model; the node name has to be substituted with the parameter name. If a parameter of an instance does not exist it is considered as a parameter of the corresponding model (associated to the instance) and the search is repeated. The returned 'y' variable can be both a scalar or an array according to the type of 'argument' passed to the `panget()` function.

The panclearwav('name') command clear the 'name' variable from the waveform database of PAN . See the get() control command for more details.

4.5 Other commands

MATLAB user graphic interface does not refresh the screen when a new character or even a string is printed. Thus the user may experience a long waiting during an analysis without seeing any message about the run progress. This is avoided by forcing MATLAB to redraw the graphic interface any time a *new character* is printed. On the other hand when MATLAB is remotedly used through a non well performing network connection, the user may experience a drastic reduction of efficiency maily due to waiting of the refresh of the graphical interface of MATLAB. The panredraw() function allows the user to turn on/off the automatic redraw of the graphic interface of MATLAB. The panredraw('on') turns on redrawing and panredraw('off') turns off redrawing. Note that this command can be used once the pannet('netlist_file_name') has been executed and not before it.

4.6 Write a new element in Matlab

In this section, we show how to implement a new element in MATLAB and use it in a circuit. Consider the very simple and linear circuit shown in Figure 4.2 The E1 independent voltage

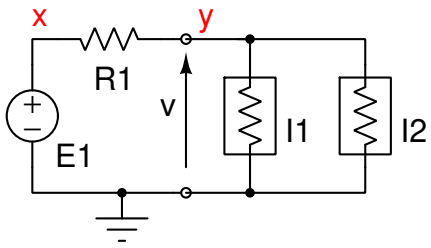

Figure 4.2: The schematic of the circuit used to maximise power transfer.

source and the R1 resistor have fixed values and are conventional built-in element of PAN. The I1 and I2 elements are *new* and we assume not available in the simulator; they are implemented in MATLAB. The netlist corresponding to the model of the circuit in Figure 4.2 is reported in Netlist 4.6.1.

Netlist 4.6.1 — PAN netlist. – File: myres.net –

```
ground electrical gnd
parameter RES=7
V1  x   gnd vsource dc=10
R1  x     y resistor r=RES
I1  y   gnd NPORT r=RES*2
I2  y   gnd NPORT r=RES*2
model NPORT nport macro=yes evaluate="myres" setup="mysetup"
```

Both the I1 and I2 new elements are of type NPORT and its model declares that they are nport implemented as a macro=yes element and that its evaluate="myres" procedure implements it. The evaluate="mysetup" procedure handles the parameters of the NPORT instances, i.e., of I1 and I2. This automatically tells PAN that the MATLAB myres and mysetup exist. In this very simple example myres implements a linear resistor. The MATLAB code that implements the NPORT element is reported in Netlist 4.6.2.

Netlist 4.6.2 — MATLAB code of the element. – File: myres.m –

```
function [f,C,R]=myres(Params,PortNum,V,I,StNum,X,dX,Time)
    f(1) = V(1) - Params.r*I(1);
    C(1,1) =  1.0;
    R(1,1) = -Params.r;
end
```

The myres(Params,PortNum,V,I,StNum,A,dA,Time) function receives as input

- Params The structure of parameter items and values. In out case from the I1 and I2 instances we see that the model supports the "r" parameter. This per instance parameter is accessed through the statement Params.r. Note that parameters can have different values in different instances. In this our specific case they have the same value.
- PortNum the number of ports of the instance. This value can be used to check that each instance has a correct number of ports; each port is formed by a pair of terminals. In our example the I1 instance has two terminals and thus it is a one-port.
- V the array of port voltages. The number of elements of V is PortNum.
- I the array of port currents. The number of elements of I is PortNum.
- StNum Ignored.
- A Ignored.
- dA Ignored.
- Time the current simulation time, it has meaning only in time-domain analysis such as for example tran.

The characteristic of the new n-port element is implemented as an implicit set of equations, more specifically as the $f(V,I) = 0$ implicit equation. We have also to compute the $C = \frac{\partial f}{\partial V} \in \mathbb{R}^{N\times N}$ and $R = \frac{\partial f}{\partial I} \in \mathbb{R}^{N\times N}$ matrices where N is equal to PortNum. The myres procedure returns

- f the residue, it is an array of PortNum element.
- C the $\frac{\partial f}{\partial V}$ sensitivity matrix.
- R the $\frac{\partial f}{\partial I}$ sensitivity matrix.

The MATLAB code that implements the mysetup procedures that defines and handles parameters is reported in Netlist 4.6.3.

Netlist 4.6.3 — MATLAB code of the element. – File: mysetup.m –

```
function [params]=mysetup()
    params.r = 1.0;
end
```

It is very simple; it defined the "params.r" structure and item and sets the default value of the parameter.

Assume that we want to determine the value of the Res global parameter such that the power dissipated by the new element is maximum. To this end we use the fminbnd() optimisation function of MATLAB and write a cost function that modifies the value of r parameter of the instances, performs a DC analysis and computes the power dissipated by the new element. Note that the r parameter of instance I1 and I2 is varied in different ways through the PAN alter analysis.
The MATLAB code implementing the cost function is reported in Netlist 4.6.4.

Netlist 4.6.4 — MATLAB cost function. – File: cost.m –

```
function y = cost(x)
    pansimc(sprintf('Al1 alter instance=I1 param="r" value=%e', x ));
    pansimc(sprintf('Al2 alter instance=I2 param="r" value=%e', 2*x ));
    pansimc('DcOpt dc print=yes mem=["^y$"]');
    v = panget('DcOpt.y');
    y = -v*v/x;
end
```

The

```
pansimc('DcOpt dc print=yes mem=["^y$"]')
```

statement directly executes the pansimc MEX function and asks to perform the DcOpt DC analysis and save the voltage of the y node in memory (mem=["^y$"]). Finally the MATLAB script in Netlist 4.6.5 loads the myres.net PAN netlist and executes the fminbnd optimisation function.

Netlist 4.6.5 — MATLAB runing script. – File: pwropt.m –

```
pannet('myres.pan');
options = optimset('Display','on');
fun     = @(x)cost(x);
[rOpt,pwrOpt] = fminbnd(fun,0.1,20,options);
```

The obtained result is rOpt=10.5. The NPORT elements models a linear resistor of resistance Res thus the maximum power transfer theorem states that NPORT dissipates the maximum power when its resistance equals that of R1 in the schematic of Figure 4.2. Recall that resistence of I1 and I2 are varied with different strategies.
Experience shows that the execution of MATLAB functions from a MEX is not particularly efficient, therefore the implementation of new elements in MATLAB has to be done only for initial testing and development. When the new element works perfectly we can automatically generate the "C" code that implements the myres procedure through the CODEGEN command of MATLAB. This code can be suitably compiled in a shared library and linked in a very simple way to the NPORT model instead of the MATLAB procedure. This is described in Section 6.

4.7 Using PAN as a MATLAB function

The AMP1DAE.M file of MATLAB implements the MNA equations of the simple amplifier shown in Figure 4.3. This circuit was originally described in [**amp1dae**] at page 377. The output of the

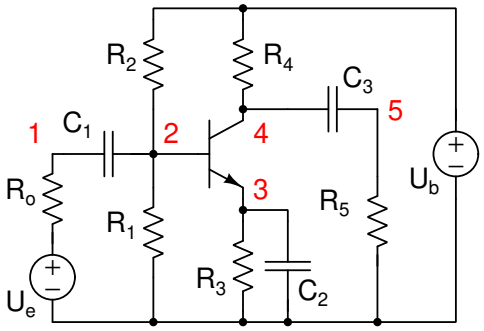

Figure 4.3: The schematic of the amplifier.

amplifier is at node 5, i.e. R_5 is the load resistor. The U_e voltage generator is the input od the amplifier. The AMP1DAE.M function is shown in Netlist 4.7.1.

Netlist 4.7.1 — MATLAB AMP1DAE.M script. – File: amp1dae.m –

```
function amp1dae
% Problem parameters
Ub = 6;
R0 = 1000;
R = 9000;
alpha = 0.99;
beta = 1e-6;
Uf = 0.026;
Ue = @(t) 0.4*sin(200*pi*t);  % Ue(t) = 0.4*sin(200*pi*t)
% The constant, singular mass matrix
c = 1e-6 * (1:3);
M = zeros(5,5);
M = [
-c(1)  c(1)     0     0     0
 c(1) -c(1)     0     0     0
   0     0   -c(2)    0     0
   0     0      0  -c(3)   c(3)
   0     0      0   c(3)  -c(3)
];
% Hairer and Wanner's RADAU5 requires consistent initial conditions
% which they take to be
u0 = zeros(5,1);
u0(2) = Ub/2;
u0(3) = Ub/2;
u0(4) = Ub;
% Perturb the algebraic variables to test initialization.
u0(4) = u0(4) + 0.1;
u0(5) = u0(5) + 0.1;
% Leave the 'MassSingular' property at its default 'maybe' to test the
% automatic detection of a DAE.
options = odeset('Mass',M);
[t,u] = ode23t(@f,[0 0.05],u0,options);

figure;
plot(t,Ue(t),t,u(:,5));
axis([0 0.05 -3 1]);
legend('input', 'output', 'Location', 'NorthWest');
title('One transistor amplifier DAE problem solved by ODE23T');
xlabel('t');
%
   function dudt = f(t,u)
      f23 = beta*(exp((u(2) - u(3))/Uf) - 1);
      dudt = [ -(Ue(t) - u(1))/R0
         -(Ub/R - u(2)*2/R - (1-alpha)*f23)
         -(f23 - u(3)/R)
         -((Ub - u(4))/R - alpha*f23)
         (u(5)/R) ];
   end
%
end
```

The target of AMP1DAE.M is to perform the time domain analysis of the amplifier circuit by using

the ode23t integration function and to show the output waveform at node 5. By observing the AMP1DAE.M script we see that $U_e(t) = 0.4\sin(200\pi t)$. The model of the BJT is a simple Ebers-Moll one with no junction capacitances. The dudt function at the end of the script implements the MNA equations; the f23 expression computes the current across a reference junction that is suitably scaled to model the base-emitter current and the collector-emitter current.
The circuit shown in Figure 4.3 can be easily implemented in PAN. The corresponding netlist is shown in Netlist 4.7.2

Netlist 4.7.2 — MATLAB runing script. – File: amp1daeGe.net –

```
ground electrical gnd
parameters UB=6 UE=0 R1=9k R2=9k R3=9k R4=9k R5=9k R0=1k

Ub    ub   gnd   vsource vdc=UB
r4    ub    n4   resistor r=R4
r2    ub    n2   resistor r=R2
r1    n2   gnd   resistor r=R1
r3    n3   gnd   resistor r=R3
Ue    ue   gnd   vsource vdc=UE
ro    ue    n1   resistor r=R0
r5    n5   gnd   resistor r=R5
#ifdef BC108
Q1    n4    n2   n3   n3  bc108
#else
Q1    n4    n2   n3   EB
#endif

#ifndef PAN
U1    n1   gnd   vsource vdc=0
U2    n2   gnd   vsource vdc=0
U3    n3   gnd   vsource vdc=0
U4    n4   gnd   vsource vdc=0
U5    n5   gnd   vsource vdc=0
#endif

.MODEL bc108 NPN IS =1.8E-14 ISE=5.0E-14 NF =.9955 NE =1.46 BF =400
+            BR =35.5 IKF=.14 IKR=.03 ISC=1.72E-13 NC =1.27 NR =1.005
+            RB =.56 RE =.6 RC =.25 VAF=80 VAR=12.5
+            CJE=13E-12 CJC=4E-12 VJC=.54 MJC=.33
+            TF =.64E-9 TR =50.72E-9

subckt EB c b e
parameters UF=26m BETA=1u ALPHA=0.99
Ibe   b  e     vccs func=(1-ALPHA)*BETA*(exp((v(b) - v(e))/UF) - 1);
Ice   c  e   b vccs func=ALPHA    *BETA*(exp((v(b) - v(e))/UF) - 1);
end
```

The Ebers-Moll model is implemented by the EB sub-circuit through two voltage controller current sources, the former connected to the base-emitter terminals, which generates the $(1-$ ALPHA$)$ current and the latter between the collector-emitter terminals, which generates the ALPHA current. In the netlist there is also the model of the famous bc108 transistor, note that this is a complex built-in model of PAN; it is not available in MATLAB. The bc108 transistor model is much more realistic than the simple Ebers-Moll; for example it accounts for the collector, base and emitter parasitic resistances.
Our target is to simulated the circuit and replicate the results by the AMP1DAE.M script of

MATLAB and after this to simulate the amplifier with the more accurate `bc108` model. The "link" between `PAN` and MATLAB is constituted by the `U1`,...,`U5` independent voltage sources. The `ode12t` integration function sets the value of the node voltages, i.e. of the `U1`,...,`U5` independent voltage sources and `PAN` gives back the current through these sources, i.e. how much the Kirchhoff current laws are violated at each node. From a more formal point of view `PAN` computes the vector field of the ordinary differential equation solved by MATLAB. This is done by the corresponding `dudt` function in the script Netlist 4.7.3

Netlist 4.7.3 — MATLAB runing script. – File: amp1daeGe.m –

```
function amp1daeGe
pannet('amp1daeGe.net -ad BC108');
% Problem parameters
Ue = @(t) 0.4*sin(200*pi*t);  % Ue(t) = 0.4*sin(200*pi*t)
% The constant, singular mass matrix
c = 1e-6 * (1:3);
M = zeros(5,5);
M = [
-c(1)  c(1)     0     0     0
 c(1) -c(1)     0     0     0
   0     0   -c(2)    0     0
   0     0      0  -c(3)   c(3)
   0     0      0   c(3)  -c(3)
];
% Initial conditions
u0 = zeros(5,1);
u0(2) = 6/2;
u0(3) = 6/2;
u0(4) = 6;
% Perturb the algebraic variables to test initialization.
u0(4) = u0(4) + 0.1;
u0(5) = u0(5) + 0.1;
%
options = odeset('Mass',M,'Reltol',1e-3,'Abstol',1e-4);
[t,u] = ode23t(@f,[0 0.05],u0,options);
figure;
plot(t,Ue(t),t,u(:,5));
axis([0 0.05 -3 1]);
legend('input', 'output', 'Location', 'NorthWest');
title('One transistor amplifier DAE problem solved by ODE23T');
xlabel('t');
%
function dudt = f(t,u)
pansimc(sprintf('Al1 alter instance=U1 param="vdc" value=%e', u(1)))
pansimc(sprintf('Al2 alter instance=U2 param="vdc" value=%e', u(2)))
pansimc(sprintf('Al3 alter instance=U3 param="vdc" value=%e', u(3)))
pansimc(sprintf('Al4 alter instance=U4 param="vdc" value=%e', u(4)))
pansimc(sprintf('Al5 alter instance=U5 param="vdc" value=%e', u(5)))
pansimc(sprintf('Ali alter instance=Ue param="vdc" value=%e', Ue(t)))
pansimc('Dc dc mem=["^U[1-5]:i$"] vreltol=1p vabstol=1f ireltol=1p iabstol=1f');
dudt(1,1) = -panget('Dc.U1:i');
dudt(2,1) = -panget('Dc.U2:i');
dudt(3,1) = -panget('Dc.U3:i');
dudt(4,1) = -panget('Dc.U4:i');
dudt(5,1) = -panget('Dc.U5:i');
end

end
```

Each time the dudt function is called, the alter statements of PAN set the U1,...,U5 independent voltage sources and the Ue value of the input source at the t time instant. The Dc analysis is performed and the currents through the mentioned sources are stored in the dudt vector. The amp1daege.pan netlist is loaded through the pannet command. If the amp1daeGe is executed with the

```
pannet('amp1daeGe.pan');
```

command the simple Ebers-Moll model is used; if it is executed with the

```
pannet('amp1daeGe.pan -ad BC108');
```

command the `BC108` directive selects the `bc108` model. The results obtained by simulating the amplifier with MATLAB and the `PAN` suite are shown in Figure 4.4

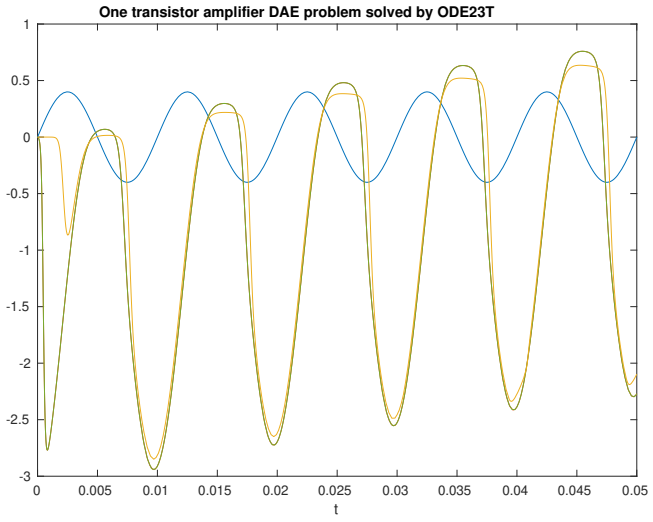

Figure 4.4: The voltage of the `5` node (output of the amplifier shown in Figure 4.3) and of $U_e(t)$ computed by MATLAB and by the `PAN` suite. Blue waveform: $U_e(t)$. Dark-green waveform: v(5) by `PAN` suite; this waveform completely hides the red one by MATLAB, i.e. there is a perfect overlap of the results. Orange waveform is the result obtained by using the BC108 model.

4.8 How to check `PAN` execution and errors

The successful execution of analyses by `PAN` can be checked through the `MPanerror` *global* variable of MATLAB. The MATLAB *global* variable must be defined before running an analysis. `PAN` checks that the `MPanerror` variable was defined and it raises its value to 1 if an error occurred. This allows to check possible failures. Note that the value of `MPanerror` must be reset to 0 in order to check further possible failures. An exemple is shown.

```
global MPanerror;
MPanerror = 0;
pansimc('Alter alter instance="R1" param="r" value=0');
if(MPanerror > 0)
        MPanerror = 0
%       An error occurred
end
```

The first and second lines set the `MPanerror`, then the `Alter` analysis is performed. If it fails the body inside the `if` block is executed.

4.9 Altering parameters

Values of parameters can be modified through the `alter` analysis invoked by the MATLAB interface. For example the

```
pansimc('Al alter param="X" value=3');
```

sets the value of the `"X"` parameter at 3. The `"X"` parameter must be previously defined in the netlist. There can be others parameters that depend on the value of `"X"`. For example consider the statement

```
parameters X=1 Y=X+3
```

defined in the netlist. When the netlist is parsed by `PAN`, we have `X=1` and `Y=4`. When the `'Al alter param="X" value=3'` analysis is invoked by the MATLAB interface we have `X=3`, but the value of `Y` is unaffected, i.e., we still have `Y=4` since the expression defining `Y` is not re-evaluated. To automatically update the expression and values of instance and model arguments when the value of `X` is altered we have to explicitly declare that derived expressions must be reevaluated any time the value of `X` is chaged, through the statement

```
Alr alter param="X" rt=true
```

This statement instructs `PAN` that the `"X"` parameter may be altered and that all the derived expressions must be updated. Therefore when from the MATLAB interfase the `'Al alter param="X" value=3'` alter analysis is executed all the depending expressions are reevaluated and parameters and instance/model arguments are updated. In our case we have `X=3` and mainly `Y=6`.

4.10 Saving data in .mat files

The simulation data, parameters and variables defined in the bodies of `control` analyses can be saved in *.mat* MATLAB files. Consider the simple circuit implemented in Netlist 4.10.1.

Netlist 4.10.1 — PAN netlist. – File: `mat5.net` –

```
ground electrical gnd
parameter RES=7

E1  x   gnd vsource dc=10
R1  x     y resistor r=RES
I1  y   gnd resistor r=RES
Dc dc instance=E1 start=1 stop=10 step=0.1 param="vdc"
Mat5 control begin
    Vx = get("Dc.x");
    save("mat5","vx.mat","Vx",Vx);
endcontrol
```

It implements a simple voltage partitioner, a DC sweep varies the magnitude of the `E1` independent voltage source. The `Mat5` control analysis collects the result of the sweep in the `Vx` vector. The

```
save("mat5","vx.mat","Vx",Vx);
```

statement save the data in the `"vx.mat"` file with the `"mat5"` format of MATLAB. The `Vx` control vector is saved as the `"Vx"` vector in the matfile. The `"vx.mat"` file can be later loaded in

MATLAB. Note that saving results in the *.mat* file firstly requires to store them in an array, i.e., `Vx`, and then in the *.mat* file. This is a *memory consuming* action that has to be carefully considered.

4.11 Accessing MATLAB variables from PAN

The variable of the calling MATLAB workspace can be accessed as arguments of analyses of PAN that are in control bodies and as variables of control bodies. These MATLAB variables can be read but not assigned. The order used to read analysis argument and control variables is: in case of analyses they are firstly searched among PAN parameters, then as control variables and lastly as MATLAB variables in the calling workspace. In case of control variables they are searched firstly as such then as PAN parameters and lastly as MATLAB variables in the calling workspace..

4.12 Executing control analyses and functions from MATLAB

The code of `control` analyses can be executed several times from the MATLAB interface. Resuls and control variables can be accessed through the `panget()` function (see Section 4.4). For example consider the code of the `X` simple control analysis

```
X control begin
        printf("Hello\n");
        z=[1,2];
endcontrol
```

From the MATLAB interface it can be executed through the statement `pansimc('X');` and the `z` vector can be read through the `mz=panget('z');` statement. The possibility to execute control functions from the MATLAB can be usefull since several analyses and features typical or peculiar of PAN can be organised in a control analysis and then easily executed.

A similar feature is applicable to functions defined in control analysis. For example consider the `S` control analysis that only defines the `Sum` function

```
S control begin
    function y=Sum(x)
        y = sum(x);
    end
endcontrol
```

The `Sum()` function can be executed from MATLAB through the statement `pansimc('z=Sum(xm)');` where `z` is saved as a PAN control variable that can be retrieved through the `mz=panget('z');` statement, where 'mz' is a variable created in the MATLAB current workspace. The 'xm' argument of the `Sum()` function is searched firstly as a control variable, then among the PAN parameters and finally among the variable of the calling MATLAB workspace. This means that the variables of the MATLAB workspace are accessed from the control analyses of PAN.

5. The Veriloga compiler

5.1 Veriloga compiler

The VERILOGA language constructs can be found in a manual (see for example [**VeriaRef**]). We thus report only the main differences and enhancements that are peculiar to its implementation in PAN.
The modules are described in several files; the "main" module is an n-port with input, output and input-output terminals. In the circuit netlist this module is linked to an n-port model. There can be several instances of the same model. For example assume to implement a linear resistor and to use this resistor with a conventional one of PAN to form a voltage partitioner. The Netlist 5.1.1 reports the implementation of the voltage partitioner.

Netlist 5.1.1 — The voltage partitioner. – File: partitioner.net –

```
ground electrical gnd
Dc dc print=yes
V1    x    gnd   vsource vdc=10
R1    x      y   RES G=1
R2    y    gnd   resistor r=1
model RES nport veriloga="res.va"
```

By observing the Netlist 5.1.1 we see the instance R1 of the RES object; its model is of nport type and the veriloga="res.va" parameter tells that the "res.va" module implements the RES model.
The code of the RES VERILOGA module is

Netlist 5.1.2 — The RES VERILOGA module. – File: `res.va` –

```
module RES(p,n);
inout p,n;
electrical p,n;
parameter real G=2;
analog begin
   i(p,n) <+ G * v(p,n); // Conductance is 2.
end
endmodule
```

The Netlist 5.1.2 is compiled, a shared library is generated and automatically linked by PAN at run-time. The set of parameters of the `nport` model which are useful in compiling the VERILOGA modules are:

verilogа The path/s of the file/s that must be compiled in order to implement a VERILOGA module.

verilogapp This is a boolean flag, when 'true' the output of the VERILOGA pre-processor is saved in a file named as the module followed by the ".pp" suffix.

verilogamodname This is the name of the VERILOGA module linked to the `nport` model. If this parameter is omitted the module with the same name of the `nport` model is looked for.

verilogaelectrical This is a boolean flag, when 'true' the `electrical` disclipline is automatically added among the disciplines, if it is not specified in the veriloga file. Note that the default value of this parameter is 'true'.

verilogashadowvar This is a boolean flag; when 'true' a warning is given when a local variable shadows a previous (global) one with same name.

verilogaprotected This is a boolean flag; when 'true' it turns-on the generation of code that is protected against main numerical exceptions such as for example division by 0. Protected code runs slowly with respect to the corresponding unprotected code. It is suggested to protect the code during its development and then to recompile it as unprotected, when it perfectly runs. PAN detects floating point faults during simulations, thus if in a veriloga module something goes wrong and a floating point exception is generated, PAN stops and shows the name of the instance that generated the fault. In these cases, it is suggested to recompile the code with the protection turned on. See also the `'compiler'` and `'cc'` parameters of the `'options'` card.

verilogatrace A list of variables defined in the veriloga module that have to be traced and written in the output file during simulations. Note that writing of these variables is effective only if activated through a specific option of the analysis. For example see the `devvars` argument of the `tran` analysis.

verilogamod This parameter is described in detail in the corresponding 5.1.1 sub-section.

verilogacase When 'true' this parameter tells the compiler to generate a veriloga module code that is not case sensitive (lower and upper case are indistiguisced) for what concerns module parameter names. Thus they will be accessible both on lower and in upper case.

Each VERILOGA module is compiled only once. If the code of a module is changed the VERILOGA module is recompiled. All modules can be forced to be recompiled through the `-fva` command line option when PAN is run.

5.1.1 The veriloga shared library

When the verilogmod="shared.so" parameter of the `nport` model is used in conjunction with the veriloga="vera.va" parameter, the veriloga model described in the "vera.va" file is compiled in the "shared.so" shared library and the "shared.mctx" context file is contemporary created. An example of the statement, which does this, is

```
model VERA nport verilogamod="shared.so" veriloga="vera.va"
```

The shared library and the context file allows the user to avoid the distribution of the original veriloga module source code. If the previous statement is substituted with

```
model VERA nport verilogmod="shared.so"
```

PAN automatically loads the shared library and the context file and does not needs to compile the veriloga source code. The user can thus distribute the shared library and the (binary) context file instead of the veriloga source code.

5.2 Veriloga language

5.2.1 Defines

PAN checks if defines are used in the corresponding VERILOGA module. If there are unused defines a warning message is give.
A define can be also given at command level, i.e., when the simulator is run. For example, the command

```
pan netlist.pan -k VeraDef
```

invokes PAN on the `netlist.pan` netlist and through the `-k` command line options the `VeraDef` VERILOGA define is defined. The `VeraDef` define acts in *each* VERILOGA module that is used. This is in contrast to defines defined in modules since they act only in the VERILOGA module where they are defined. Note that if a define is given at command line *all* VERILOGA modules are recompiled. Normally VERILOGA modules are compiled only once and subsequently only if their veriloga code is modified.
If the command line given `VeraDef` define is not used in any module, PAN displays a fatal error and the run aborts.

5.2.2 Parameters

All the VERILOGA parameters declared inside the body of a module can be accessed by the instance of the module inside the PAN netlist. Consider the Netlist 5.1.2, the statement

```
parameter real G=2;
```

defines the `G` real parameter of default value 2. In the Netlist 5.1.1 the instance

```
R1 x y RES G=1
```

of the `RES` model, which is a VERILOGA module, has the `G=1` parameter. This means that the defauls value of the parameter used in the VERILOGA module is changed to 1.

5.2.3 Assignment and reading of potential and flow variables

VERILOGA implements *potential* and *flow* unknowns. In case of an electric circuit, potentials are node voltages and flows are currents. Node voltages are accessed through the $v()$ function and currents through the $i()$ function. These functions can be used in the right hand side of expressions and can be assigned in th left hand side of expressions. Since PAN uses the MNA formulation to solve circuits, how the $v()$ and $i()$ functions are used in the right and left sides of expressins determines the structure of the algebraic (differential) problem to be solved.
Consider the VERILOGA statement

```
i(p,n) <+ G*v(p,n)
```

The $v(p,n)$ senses the voltage across the p and n nodes. This does not introduce any new equation in the MNA formulation. The assignment of the $i(p,n)$ current between the p and n nodes introduces and equivalent *voltage controlled current source* that is driven by $v(p,n)$ scaled by G. The $i(p,n)$ *adds* to the surrent snnken from the p nodes and sourced in the n node. Once more this does not introduce any new equation in the MNA formulation. What the above statement implements is a linear resistor. The statement

```
i(p,n) <+ G1*v(p,n)
i(p,n) <+ G2*v(p,n)
```

defines *two distinct* voltage controlled current sources; thus current sunked from the p node and injected in the n node is the *sum* of the two currents, that is, the above statement is equivalent to

```
i(p,n) <+ (G1+G2)*v(p,n)
```

Consider the VERILOGA statement

```
v(p,n) <+ i(p,n)/G;
```

The sensing of the $i(p,n)$ in the right-hand side of the expression requires the insertion of a *bad-branch* in the MNA formulation. It is required, for example but not exclusively, by an independent voltage generator. The $v(p,n)$ on the left-hand side of the expression allows the VERILOGA compiler to understand that the $i(p,n)$ current is that of the same port and thus the same bad-branch can be used to implement the statement, i.e. an implicit equation of a linear resistor formulated as a current controlled element and not as a current controlled element. This VERILOGA statement adds a new equation to the MNA formulation.

When possible it is better to implement characteristics of elements that do not add any implicit equation to the MNA formulation. In general, less is the number of new added equations and more efficient is the simulation.
Consider the statements

```
v(phi) <+ L*tanh(i(p,n));
i(p,n) <+ G*(v(p,n) - ddt(v(phi)));
```

The red $i(p,n)$ current is sensed in the right-hand side of the first expressions. It is also assigned in the second expression. Since it must be sensed, it introduces a bad-branch. $v(p,n)$ senses the voltage of the newly introduced bad-branch.
The VERILOGA compiler understands that the $(v(p,n), i(p,n))$ pair constitutes an implicit equation potentially involving both electrical variables.
The key aspect is that a bad-branch is introduced to sense the i(p,n) current and that its branch voltage must be assigned. In the above expression $v(p,n)$ is sensed (right hand side of the expression) but it is not assigned, i.e., it does not appear on the left hand side of any expression. In this case the compiler gives the warning message *"$v(p,n)$ is forced to 0"*. Thus the bad-branch implements a short circuit that completely changes the characteristic of the element implemented by the VERILOGA code. This can be avoided by removing the sensing and assignment of the same $i(p,n)$ current. The following VERILOGA code does this

```
real Curr;
analog begin
v(phi) <+ L*tanh(Curr))
Curr = G*(v(p,n) - ddt(v(phi)));
i(p,n) <+ Curr;
end
```

The VERILOGA code implements a multi-port voltage controlled current source. The output current is $i(p,n)$, the driving voltages are $v(p,n)$ and $v(phi)$. The `Curr` algebraic variable is the current that flows across the output port. A volutary imperfection is introduced in writing the code. During the compilation of the above statement, the compiler will complain that the `Curr` variable is used before being assigned and that this can cause convergence problems. Swaping of the statements as shown in the following VERILOGA code solves the problem.

```
real Curr;
analog begin
Curr = G*(v(p,n) - ddt(v(phi)));
v(phi) <+ L*tanh(Curr))
i(p,n) <+ Curr;
end
```

Assume that for any reason swap is impossible since effectively the use of the `Curr` variable introduces an implicit equation. In this case when the compiler complains, we suggest to add a new unknown to the problem, say x, that replaces `Curr`. This action breaks the algebraic loop. The new VERILOGA code is

```
electrical x;
analog begin
v(phi) <+ L*tanh(v(x))
v(x) <+ G*(v(p,n) - ddt(v(phi)));
i(p,n) <+ v(x);
end
```

Note that $v(x)$ appers on the both the right hand side of an espression, and is assigned (left hand side of an expression) thus a *new bad branch* is introduced. Thus this vanished the use of an algebraic VERILOGA variable to avoid the addition of a new bad branch and introduces an implicit equation that is handled and solved by the simulator.

5.2.4 ddt operator

The `y=ddt(x,abs,nature,ic,reset,reltol)` operator does the time derivative of the x expression. The resulting derivative is assigned to y. The derivative operator is based on the Gear integration method or order 2. Consider the t_n time instant and the $t_{n+1} = t_n + h_n$, $t_{n-1} = t_n - h_{n-1}$ ones, we have

$$y(t_{n+1}) = \frac{1}{h_n}\left(ax(t_{n+1}) + bx(t_n) + cx(t_{n-1})\right) \tag{5.1}$$

Since in (5.1) the $x(t_n)$ and $x(t_{n-1})$ previous values are needed an hidden internal state variable is used in the simulator. These hidden state variables are named with the "$stvar" prefix followed by an integer number. This integer number is incremented any time a new `ddt` instance is found. These hidden state variables are linked to an internal simulator node with the same name. This node is grounded by a "parassitic" conductance of 1 pS.
The meaning of the other parameters is:

`abs` the absolute value of the error tolerance used to control the derivation. If not given as argument that specified in the analysis that invokes the `ddt` operator is used.

`nature` this parameter is currently ignored.

`ic` this parameter is *not standard*. It is the initial value at which is set $x(t_0)$ at the beginning of the a time domain analysis. In this case the $x(t_0) - \texttt{ic} = 0$ equation is solved to assign the initial value. Note that x can be an expression and this can pose numerical problems or even lead to an equation that can not be solved. As an example consider the VERILOGA expression `y = ddt(cos(z),,,2);`. It does not admit a solution since `cos(z)` can not assume the 2 value. In this case the simulator will fail the initialisation.
Note that the `ic` argument is effective only in the time domain analyses where the initial conditions are used (through suitable paramaters, see for example the `uic` parameter of the tran analysis), otherwise this argument is ignored.

`reset` this parameter is *not standard*; when it is different from 0 the derivator is reset to its initial condition. The `reset` expression can be a manifold.

`reltol` the relative value of the error tolerance used to control the derivation. If not given as argument that specified in the analysis that invokes the `ddt` operator is used.

5.2.5 idt operator

The `y=idt(x,ic,reset,abstol,nature,min,max)` operator does the time integration of the x expression. The same integration method of the `ddt` operator is used, thus refer to it for more datails.
The meaning of the other parameters is:

`ic` this parameter is the optional initial condition. If not given the initial condition is 0. The `ic` parameter has meaning only in time domain analyses, if the options of these analyses are suitably set. For example if we have `uic=yes` in the TRAN analysis thus the `ic` parameter of the `idt` operator is effective. Note that if the `reset` parameter of the `idt` operator is 'true', the `ic` parameter is effective also in the DC analysis. If the `reset` is not active during the DC analysis, the value of the `y` output is automatically determined by the DC analysis. Note that in some cases the DC analysis shows convergence problems since the initial condition can not be determined. Therefore in this cases the user must explicitly set the `ic` initial condition. As an example, think of a floating output of a proportional-integral controller. The initial condition of the integrator can not be determined.

`reset` When this parameter is different from 0 the integral value is reset to `ic`. The `reset` argument can be a manifold.

`abstol` This parameter is ignored

`nature` This parameter is ignored.

`min` This parameter is optiona and sets the minimum value that can be assumed by the integral.

`max` This parameter is optiona and sets the maximum value that can be assumed by the integral.

5.2.6 idtmod operator

The `y=idtmod(x,ic,modulus,offset)` operator integrates the x expression with respect to time. The same integration method of the `ddt` operator is used, thus refer to it for more datails.
The meaning of the other parameters is:

`ic` this parameter is the optional initial condition. If not given the initial condition is 0. The `ic` parameter has meaning only in time domain analysis, if the options of these analyses are suitably set. For example if we have `uic=yes` in the TRAN analysis thus the `ic` parameter of the `idt` operator is effective. Note that if the `reset` parameter of the `idt` operator is 'true', the `ic` parameter is effective also in the DC analysis.

`modulus` Any time the absolute value of the x integral goes above the value of this parameter, the

integral in reset to 0.

`offset` is a constant value that is added to the integral, i.e. the integral is offset by this value.

5.2.7 cross and above functions

The `cross(expr,dir,tabs,eabs)` function returns a true boolean value when the value of the `expr` expression crosses 0. The direction of zero-crossing is specified by `dir`, the 0 value means any direction, `+1` means with positive derivative and -1 means with negative derivative. The `tabs` parameter governs the absolute time step resolution in zero-crossing. The time sampled `expr` crosses zero with two time-steps separated by a time interval not larger than `tabs`, one before zero-crossing and one after zero-crossing. `eabs` is the absolute error assumed by `expr` in zero-crossing. It is ensured that the values assumed by `expr` at the time steps before and after the zero-crossing are not larger in absolute value than `expr`. Both `tabs` and `eabs` can be specified or only one of those.
To better explain how the `cross()` function works we use the following example

```
v(o) <+ -10;
@(cross(v(x)-1,+1,1n)
    v(o) <+ 10;
```

and refer to the Figure 5.1, The idea is that the voltage at node `o` is set at $+10\,\mathrm{V}$ when the `v(x)`

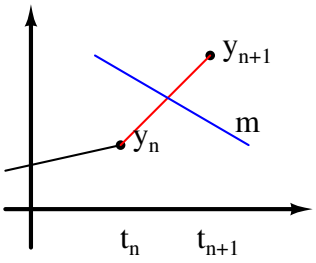

Figure 5.1: Trajectory crossing the `m` manifold between the t_n and t_{n+1} time instants.

voltage crossed the $+1$ threshold with positive derivative. Unfortunately, we will never see the `v(o)` voltage at $+10\,\mathrm{V}$, the VERILOGA code does not perform as we want. When the $+1$ threshold is crossed, `PAN` has computed the y_n solution at the t_n time instant and has also computed the y_{n+1} solution at t_{n+1}. The solution crosses the `m` manifold (a more generic version of the $+1$ simple threshold). Doing this the body of the `cross()` *was not executed*. `PAN` recomputes the solution at t_{n+1} and during the first and only the first iteration of the Newton method, that recomputes the solution, the body of the `cross()` function is executed. Note that the body of the `cross()` function is thus executed only once, per 0 crossing. This is the reason why the `v(o)` is not set at $+10\,\mathrm{V}$; in fact during the subsequent Newton iterations the `v(o)` voltage is not set again at $+10\,\mathrm{V}$, since the body of the `cross` function is no longer executed and the `v(o) <+ -10` VERILOGA statement resets the `v(o)` voltage. Keeping this is mind the correct statements are

```
real Out;
analog begin
@(initial_step) Out = -10;
@(cross(v(x)-1,+1,1n)
    Out = 10;
v(o) <+ Out;
end
```

We recall that the `Out` VERILOGA variable is an almost static variable, i.e., its value is persistent between two successfull subsequent executions of the VERILOGA module. This means that the value of `Out` at the t_n time instant is inherited at the t_{n+1} time instant. The `Out` VERILOGA variable is initialised at $-10\,\mathrm{V}$ at the beginning of the time domain analysis and thus the `v(o)` is set at $-10\,\mathrm{V}$. When the manifold is crossed, the y_{n+1} solution is recomputed and at the first iteration of the Newton method, the body of the cross function is executed; this sets the value of `Out` at $+10\,\mathrm{V}$. Since the `Out` VERILOGA variable is an almost static variable, its value is kept during all the subsequent Newton iterations since it is not further updated. At convergence the `v(o)` output will be set at $+10\,\mathrm{V}$, i.e., at the current value of the `Out` VERILOGA variable.
The `expr` expression of the `cross()` function automatically introduces a *manifold*; during analyses that need manifolds to correctly generate and handle *saltation matrices* to compute the state transition matrix, the integration time step is regulated to hit the manifold with the maximum possible time accuracy, i.e. integration time step is regulated to hit the 0 value of the `expr` expression with the maximum possible accuracy. This can slow down simulation runs; if manifolds are not needed the `above(expr,dir,tabs)` function can be used instead of the
tt cross() one. It acts in indentical way but does not handle manifold hitting.

5.2.8 The `timer()` function

The `timer(time,period,timetol)` function can generate a timer event at specified times during a simulation. Use the `timer` function to specify the `time` value when the simulator generates a timer event. When a time event is generated the body of the `time` function is executed. The meaning of the arguments are:

`time` is a expression specifying an initial time. The simulator places a first time step at, or just beyond, the `time` that is specified and generates a timer event. The `time` value can change during the execution, if the `timer` is already armed, new values may be dropped. This means that when the timer is armed by the first time, it ignores the other different values of `time` until when it expires.

`period` is optional. It is a dynamic expression specifying a time interval. The simulator places time steps and generates events at each multiple of period after `time`. This values is sampled just when the time is armed. Each time the `timer` expires the current value of `period` is picked and the `timer` is immediately re-armed.

`timetol` is optional. It is an expression specifying the maximum time window inside which the body of the just expired `timer` must be executed. If this argument is not given, as default the body is executed within a time window not greater that 20 times the minimum integration time step of the time domain analysis.

The module `squarewave`, below, illustrates how the timer function may be used to generate timer events. In `squarewave`, the output voltage changes from positive to negative or from negative to positive at each time interval of length `period/2`.

```
module squarewave(out);
output out;
electrical out;
parameter real period = 1.0;
integer x ;
analog begin
    @(initial\_step) x = 1 ;
    @(timer(0, period/2)) x = -x ;
    V(out) <+ transition(x,0.0,period/100.0);
end
endmodule
```

Note that the `timer` function checks the simulation evolution also after the event was generated, thus if it is enclosed in a conditional statement its behaviour can vary if the body of the condtional statement in which it is enclose it is not executed. For example, if the `timer` is periodic and the body it is not executed, it can expire without executing its own code body and rearming, as required being periodic. In general the `timer` function remains active for a few (2 or 3) time points after the timer activation.

5.2.9 $table_model function

The `$table_model(cntr_string,args,data)` allows to organise data in a multidimensional look-up table and to interpolate them. The `cntr_string` defines how the look-up table and data are organised. This string is formed by comma separated substrings. There must be as many substrings as the dimentionality of the look-up table. For example in case of a one-dimensional look-up table the substring and the given string coincides. Each substring consists of 3 digits. When the look-up table is formed by *control* variables, the first character of the substring related to this variable, is 'C'; the following 2 characters specify how the look-up table is continued when its entries go outside its definition interval. The first of these two characters determines how the table is continue below the lower bound of its entry definition, the second of these characters determines how the table is continued above the upper bound of its entry definition. Available options are: 'C' which means *continue* and 'E' which means *error*, i.e. an error is issued if the entry of the table goes above or below its definition interval. For example in a two-dimensional table, where data come from *control* variables, the "CCC,CEE" string tells that the table is continued if the first dimension goes below and above its definition interval, and that instead an error is generated if the second dimension goes outside the definition interval.
The `args` argument is a set of comma separated query arguments. These query arguments must equal the number of sub-strings in `cntr_string` (number of dimentions of the table).
The data argument can be a list of comma separated strings, each specifying the name of a variable defined in a *control* block. The number of these variables must equal that of the query arguments in `args`, plus one. The last argument in data is the variable containig the data of the table. In an n-dimensional table with $n > 1$, the entries in this last variable are organised by considering that the fastest varying dimension is the right most argument is `args`. In a 2-dimensional table the entries in this last variable can be organised as a (rectangular) matrix where the row index corresponds to the index of the first query vector, i.e., the first arguement in `args` and the column index corresponds to the index of the entry of the second query vector, i.e., the second arguement in `args`. To better describe how table data can be organised, consider the following statement

```
$y=table_model("CCC,CCC",arg1,arg2,"Dim1","Dim2","Table");
```

The data argument is composed of the `"Dim1(1)"` and `"Dim2(1)"` and `"Table"` vectors. `"Dim1"` stores the access values of the first dimention of the table; the `arg1` values of the first query variable are compoared to the entries in `"Dim1"`. `"Dim2"` stores the access values of the second and last dimention of the table. The first entry of the `"Table"` vector stores the table value computed at `"Dim1(1)"` and `"Dim2(1)"`; the second entry that at `"Dim1(1)"` and `"Dim2(2)"`; the n-th entry that at `"Dim1(1)"` and `"Dim2(n)"`; the (n+1)-th entry that at `"Dim1(2)"` and `"Dim2(1)"`, and so on. The number of entries in `"Table"`, i.e., the vector storing the table values, must be equal to the product of the number of entries in the `"Dim1"` and `"Dim2"` vectors, i.e. the entries of the `args` arguments.

5.2.10 The $stop() and $finish() functions

The `$stop(string)` function suspends the execution of an analysis and gives the control back to the keyboard. When the keyboard is closed for example by the cntrl-D special character the execution continues. The `string` argument is optional and it is displayed when the `$stop(string)` command is executed.
The `$finish(string)` function aborts the execution of an analysis and generates and error that is propagated to the calling statements, if any. The `string` argument is optional and it is displayed when the `$finish(string)` command is executed.
See also the non-standard `$interrupt(string)` function.

5.2.11 Real and integer number representation

VERILOGA considers all numbers in expressions as real. This rule does not apply to expressions that governs integer indices used in loop statements such as for example "for". As an example consider the VERILOGA code

```
real V1, V2, V3;
V1 = 2/3     * V3;
V2 = 2.0/3.0 * V3;
```

Both expressions give the same result. In the first one the "2" constant number is considered as real. The same rules of the "C" programming language are followed when mixed integer and real values in the same expression. Conversion of an integer value to a real one is performed by the "toreal()" function. Conversion from a real value to an integer one is performed by the "tointeger()" function.

5.2.12 Variables and Arrays

VERILOGA variables can be real and integer arrays; their minimum and maximum allowed indices are specified in their definitions. Arrays can be multidimensional, i.e. they can be accessed through a set of indices. For example

```
real X[1:10];
real Y[1:10][5:8][0:-20];
```

defines the "X" array of 11 real elements, where the lower index value must be no less than 1 and the upper index value must be no larger than 10. The "Y" array is accessed through a triplet of indices, the second index must be no less that 5 and no greater than 8. The third index must be no greater than 0 and no less than -20.
Parameters can be used in defining minimum and maximum indices.
Variables and arrays are *static*, their values are not cleared among subsequent executions of analyses. Their values are set to 0 when the netlist is loaded and parsed.
If values of parameters used in defining the size of arrays at their declarations are modified by a subsequent `alter` action, the original size of the arrays is modified accordingly. In this case all the entries of the multidimensional arrays are reset at 0.

5.3 Handling of simulator time step

`$abstime` is the value of simulation time at which the o current solution is computed during a time domain analysis.

`$timestep` is the value of the current integration time step.

`$prevtime` is the time instant at which the solution just before `$abstime` was computed.
By referring to expression (5.1), we have `$abstime` $= t_{n+1}$, `$prevtime` $= t_{n-1}$ and thus `$prevtime` + `$timestep` $= t_{n+1}$.

`$mintimestep` is the value of the minimum allowed time step in a time domain analysis.

`$bound_maxstep` is the value of the maximum allowed time step in a time domain analysis.

`$bound_singlestep` is the value of the maximum allowed *current* or *next* time step in a time domain analysis. After the time step is used, the simulator ignores this directives and uses the maximum time step length set by the running time domain analysis statement.

`$reject_step` the solution at the current time step in a time domain analysis is rejected and the Newton method recomputes the solution. If the integration time step has to be shortened, this action must be explicitly done by the user through the `$bound_maxstep` or `$bound_singlestep` commands.

`$break_point(time)` force the simulator to compute the solution at the `time` instance. The simulator can reject the solution and roll time back; furthermore minimum time step limitation and round-off errors can cause tiny variations of the `time` value and in its hitting. In the following veriloga code these an example of the `$break_point(time)` function usage. The target is to switch the `Vout1` value from 0 to 1 in the time window between `TBRK1` and `TBRK1 + TSTEP`. The code handles the fact that the simulator can reject the time point and roll time back.

```
module BRK(out1,out2);
input out1, out2;
electrical out1, out2;
parameter real TBRK1 = 1  from (0:inf); // Time instant where 'v(out1)'
                                        // must change
parameter real TBRK2 = 1  from (0:inf); // Time instant where 'v(out2)'
                                        // must change
parameter real TSTEP = 1u from (0:1);   // Time step limitation
integer Vout1, Vout2;

analog begin
    @(initial_model)
    {
        Vout1 = 0;  // Initial 'v(out1)' value
        Vout2 = 0;  // Initial 'v(out2)' value
        $break_point( TBRK1 ); // Insert a 'breakpoint', it forces the
                               // simulator to compute the solution at this
                               // very time instant.
    }
    if( $prevtime >= TBRK1 - 10*$mintimestep && Vout1 <= 0.5 )
    {
        $display("Time: %e\n", $prevtime );
        $bound_singlestep(TSTEP); // Limit the maximum integration time step
                              // at this time instant; the next solution
  // will be computed at TBRK1 + TSTEP.
        Vout1 = 1; // Update the output value.
    }
    if( timer( TBRK2 ) && Vout2 < 0.5 )
        Vout2 = 1;

    v(out1) <+ Vout1;
    v(out2) <+ Vout2;
end
endmodule
```

In the same veriloga code the `Vout2` variable is switched from 0 to 1 in the time window at `TBRK2` with the minimum allowed simulator time step length. This is much more easily

accomplished by the `timer()` function.

5.4 Non standard statements and functions

`$gmin` This variable variable stores the value of the gmin conductance swept by `PAN` when the gmin-stepping continuation method is activated. The user can use `$gmin` in developing her/his veriloga code to aid the simulator to better converge to a solution of the circuit. `$gmin` is set to 0 when the simulator checks the accuracy of the solution agains the insertion of the `$gmin` parasitic conductances.

`$initialgmin` The initial value of the parasitic conductance employed in non-linear devices to help convergence. This value is not swept and determines the initial value of gmin used at the begining of the analysis. The `$initialgmin` value is never set to 0.

`$pwrlevel` This variable stores the current level of power generation that has to be set during the application of the power-stepping continuation method by `PAN`. The user can thus vary the level of generated power in veriloga models that are employed in simulating power sub-networks. As an example, consider the veriloga code reported in Netlist 5.4.1.

Netlist 5.4.1 — The DCDC VERILOGA module. – File: `dcdc.va` –

```
module DCDC(pi,ni,po,no);
inout pi,ni,po,no;
electrical pi,ni,po,no;
parameter real P = 100M;
real Ii, Io;

analog begin
    Ii = $pwrlevel*P / max(10k,v(pi,ni));
    Io = $pwrlevel*P / max(10k,v(po,no));
    i(pi) <+  Ii;
    i(ni) <+ -Ii;
    i(po) <+ -Io;
    i(no) <+  Io;
end

endmodule
```

It implements a DC/DC converter that transfer the fixed P power from the first port (pi,ni) to the second port (po,no). The P power is multiplied by the `$pwrlevel` variable. If the power-stepping continuation method is activated by `PAN` the P power transferred by the DC/DC converter is suitably modified. The DC/DC converter thus partecipates to the continuation method. This can aid finding a solution of the circuit in difficult cases.

`$srclevel` This variable stores the current magnitude of sources that has to be set during the application of the source-stepping continuation method by `PAN`. The user can thus vary the magnitude of sources in veriloga models.

`$cntrsetvar ("varname",value,idx1,idx2)` This function stores data in *control* variables, arrays and matrices. For example the statement

```
$cntrsetvar("Vx", v(p,n) );
```

writes the `v(p1,n1)` scalar voltage in the `Vx` scalar control variable. If the `Vx` variable does not exist yet, it is created. This example shows that the `idx1` and `idx2` arguments

are useless in this case.
Consider the veriloga statement

```
$cntrsetvar("Va", v(p,n), 1 );
$cntrsetvar("Va", $abstime, 2 );
```

It writes the `v(p,n)` voltage in the first entry of the `Va` array and the `$abstime` value in the second entry of the `Va` array. If the `Va` array does not exist, it is created. This example shows that the `idx1` (third) argument of the `$cntrsetvar` function specifies the index of the entry of the array that stores the data.
Consider the veriloga statement

```
$cntrsetvar("Vm", v(p,n), 1, 1 );
$cntrsetvar("Vm", $abstime, 2, 2 );
```

It writes the `v(p,n)` voltage and the `$abstime` values in the `Vm` matrix. If the `Vm` matrix does not exist, it is created. This example shows that the `idx1` (third) and `idx2` (fourth) arguments of the `$cntrsetvar` function specify the indices, i.e. the row and column of the entry that stores the data.

`$cntrgetvar ("varname",idx1,idx2)` This function gets data from *control* variables, arrays and matrices. For example the statement

```
Vy = $cntrgetvar("Vx");
```

gets the value of the `"Vx"` control scalar variable and stores it in the `Vy` veriloga variable. The statement

```
Vy = $cntrgetvar("Ax",idx1);
```

gets the value stored in the `idx1` entry of the `"Ax"` control array and stores it in the `Vy` veriloga variable. Similarly the statement

```
Vy = $cntrgetvar("Mx",idx1,idx2);
```

gets the value stored in the `idx1,idx2` entry of the `"mx"` control matrix and stores it in the `Vy` veriloga variable. In this case `idx1` is the row index and `idx2` is the column index of the `"mx"` control matrix.

`$cntrevaluate` function allows to execute the control statement given as its argument. For example the statement

```
Vy = $cntrevaluate("MyFunction(Ax)");
```

executes the `"MyFunction(Ax)"` command that in this specific case is the execution of the `MyFunction` control function with the `Ax` argument.

`$frewind(arg)` This function rewinds the 'arg' file.

`break` statement plays the same role of the "C" same statement. It allows to interrupt and exit loops for example such those buit by `for` and `while`.

`toreal(arg)` This function converts the "arg" value in a "real" value.

`tointeger(arg)` This function converts the "arg" value in an "integer" value.

`dropd(arg)` This function drops derivatives of the "arg" expression with respect to problem unknowns, i.e., in an electrical environment voltages and currents. For example assume to use the implicit Euler method to implement the companion network of a capacitor. The constitutive equation of the linear capacitor is $\imath_c(t) = C\dfrac{dv_c(t)}{dt}$, where $\imath_c(t)$ is the capacitor current, C is capacitance and $v_c(t)$ is the capacitor voltage. The application of the implicit Euler integration method transforms the differential constitutive equation in the algebraic one $\imath_c(t_{n+1}) = \dfrac{C}{h_{n+1}}\left(v_c(t_{n+1}) - v_c(t_n)\right)$, where $v_c(t_{n+1})$ is the voltage at the current t_{n+1} time point of the analysis, $v_c(t_n$ is the voltage at the *previous* time point and h_{n+1} is the current integration time step. The differential conductance is $g_c = \dfrac{\partial \imath_c(t_{n+1})}{\partial v_c(t_{n+1})}$. To tell the simulator that derivatives of $\imath_c(t_{n+1})$ must be computed *only* with respect to $v_c(t_{n+1})$ we use the `dropd()` function in this way:

```
i(p,n) <+ C/$timestep*(v(p,n)-dropd(V[1]);
```

where `v(p,n)` is the voltage at the current time point and `V[1]` is the voltage at the previous time step. The complete code implementing the capacitor is

Netlist 5.4.2 — The capacitor VERILOGA module. `module CAP(p,n);`

```
input p,n;
electrical p,n;
parameter real C=1;
real PrevTime; // Stores previous time instant
real V[0:1];   // Stores current and previous voltage values
analog begin
    @(initial_step) begin // First step of time domain analysis
        PrevTime = $prevtime;
        V[1] = v(p,n);
        V[0] = V[1];
    end
    if( analysis( "tran" ) ) begin // Time domain analysis
        if($prevtime > PrevTime + $mintimestep * 0.1) begin // Time advances
            V[1]     = V[0];      // Update 'previous' solution.
            PrevTime = $prevtime; // Advance previous time instant
        end
        V[0] = v(p,n);
        i(p,n) <+ C * (V[0] - dropd(V[1])) / $timestep; // Implicit Euler
    end
    else
        i(p,n) <+ 0.0;
end
endmodule
```

`$interrupt(string)` This function stops the execution of the current analysis without generating any error. The `string` parameter is options; if given the string is displayed when the `$interrupt(string)` command is executed.

5.5 Special formats

`%A` when used in the `$display()` function, this format prints the name of the running analysis.
`%I` when used in the `$display()` function, this format prints the name of the veriloga intance.

5.6 Control functions

Functions defined in control blocks can be executed by VERILOGA modules.

5.7 The `analysis()` function

Use the `analysis` function to determine whether the current analysis type matches a specified type. By using this function, you can design modules that change their behavior during different kinds of analyses or during different phases of the same analyses. The `analysis()` function *was extended* by adding the some new keywords; all keywords are desribed in the following.

restart allows to check if an analysis is started from scratch or if it is started from results by a previous analysis. The `analysis("restart")` returns *true* is the current analysis is started from scratch and *false* if it is started from results by previous analyses. This helps for example to set initial values of variables according to the fact that the current analysis is the first one or follows others. Consider the VERILOGA module

Netlist 5.7.1 — The MPPT VERILOGA module. – File: `mppt.va` –

```
module MPPT(clk,vpv,ipv,out);
input clk,vpv,ipv;
output out;
electrical clk,vpv,ipv;
electrical out;
parameter real VDIG  = 1;
parameter real VSTEP = 0.36;
parameter real VREF  = 12;
real PwrNew, PwrOld;
real Sign;
real VrefNew, VrefOld;

analog begin
    @(initial_model) begin
        if( analysis( "restart" ) begin
            Sign = 1;
            VrefNew = VREF;
            VrefOld = VREF;
        end
    end
    @(initial_step) begin
        if( analysis( "tran" ) && analysis( "restart" ) begin
            PwrOld = v(vpv)*v(ipv);
            Sign = 1;
            VrefNew = VREF;
            VrefOld = VREF;
        end
    end
    @(cross( v(clk) - VDIG/2, +1, 1n )) begin
        PwrNew = v(vpv)*v(ipv);
        if( PwrNew < PwrOld ) Sign = -Sign;
        VrefNew = VrefOld + Sign * VSTEP;
    end
    @(cross( v(clk) - VDIG/2, -1, 1n )) begin
        PwrOld  = PwrNew;
        VrefOld = VrefNew;
    end
    v(out) <+ VrefNew;
end
endmodule
```

that implements a perturb&observe maximum power point tracker. Each time there is the `clk` clock transition, i.e. the clock signal has a rising front that goes above `VDIG/2` the body of the `@cross()` functions is executed. The `PwrOld` previous power value is compared with the `PwrNew` current value of power. Power is the product of the `vpv` voltage and `ipv` current. Note that `ipv` is seen as an input voltage to the `MPPT` module, i.e. current is converted to voltage at the input port of the `MPPT` instance. The sign of the `VrefNew` output reference voltage of the `MPPT` is changed according to the comparison of the new and old values of sampled power. The `PwrOld` value is updated at the falling front of the clock waveform. What matters in this context is the `analysis("restart")` statements. Some variables of the modules are set at the initial step in time domain analyses only if these analyses are started from scratch, i.e. the `analysis("restart")` returns a true value.

dc A DC analysis is running.

dcinit The initial guess, used by the Newton method of a DC analysis, is running. The initial guess is computed just before starting a DC analysis. When the computation of the initial guess is complete, the DC runs (several) Newton iterations to compute the solution.

dcop The operating point, i.e. the working condition, of elements is computed. For example charges of capacitors are computed toghether with their derivatives, that is capacitances. An operating point analysis is almost always executed at the and of a *dc* analysis or it can be requested during the execution of some time domain analyses.

ac A linear phasor analysis is running.

pz A pole/zero analysis is running.

tran A time domain analysis is running.

trnwconv A Newton iteration at a time point of the time domain analysis, not necessarily a TRAN analysis, converged to a solution. This information can be usefull to update some variables in verilоga module that has to vary in a discrete way from one time point to the subsequent one of the time domain analysis.

shooting A `shootinf` periodic time domain analysis is running or a `pac` periodic small-signal analysis is running. Note that the `pac` analysis can run only after a `shooting` analysis and uses results by it.

5.8 The `initial_model` event variable

The `initial_model` event variable is a *nonstandard extension* of the VERILOGA language. It is triggered by each analysis when it initialises the instances in the netlist just before performing the analysis itself. A usage example is shown in Netlist 5.7.1. For example when a `tran` analysis is performed and instances are initialised the `initial_model` is triggered. Then in a `tran` analysis the initial conditions are determined with the body of the `initial_model` already executed and later the first time step of the time domain analysis is performed. Execution of the first time step triggers the `initial_step` event that for example can be used to perform further actions at the beginning of the `tran` analysis.

5.9 The `new_step` event variable

The `new_step` event variable is a *nonstandard extension* of the VERILOGA language. It is triggered by each time domain analysis when it steps forward in time to a new time point. When this event is triggered the code of its body is executed only once, i.e, just when the time domain analysis steps forward.

5.10 Manifolds

The manifold keyword implements a VERILOGA non-standard statement. A manifold is described by a boolean expression involving electrical quantities, i.e., circuit voltages and currents. These expressions define hyper-planes in the solution space can can be crossed by the trajectory described by the solution in the time domain. The boolean expression is ‘true’ or ‘false’ according that the trajectory is on one side or the other of the manifold. A manifold can be the clause of the ’if’ VERILOGA statement. When the clause is true a portion of the VERILOGA code is executed, when it is false another portion is executed. This can make the vector field of the DAE describing the circuit *discontinuous*. Note that discontinuities are due to the “decisional” process introduces by the ’if’ and not by the fact that we use the ‘manifold’ as the ‘if’ clause. In the shooting analysis the vector field is used in deriving the sensitivity of the circuit solutions with respect to the initial conditions. The sensitivity matrix is a key aspect in deriving the variational

model of the circuit. Recall that the variational model is exploited in computing periodic noise. In principle discontinuities in the vector field does not allow the derivation of the sensitivity matrix. The sensitivity matrix can be regularised by introducing "ad-hoc" matrices referred to as *saltation matrices*, when the vector field undergoes discontinuities. Saltation matrices are computed by exploiting manifolds. Therefore by declaring that a VERILOGA 'if' clause is a manifold, this allows the introdution of a saltation matrix, that regularises the sensitivity matrix

The use of manifolds in the VERILOGA code is introduced through the circuit example shown in the Figure 5.2, that we anticipate implements an oscillator. The Netlist 5.10.1 implements the

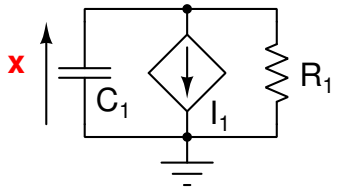

Figure 5.2: The schematic of the simple oscillator.

simple oscillator.

Netlist 5.10.1 — The simple oscillator model. – File: `crossOsc.net` –

```
ground electrical gnd
options digits=12

Tr tran stop=10 uic=1 devvars=1
Sh shooting period=4 autonomous=1 restart=0 floquet=1 annotate=3 \
            acntrl=3 solver=1
Pn pnoise start=1u stop=10000m lin=100 onodes=["x"]

C1   x    gnd  capacitor c=1 ic=0
R1   x    gnd  resistor  r=10
I1   x         NL
model NL nport veriloga="nl.va"
```

The characteristic of the `I1` voltage controlled generator is the key aspect to make the simple circuit work as an oscillator. The VERILOGA code of the `NL` nport is reported in 5.10.2.

Netlist 5.10.2 — The NL VERILOGA module. – File: `nl.va` –

```
module NL(x);
inout x;
electrical x;
real Out;

analog begin
    @(initial_model) begin
        if( v(x) >= 1 )          Out =  1;
        else if( v(x) <= -1 )    Out = -1;
        else                     Out =  1;
    end
    @(cross(v(x) - 1, +1, 1n)) Out =  1;
    @(cross(v(x) + 1, -1, 1n)) Out = -1;

    i(x) <+ Out;
end
endmodule
```

In the VERILOGA code we see the definition of the two `mu` and `ml` manifolds.

```
manifold mu: v(x) - 1 > 0;
manifold ml: v(x) + 1 < 0;
```

These two manifolds are used in two distinct `cross` VERILOGA functions

```
@(cross(mu,+1,1n)) Out =  1;
@(cross(ml,+1,1n)) Out = -1;
```

The `cross` function implements a "decisional" action. When its argument becomes positive/negative, i.e., crosses a manifold, the body of the `cross` function is execute. In our specific case, when the `mu` manifold is crossed, i.e., `v(x)` becomes greater than 1, the body of the first `cross` function is executed and the `Out` variable is set at 1. Similarly when the `ml` manifold is crossed, i.e., `v(x)` becomes less than -1, the body of the second `cross` function is executed and the `Out` variable is set at -1. The `Out` VERILOGA variable sets the current of the controlled current source `I1` in the circuit shown in Figure 5.2. All this means that when `vx` across the `C1` capacitor hits the `mu` manifold, the `I1` controlled current source sinks $+1$ A current, discharging the `C1` capacitor, and when `vx` hits the `ml` manifold, `I1` delivers $+1$ A current, charging `C1`. This charging/discharging action cycles in time giving rise to a period `v(x)` waveform. In the Netlist 5.10.2 the card

```
Tr tran stop=10 uic=1 devvars=1
```

performs a time domain analysis using the initial conditions given by the user. The following shooting analysis computed the steady state solution

```
Sh shooting period=4 autonomous=1 restart=0 floquet=1 annotate=3 \
            acntrl=3 solver=1
```

The computation of the Floquet multiplies is requested. It can be easily understood that if different initial conditions of the capacitor voltage are used, we will obtain solution of identical shapes but shifted in time. The trajectory will reach the `mu` or `mu` leading/lagging. This property of the non existence of a reference trajectory, it captured by the Floquet eigenvalue that is equal to 1. Since the circuit in Figure 5.2 implements an oscillator we expect that `PAN` computes a Floquet

multiplier equal to 1. The Floquet multipliers are derived from the sensitivity matrix, thus its correct computation through the insertion of the *saltation matrices* is an important aspect.

6. Notes on circuit elements

In this chapter some non-conventional built-in models and elements are described in detail.

6.1 The nport element

The `nport` element and model has different meaning:

case "a" It implements the nonlinear characteristic of a nport. In this case instances are not associated to any model;

case "b" The model associated to the instance collects information about the VERILOGA model that implements the `nport`;

case "c" The model collects information about the MATLAB function that implements the `nport`; This case is described in detail in Chapter 4.

case "d" The model collects information about the function in a `control` body that implements the `nport`;

case "e" The model collects information about the shared library, i.e. the compiled source code that implements the `nport`.

Thus the `n-port` and its model can link `PAN` to peculiar elements that were implemented outside. To better describe the different implementation of the nport we use the circuit shown in Figure 6.1, where the two controlled sources implement the model of an ideal transformer with a voltage ratio equal to n.

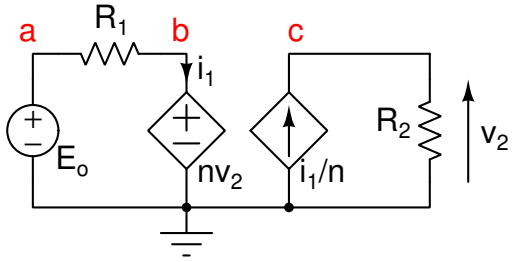

Figure 6.1: The test circuit used to implement several version of a nport.

6.1.1 Case "a"

Netlist 6.1.1 — Nport netlist. – File: `idealtransfA.net` –

```
ground electrical gnd
parameter N=sqrt(10)

Dc dc print=yes
Eo  a    gnd   vsource vdc=10
R1  a      b   resistor r=10
Np  b    gnd   c   gnd  nport func1=v(p1)-N*v(p2) func2=i(p1)*N+i(p2)
R2  c    gnd   resistor r=1
```

The `Np` nport linear characteristic is implemented through the two `func1` and `func2` implicit functions. There must be an implicit function per port of the nport. A port is composed of a pair of terminals, therefore the nport must have an even number of terminals. In this case the characteristic of the voltage controlled voltage source at the first port (left one in Figure 6.1) is described by the $v(p1) - N\,v(p2) = 0$ implicit function (`func1`), where $p1$ identifies the first port and $p2$ the second port and N is the voltage ratio. The characteristic of the current controlled current source at the second port (right one in Figure 6.1) is described by the $N\,i(p1) + i(p2) = 0$ implicit function (`func2`). Note the positive direction of the $i(p2)$ current, `PAN` adopts the *normal convention* voltage/current of ports.

6.1.2 Case "b"

An example of a nport model implemented through VERILOGA was already given in the Netlist 5.1.1. In this case a model must be introduced and thus the `Np` element must be an instance of the nport model. The Netlist 6.1.1 is modified accordingly in Netlist 6.1.2.

Netlist 6.1.2 — Nport netlist. – File: `idealtransfB.net` –

```
ground electrical gnd
parameter N=sqrt(10)

Dc dc print=yes
Eo  a    gnd   vsource vdc=10
R1  a      b   resistor r=10
Np  b      c   NPORT N=N
R2  c    gnd   resistor r=1
model NPORT nport veriloga="nportTr.va"
```

The `Np` instance has only two terminals. The code of the `NPORT` Veriloga module is

Netlist 6.1.3 — The `NPORT` Veriloga module. – File: `nportTr.va` –

```
module NPORT(b,c);
inout b,c;
electrical b,c;
parameter real N = sqrt(10);
analog begin
    v(b) <+  N*v(c);
    i(c) <+ -i(b)*N;
end
endmodule
```

The voltages and currents of the VERILOGA module involve only one terminal and thus the second implicit terminal of each port is the hidden *ground*.

6.1.3 Case “d”

PAN implements a subset of the MATLAB/OCTAVE language that can be used to write functions and algorithms that elaborate the results by the simulator or that control the behaviour of the simulator. These code sections are considered as the conventional analyses and are executed as such. These peculiar analyses are referred to as *control*. A new element can be implemented as a MATLAB/OCTAVE function in a control block. The Netlist 6.1.4 implements the circuit shown in Figure 6.1 where the electrical characteristic of the new element named `Trans` is described in the `Elm` control block.

Netlist 6.1.4 — Nport netlist. – File: `idealtransfC.net` –

```
ground electrical gnd
parameter N=sqrt(10)

Elm control begin
function [f,C,R]=Trans(Params,PortNum,V,I,StNum,X,dX,Time)
    f(1) = V(1) - N*V(2);
    f(2) = I(2) + N*I(1);
    C(2,2) =  0.0;
    C(1,1) =  1.0;
    C(1,2) = -N;
    R(2,1) =  N;
    R(2,2) =  1;
end
endcontrol

Dc dc print=yes
Eo  a    gnd   vsource vdc=10
R1  a      b   resistor r=10
Np  b    gnd   c   gnd   NPORT
R2  c    gnd   resistor r=1
model NPORT nport macro=yes evaluate="Trans"
```

By observing the `model NPORT nport` card in the Netlist 6.1.4 we see that the `macro=yes` parameter tells that the model is described as an “external” module. The `evaluate="Trans"` parameter tells that there exists the `"Trans"` procedure that implements the element model. PAN looks for this procedures in control blocks. This procudure must have a fixed number of input and output arguments. Note that the code of this procudure is interpreted at run-time and thus it is not particular efficient.

6.1.4 Case “e”

PAN supports the implementation of a new element as an external shared library that is linked at run time. The external shared library can be generated by starting from any programming language that can be compiled and linked in a shared library. As an example we use the same circuit and elements of the previous cases, but in this case the element is implemented as a set of “C” functions. The `NPORT` element in the Netlist 6.1.5 is tagged as `macro=yes`, the external function that evaluates the new element is ‘‘`Trans`’’ and the function that sets up the parameters of the new element is ‘‘`TransSetUpParams`’’. The function that handles parameters is not mandatory. The shared library that contains the compiled and linked function is ‘‘`nportTr.so`’’.

Netlist 6.1.5 — Nport netlist. – File: idealtransfD.net –

```
#system "gcc -shared -fPIC -o nportTr.so nportTr.c"
ground electrical gnd

Dc dc print=yes
Eo  a    gnd   vsource vdc=10
R1  a      b   resistor r=10
Np  b    gnd   c   gnd   NPORT  N=sqrt(10)
R2  c    gnd   resistor r=1
model NPORT nport macro=yes evaluate="Trans" setup="TransSetUpParams" \
                  module="nportTr.so"
```

The Np instance of the NPORT element has a pair of ports and the N=sqrt(10) parameter.
The TransSetUpParams function in the Netlist 6.1.6 handles the parameters of the NPORT element; its arguments are *fixed* and have a *fixed type*. More precisely we have:

ppParams The array storing the values of the parameters of the NPORT element. It must be allocated and its size must be adequate to store all the possible parameters.

ppKeywords The array of strings storing the names of the parameters; in our case we have only one parameter labelled as N.

pParamNumber The number of parameters of the NPORT element. In this case it is equal to 1 since we have only one parameter N.

In the body of the module the number of parameters is set, then the static array of strings with the names of parameters is assigned. The array storing the values of parameters is allocated and finally the initial values of parameters are assigned. The function that handles parameters must return 1 on success and 0 on failure.
The Trans function implements the characteristic of the NPORT element. This function has *fixed* arguments with *fixed types*, they are:

pParams the array storing the values of parameters.

NportNumber The number of ports (pair of terminals) of the NPORT. If in the netlist a wrong number of port is given for an instance of NPORT this can be checked in the code implementing the element.

V The array of port voltages that are used in the implicit algebraic equations implementing the characteristic of the NPORT.

I The array of port currents that are used in the implicit algebraic equations.

f The values of each function implementing the implicit characteristic of the NPORT. The simulator finds a zero of these functions.

C This is a square matrix whose entries are the derivatives of each f function with respect to the entries of the V port voltages.

R This is a square matrix whose entries are the derivatives of each f function with respect to the entries of the I port currents.

In our case the implicit chacteristic of the NPORT is

$$f_0(V,I) = V_0 - NV_1$$
$$f_1(V,I) = NI_0 + I_1$$

The C and R matrices are thus

$$C = \begin{bmatrix} 1 & -N \\ 0 & 0 \end{bmatrix} \qquad R = \begin{bmatrix} 0 & 0 \\ N & 1 \end{bmatrix}$$

Netlist 6.1.6 — Nport "C" source code. – File: `nportTr.c` –

```
#include <stdio.h>
#include <math.h>

int Trans(pParams, NportNumber, V, I, f, C, R, Time)
double *pParams;
int NportNumber;
double *V;
double *I;
double *f;
double **C;
double **R;
double   Time;
{
    if( NportNumber != 2 )
    {
        fprintf( stderr, "Wrong number of ports.\n" );
        return( 0 );
    }
    double N = pParams[0];
    f[0] = V[0] - N*V[1];
    f[1] = I[1] + N*I[0];
    C[0][0] =  1.0; C[0][1] = -N;
    C[1][0] =  0.0; C[1][1] =  0.0;
    R[0][0] =  0.0; R[0][1] =  0.0;
    R[1][0] =  N;   R[1][1] =  1;
    return( 1 );
}

static char *Keywords[] = {"N"};

int
TransSetUpParams(ppParams, ppKeywords, pParamNumber)
double  **ppParams;
char   ***ppKeywords;
int      *pParamNumber;
{
    extern void *PanMalloc();
    (*pParamNumber) = 1; // Number of parameters
    (*ppKeywords)   = Keywords; // Array storing the names of parameters.
    (*ppParams) = (double *)
      PanMalloc((1+(*pParamNumber))*sizeof(double)); // Parameter allocation
    if( ! (*ppParams) )
        return( 0 );
    (*ppParams)[0] = sqrt(10.0); // Initialise the parameter
    return( 1 );
}
```

By observing the Netlist 6.1.5, we see that card

```
#system "gcc -shared -fPIC -o nportTr.so nportTr.c"
```

that runs the `gcc` compiler through the `system` command. The compiler generates the `nportTr.so` shared library just before reading the remainig part of the netlist and later running the analyses.

6.2 The a2d and d2a converters

The a2d and d2a elements implement the *pseudo analog to digital converter* and the *pseudo digital to analog converter*, respectively. The word "pseudo" means that they are not "real" elements present in the model of the real circuit, but added elements needed to link two different ways of modeling the complete circuit. These different ways are "analog" and "digital". These elements play thus a key role in a mixed analog/digital simulation where a section of the circuit is modeled at transistor level (analog) and a section is modeled at digital level (register transfer logic or gate level, through VERILOG and/or VHLD languages). They transform the analog signal to the digital counterpart and vice versa.

6.2.1 The a2d converter

The a2d element implements a pseudo analog to digital converter, that links the electrical circuit to the digital circuit. A simple example of an a2d instance is

```
I1 pos neg enable ad2 dignet="D1.x" vl=-1 vh=1
```

I1 is the instance name of the a2d. It is connected between the pos and neg nodes and converts the voltage across these nodes to the corresponding digital code. The enable terminal is optional and can be omitted. It can be used to enable/disable the conversion; a voltage greater than 0.5 enables the conversion. The dignet parameter is the hierarchical name of the net in the digital design to which the digital conversion is applied; vl is the low voltage level of the conversion coding and vh is the high voltage level. This means that if D1.x is an 8-bit unsigned register, the $[-1,+1]$ voltage interval is converted in 2^8 discrete levels. As an example if the input voltage of the a2d converter is $+1$ the corresponding digital coding is 0xff, corresponding to 256.

The a2d can also be connected to a *real* variable in the digital circuit. In this case the [vl, vh] interval is partitioned in nl sub-intervals, where nl is a parameter of the a2d built-in element (not used in our simple example). Each time the voltage across the a2d crosses one of these thresholds, the value of the crossed threshold is propagated to the real variable in the digital circuit. The real variable in the digital circuit can thus assume only the discrete values defined by the thresholds.

The dignet="D1.x" selects the hierarchical "D1.x" net in the digital design. More precisely "D1.x" must be a *register* and not net, i.e. it must keep permanently its assignment. If possible PAN automatically converts the "D1.x" type to *register*. Even though in principle it is possible to reassign the content of the "x" register in the digital design, the a2d forces its value at each successful iteration of the Newton method in the various analyses. Thus it is suggested and preferable to not reassign the "x" register internally to the digital code.

The enable terminal can be useful to disable the a2d when the time variations of the driving analog signal at the pos and neg terminal causes a lot of activity of the a2d converter that is useless in the digital design, i.e. it does not have any effect. Thresholds crossings that cause variations of the corresponding digital coding are met with a time accuracy specified by the trtol parameter. This means that a very short time integration step may be used and this slows down simulation. If the a2d is disabled threshold crossing is ignored and simulation may run much faster.

The trtime parameter (not shown in the simple instance example) sets the minimum allowed time window into which a variation of the digital output generated by the a2d must fall. During a time domain analysis, the integration time step is shortened so that any bit change of the coding of the digital net corresponding to a variation of the analog voltage falls inside a time window of length less than the value of trtime. If trtime is not specified, a value that depends on the minimum allowed length of the integration time step is chosen. If the trtime="digital" is specified, the time resolution specified in the digital design is used.

6.3 Capacitors with voltage controlled capacitance

There are capacitors whose characteristic is not conventionally given as a nonlinear function of charge versus voltage, but they are described through thier nonlinear capacitance versus voltage. For example the `C1` capacitor instance

```
C1 pos neg capacitor cv=C0+C1*tanh(v(pos,neg))
```

uses the `cv` parameter to describe the $c(v) = C0 + C1\tanh(v(v))$ capacitance function. In this case we have that the capacitor branch current is $i(t) = c(v)\dfrac{dv}{dt}$. [1]

6.4 Inductors with current controlled inductance

There are inductors whose nonlinear characteristic is not conventionally given as flux versus current, but they are described through their nonlinear inductance. For example the `L1` inductor instance

```
C1 pos neg capacitor cv=C0+C1*tanh(v(pos,neg))
```

uses the `li` parameter to describe the $l(i) = C0 + C1\tanh(i(L1))$ inductance function. In this case we have that the branch voltage of the inductor is $v(t) = l(\imath)\dfrac{d\imath}{dt}$. [2]

---

[1] This constitutive relation is numerically implemented in this way in the simulator. For example assume to employ the Gear order 2 integration method and the h integration time step. We have that the $i(v)$ capacitor current is

$$i(v_{n+1}) = c(v_{n+1})\frac{1}{h}\left(k_0 v_{n+1} + k_1 v_n + k_2 v_{n-1}\right)$$

We know that

$$\frac{dQ}{dt} = i(v_{n+1}) = c(v_{n+1})\frac{1}{h}\left(k_0 v_{n+1} + k_1 v_n + k_2 v_{n-1}\right)$$

and thus

$$\frac{1}{h}\left(k_0 Q_{n+1} + k_1 Q_n + k_2 Q_{n-1}\right) = c(v_{n+1})\frac{1}{h}\left(k_0 v_{n+1} + k_1 v_n + k_2 v_{n-1}\right)$$

Few algebraic handling leads to

$$Q_{n+1} = \frac{1}{k_0}\left(c(v_{n+1})\left(k_0 v_{n+1} + k_1 v_n + k_2 v_{n-1}\right) - k_1 Q_n - k_2 Q_{n-1}\right)$$

The $\widehat{c}(v)$ numerical version of the capacitance is

$$\widehat{c}(v_{n+1}) = \frac{dQ_{n+1}}{dv_{n+1}} = \frac{1}{k_0}\frac{dc(v_{n+1})}{dv_{n+1}}\left(k_0 v_{n+1} + k_1 v_n + k_2 v_{n-1}\right) + c(v_{n+1})$$

The $\widehat{c}(v_{n+1})$ tends to $c(v_{n+1})$ as the integration time step h goes to 0, that is $c(v_{n+1}) = \lim_{h\to 0}\widehat{c}(v_{n+1})$.

[2] The constitutive equation of a capacitor with voltage controlled capacitance, see Section 6.3, was detailed when it is used in the time domain simulations. This similar constitutive relation of the volatge controlled inductor is now considered for what concerns its implementatin in the harmonic balance analysis.
The operator that implements the Fast Fourier transforms is the Γ matrix. For example given the v vector of time samples the corresponding coefficients of the spectrum are $\mathscr{V} = \Gamma v$. The inverse Fast Fourier tranform is represented by the Γ^{-1} matrix. We have that the sensitivity of the $\mathscr{V}$ voltage spectrum with respecto to the $\mathscr{I}$ current spectrum is

$$\frac{\partial\mathscr{V}}{\partial\mathscr{I}} = \frac{\partial\mathscr{V}}{\partial v}\frac{\partial v}{\partial\imath}\frac{\partial\imath}{\partial\mathscr{I}} = \Gamma\frac{dv}{d\imath}\Gamma^{-1} = \Gamma\left(\frac{\partial}{\partial\imath}\left(l(\imath)\frac{d\imath}{dt}\right)\right)\Gamma^{-1} = \Gamma\left(\frac{\partial l(\imath)}{\partial\imath}\frac{d\imath}{dt} + l(\imath)\frac{d}{d\imath}\left(\frac{d\imath}{dt}\right)\right)\Gamma^{-1} =$$
$$= \Gamma\left(\frac{\partial l(\imath)}{\partial\imath}\frac{d\imath}{dt}\right)\Gamma^{-1} + \Gamma\left[l(\imath)\frac{d}{d\imath}\left(\frac{d\imath}{dt}\right)\right]\Gamma^{-1}$$

6.5 The s-domain voltage controlled voltage source (svcvs)

The svcvs input/output characteristic is a rational polynomial of the s variable

$$\mathscr{Y} = k\frac{n_0 + n_1 s + n_2 s^2 + \cdots + n_n s^n}{d_0 + d_1 s + d_2 s^2 + \cdots + d_m s^m}\mathscr{X}$$

where $\mathscr{X}$ is the input variable in the frequency domain and $\mathscr{Y}$ is the output variable. In the time domain the input/output characteristic is implemented as a differential algebraic system. As an example consider the

$$\mathscr{Y} = k\frac{n_0 + n_1 s + n_2 s^2}{d_0 + d_1 s + d_2 s^2}\mathscr{X} \tag{6.1}$$

input/output characteristic composed of two zeros and two poles, it can be recast

$$\mathscr{Y}\left(d_0 + d_1 s + d_2 s^2\right) = k\left(n_0 + n_1 s + n_2 s^2\right)\mathscr{X}$$

and thus written as the differential equation

$$\left(d_0 y + d_1\frac{dy}{dt} + d_2\frac{d^2y}{dt^2}\right) - \overbrace{k\left(n_0 x + n_1\frac{dx}{dt} + n_2\frac{d^2x}{dt^2}\right)}^{\zeta} = 0 \tag{6.2}$$

The equivalent circuit implementing Eq. (6.2) is shown in Figure 6.2. The differential algebraic

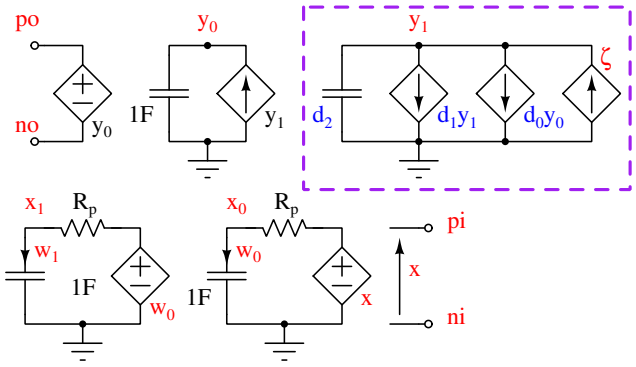

Figure 6.2: The equivalent circuit implementing the s-domain svcvs.

The computation of the $\left[l(\iota)\frac{d}{d\iota}\left(\frac{d\iota}{dt}\right)\right]$ term requires some attention. It can be rewritten as

$$\Gamma\left[l(\iota)\frac{d}{d\iota}\left(\frac{d\iota}{dt}\right)\right]\Gamma^{-1} = \Gamma\left[l(\iota)\frac{d}{d\iota}\left(\frac{d}{dt}\left(\Gamma^{-1}\mathscr{I}\right)\right)\right]\Gamma^{-1} = \Gamma\left[l(\iota)\frac{d}{d\iota}\left(\Gamma^{-1}\frac{d\mathscr{I}}{dt}\right)\right]\Gamma^{-1} = \Gamma\left[l(\iota)\frac{d}{d\iota}\left(\Gamma^{-1}\Omega\mathscr{I}\right)\right]\Gamma^{-1} =$$
$$= \Gamma\left[l(\iota)\left(\Gamma^{-1}\Omega\frac{d\mathscr{I}}{dt}\right)\right]\Gamma^{-1} = \Gamma\left[l(\iota)\Gamma^{-1}\Omega\Gamma\right]\Gamma^{-1}$$

Recombining the terms we finally obtain

$$\frac{\partial\mathscr{V}}{\partial\mathscr{I}} = \Gamma\left(\frac{\partial l(\iota)}{\partial\iota}\frac{d\iota}{dt}\Gamma^{-1} + l(\iota)\Gamma^{-1}\Omega\right)$$

equation modeling the circuit in Figure 6.2 is

$$\begin{aligned}
&d_2\frac{dy_1}{dt}+d_1y_1+d_oy_0-\zeta=0\\
&\frac{dy_0}{dt}-y_1=0\\
&\frac{dx_0}{dt}-w_0=0\\
&\frac{dx_1}{dt}-w_1=0\\
&R_pw_0+x_0-x=0\\
&R_pw_1+x_1-w_0=0\\
&\zeta=k\left(n_0x+n_1w_0+n_2w_1\right)\\
&y=y_0
\end{aligned}\tag{6.3}$$

where the $y(t)$ output voltage is across the `po` and `no` nodes of the circuit in Figure 6.2 and $x(t)$ is the voltage across the `pi` and `ni` nodes. The sub-circuit enclosed in the **amethyst** dashed box implements the first equation of Eq. (6.3). The auxiliary y_o and y_1 variables implement the $\frac{d}{dt}$ operator that introduces poles (left hand side of Eq. (6.3)). The auxiliary x_0 and x_1 variables implement the $\frac{d}{dt}$ operator that introduces zeros (right hand side of Eq. (6.3)). The R_p resistor removes degenerations in the equivalent circuit since capacitors and controlled voltage sources are connected in parallel. These resistors introduce additional poles. The value of these resistors is chosen so that these additional poles fall at very high frequencies corresponding to time constant comparable with the integration time step used to solve the dynamic equivalent circuit. Some algebraic handling applied to Eq. (6.3) leads to

$$\begin{aligned}
&d_2\frac{d^2y}{dt^2}+d_1\frac{dy}{dt}+d_oy-k\left(n_0x+n_1w_0+n_2w_1\right)=0\\
&R_p\frac{dx_0}{dt}+x_0-x=0\\
&R_p\frac{dx_1}{dt}+x_1-w_0=0
\end{aligned}$$

By applying the Laplace transform we obtain

$$\begin{aligned}
&\left(d_2s^2+d_1s+d_o\right)\mathscr{Y}-k\left(n_0\mathscr{X}+n_1s\mathscr{X}_0+n_2s\mathscr{X}_1\right)=0\\
&\left(R_ps+1\right)\mathscr{X}_0-\mathscr{X}=0\\
&\left(R_ps+1\right)\mathscr{X}_1-s\mathscr{X}_0=0
\end{aligned}$$

and back substitution leads to

$$\mathscr{Y}=k\frac{n_0+\left(n_1+n_oR_P\right)s+n_2s^2}{d_2s^2+d_1s+d_o}\left(\frac{1}{R_ps+1}\right)\mathscr{X}\tag{6.4}$$

By comparing Eq. (6.1) with Eq. (6.4) we see that the n_0R_p term at the numerator and the R_ps+1 term at the denominator modify the original transfer function. We see also that the "amount" of modification goes to 0 as $R_p\to 0$.

If the order of the denominator of Eq. (6.4) is increased by 1 and the new d_3 coefficient is introduced, a new y_2 electric variable is introduces together with a parallel connection of voltage controlled current source and 1 F capacitor. If the order of the numerator is increased by 1 and the new n_3 coefficient is introduced, a new x_2 electric variable is introduced together with a new series connection of a voltage controlled voltage source, an R_p resistor and 1 F capacitor. The structure of the circuit enclosed in the **amethyst** dashed box remains the same, what changes is the connection of controlled sources in parallel.

6.5.1 Some comments on the use of the `svcvs`

The numerator is implemented by subsequently deriving the input signal. This is a problematic action since "numerical noise" can be largely amplified and cause convergence problems and/or introduce inaccuracy. A recommendation is when possible to split the transfer function in products of terms where at the numerator there is only one zero and at the denominator at least one pole whose frequency is above that of the zero.
If the `svcvs` is used in time domain analyses, if possible do a test analysis with only the `svcvs` to check if R_p is correctly set.
If the R_p resistor is not specified, in time domain analyses it is chosen in accordance to the minimum integration time step. If the user specifies its value, the given value is used in all analyses.

6.5.2 The sum of fraction model of the `svcvs`

Due to the issues describe in Section 6.5 and 6.5.1, they can be avoided if the transfer function is implemented as a sum of fractions. This format is suited to the output of the vector fitting method [**Gustafsson1972**]. When this model is used a list of poles, each real pole or each pair of complex conjugate poles forms the denominator of a fraction. The corresponding real or complex conjugate residues form the numerator of each fraction. The maximum degree of a denominator is thus 2. The results from each fraction are suitably summed at the output. With this model it is no longer needed to specify the value of the R_p resistor.

6.5.3 An application example

The circuit of the Tow-Thomas oscillator is exploited as example of use of the `svcvs` element. The schematic of the Tow-Thomas oscillator is shown in Figure 6.3. A detailed description

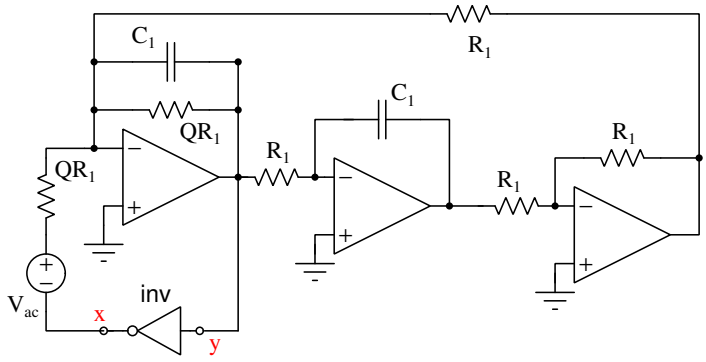

Figure 6.3: The circuit implementing the Tow-Thomas oscillator.

of this oscillator can be found in [**tow-thomas**]. The oscillator is composed of a variable Q band-pass filter implemented through three operational amplifiers and a passive network and a nonlinear element, i.e. the `inv` inverter. The characteristic of the band-pass filter is

$$\mathscr{Y} = -\frac{R_1^2 C_1 s}{R_q + R_1^2 C_1 s + R_q R_1^2 C_1^2 s^2}\mathscr{X}$$

The input/output characteristic of the inverter is implemented as an algebraic discontinuous function, i.e. x is equal to 1 if $y < 0$ and x is equal to -1 if $y \geq 0$. The Netlist 6.5.1 implements the oscillator.

Netlist 6.5.1 — The netlist of the Tow-Thomas oscillator. – File: svcvsTowThomas.pan –

```
ground electrical gnd
parameters Q=2 C1=20n R1=1k RQ=Q*R1

Dc dc print=true
Ac ac start=0.1 stop=100M dec=100
Tr tran stop=10m tmax=1u
Sh shooting autonomous=yes period=130u method=2 maxord=2 floquet=true

Filter  y   gnd   x   gnd   svcvs dcgain=-1 numer=[0,R1**2*C1] \
                                 denom=[RQ,R1**2*C1,(R1*C1)**2*RQ]
Inv     x    ac   y   gnd   vcvs func=(v(y) >= 0) ? -1 : 1 discontinuous=yes
Vac    ac   gnd             vsource mag=1
```

The card

```
Filter y gnd x gnd svcvs dcgain=-1 numer=[0,R1**2*C1]
                                  denom=[RQ,R1**2*C1,(R1*C1)**2*RQ]
```

implements the transfer function of the band-pass filter. The polynomials at the numerator and denominator are defined through their coefficients. The `inv` inverter is modeled as a behavioural element. The `digital=yes` parameter specifies that it behaves as a digital element that gives an output voltage jumping between discrete values. This tells the simulator that the circuit is described by an *hybrid dynamic algebraic differential equation* and triggers the insertion of the *saltation matrices* during the shooting analysis.
Several analyses are performed, we report the results by the `Ac` ac analysis that shows the input/output characteristic of the band-pass filter in the left panel of Figure 6.4 and those by the `Sh` shooting analysis that computes the steady state solution in the right panel of Figure 6.4. The

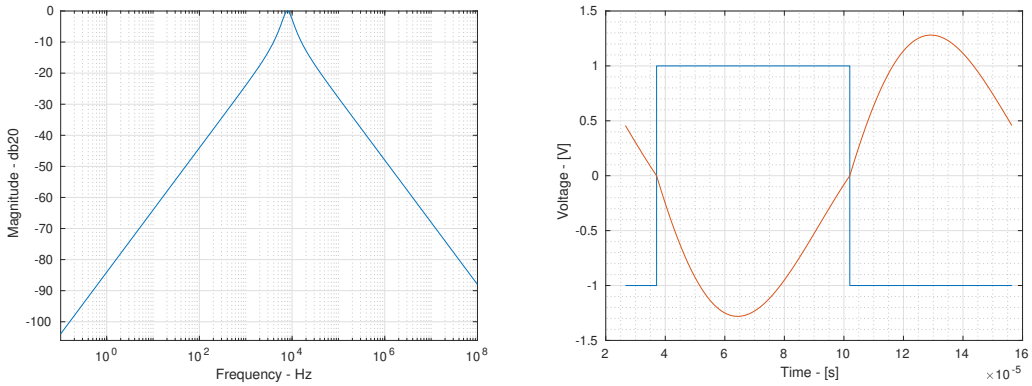

Figure 6.4: Left panel: the x-y input/output characteristic of the band-pass filter of the oscillator by the `Ac` analysis. Right panel: red trace, the periodic voltage at node y by the `Sh` shooting analysis, blue trace, the periodic voltage at node x.

shooting analysis computes also the Floquet multipliers, those with largest modulus are $\lambda_1 = 1$ and $\lambda_2 = 0.19749$. We see that we have two significant multipliers and one is equal to 1 since we are dealing with an oscillator.

6.5.4 Integrator: a pole in the frequency origin

When the `svcvs` implements a transfer function with a pole in the origin its input/output gain is ∞ when a DC analysis ($\omega = 0$) is performed. This can cause convergence problems. To have a clear idea about what may happen we use simple circuits as demonstrators. The first circuit is shown in Figure 6.5; the netlist implementing it is shown in Netlist 6.5.2.

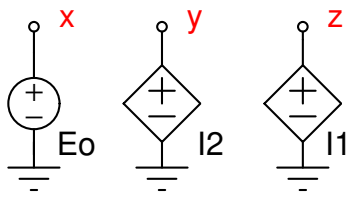

Figure 6.5: A simple power system.

Netlist 6.5.2 — Svcvs at Dc. – File: `svcvsDc1.net` –

```
ground electrical gnd
Dc dc print=yes
I1  z   gnd   y   gnd  svcvs numer=[1] denom=[0,1]
I2  y   gnd   z     x  vcvs func=v(z) + v(x)
Eo  x   gnd   vsource vdc=0.5
```

The `I1` instance is a `svcvs` that implements an ideal integrator. Its input is the `y` voltage. The `y` voltage is set by the `I2` voltage controlled voltage source that implements the $v(y) = v(x) + v(z)$ linear transfer function. The `Eo` independent voltage source sets the value of the $v(x)$ voltage. At DC the infinite gain of the `svcvs` leads it to degenerate in a nullor. Its input port is thus a nullator, i.e. the input voltage must be zero and so is its input current. This means that $v(y)$ must be null. The output port of the `svcvs` is a norator, i.e., it does not introduce any constraint on the port current and voltage. Since the characteristic of `I1` is $v(y) = v(x) + v(z)$, the nullator forces $v(z) = -Eo$.

By referring to the circuit in Figure 6.5, we have $v(z) = -0.5\,\mathrm{V}$. The important aspect is that $v(z)$ can be derived from the other electrical quantities, since there is a "path" that links the `z` node (the norator terminal) to the other circuit nodes (electrical variables). The `Dc` DC analysis successfully finds a solution of this circuit.

Now we consider the same circuit shown in Figure 6.5, but modify the characteristic of `I2` as $v(y) = \cos(v(x)) + v(z)$. The corresponding netlist is shown in Netlist 6.5.3.

Netlist 6.5.3 — Svcvs at Dc – degeneration. – File: `svcvsDc2.net` –

```
ground electrical gnd
Dc dc print=yes
I1  z   gnd   y   gnd  svcvs numer=[1] denom=[0,1]
I2  y   gnd   z     x  vcvs func=cos(v(z)) + v(x)
Eo  x   gnd   vsource vdc=0.5
```

The algebraic equation modeling this version of the circuit is

$$\begin{cases} v_y = 0 \\ \cos(v_z) + v_x - v_y = 0 \\ v_x - E_o = 0 \\ i_x = 0 \\ i_y = 0 \\ i_z = 0 \end{cases}$$

PAN solves this circuit by using the Newton method. At the n-th iteration it solves the linear problem

$$\begin{bmatrix} v_x^{n+1} \\ v_y^{n+1} \\ v_z^{n+1} \\ i_x^{n+1} \\ i_y^{n+1} \\ i_z^{n+1} \end{bmatrix} = \begin{bmatrix} v_x^{n} \\ v_y^{n} \\ v_z^{n} \\ i_x^{n} \\ i_y^{n} \\ i_z^{n} \end{bmatrix} - \overbrace{\begin{bmatrix} 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & -1 & -\sin\left(v_z^n\right) & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}}^{\mathbf{J}^n} \begin{bmatrix} v_y^n \\ \cos(v_z^n) + v_x^n - v_y^n \\ v_x^n - E_o \\ i_x^n \\ i_y^n \\ i_z^n \end{bmatrix}$$

due to the Newton method. During the Dc analysis PAN "complains" by displaying the message

```
Fatal error  detected during DC analysis "Dc".
    Matrix is singular.
    Singularity is at row <8> and column <1> corresponding to equations <I2:i>
    and <z> respectively.
```

By observing the $\mathbf{J}^n$ Jacobian matrix we see that if $v_z^n = 0$ the matrix is singular since $\sin\left(v_z^n\right) = 0$. The Newton method starts from an initial guess where $v_z^n = 0$. This "opens the connection" that link the z node and the y node. The norator is thus "isolated". Since it does not impose any constraint to the z electrical variable we get a pathological circuit.
The onset of a pathological circuit is initially triggered by the fact that the ideal integrator degenerates in a nullator since the gain goes to infinity. This can be avoided by limiting the gain of the svcvs during the DC analysis. The dcmaxgain parameter of the svcvs instance limits the gain of the svcvs during and only during the DC analysis avoiding that the svcvs degenrates in a nullator. When this parameter is used, the corresponding instance modifies as

```
I1 z gnd y gnd svcvs numer=[1] denom=[0,1] maxdcgain=1M
```

In this case the maximim gain at DC is set at 10^6. This obviously alters the nature of the original circuit, but a suitable choice of the maximum gain can lead to an irrelevant modification. It is left to the user to check this.

A way to help PAN to determine a solution is to suggest a suitable initial condition. By considering the circuit in Figure 6.5 and the fact that it causes convergence problems since $v_z^n = 0$ is used as initial guess to the Newton method, we can exploit the ic parameter to suggest a different and better initial guess. The ic parameter of the DC analysis is described in Section 2.9.3. We thus use the statement

```
Dc dc print=yes ic=["z",10m]
```

to help PAN. PAN finds the correct solution $v_z = \text{acos}(0.5)$ of the circuit in Figure 6.5 when it is modeled by the Netlist 6.5.3.

The use of the ic parameter may be not straightforward. In fact, if the circuit shown in Figure 6.5 were a sub-circuit of a more complex one, i.e., it represented a small portion of a large and complex circuit, it would be very difficult to isolate it as the problematic sub-circuit, to understand what happens and to suggest a suitable initial guess. We thus suggest to consider and used also the fixsm parameter of the DC analysis. Its description is in Section 2.9.4. When set "true", the simulator automatically inserts parasitic elements that try to solve the singularity problems of the Jacobian matrix. The statement that does this is

```
Dc dc print=yes fixsm=yes
```

By considering the error message by PAN reported in Section 6.5.4, a linear controlled current source that injects current into the I2:i node of the circuit and that is driven by the voltage at node z is automatically inserted by the simulator to try to solve the problem. The transconductance value of this linear voltage controlled current source is $g_{\min}$.

6.6 The s-domain current controlled voltage source (sccvs)

The sccvs replicates the features of the svcvs, i.e the Laplace domain voltage controlled voltage source, that was extensively described in Section 6.5.

6.7 Floating point fault in behavioural elements

The constitutive equations and characteristics of the controlled sources, capacitors, inductors and resistors can be described by algebraic expressions. These expressions can be interpreted or compiled. The compiled version runs faster than the interpreted ones. During the development of these behavioural elements it is suggested to use the interpreted version or to compile the code with the protected mode. The protected code runs slowly with respect to the unprotected mode, but it detects all major causes that generate floating point faults, such ad for example division by 0. When the model is tested and runs correctly the code can be recompiled with the unprotected mode. PAN detects floating point errors and gives indications on the instance that generated those. In this case we suggest to recompile the behavioural code in the protected mode to get more information about the element that generated the fault and on the fault type. To this end see the specific options of each element and the 'compile' and 'cc' parameters of the 'options' card of PAN.

7. Api functions

PAN makes available to the user functions to access some useful internal variables. These functions can be used in shared modules that implement behavioural elements as described in Section 6.1.4. The available functions are:

`double PanApiGetCircuitTempK()` Returns the ambient temperature in K at which the simulations are performed.

`double PanApiGetCircuitTempC()` Returns the ambient temperature in °C at which the simulations are performed.

`double PanApiGetGmin(int Thread)` Returns the value of the `gmin` parasitic conductance automatically inserted by PAN and for example swept during the gmin continuation method. `Thread` is an integer input argument that identifies the thread executing the code of the behavioural element. DC analysis may run in parallel using multiple threads. In a single thread running analysis, the `Thread` argument must be equal to 0.

`double PanApiGetInitialGmin(int Thread)` Returns the initial value of the `gmin` parasitic conductance automatically inserted by PAN when the circuit model is built. `Thread` is an integer input argument that identifies the thread executing the code of the behavioural element. DC analysis may run in parallel using multiple threads. In a single thread running analysis, the `Thread` argument must be equal to 0.

`double PanApiGetPwrLevel(int Thread)` Returns the power level used by the power-stepping continuation method used by the DC analysis to compute the solution of power sub-networks. `Thread` is an integer input argument that identifies the thread executing the code of the behavioural element. DC analysis may run in parallel using multiple threads. In a single thread running analysis, the `Thread` argument must be equal to 0.

`double PanApiGetAbsTime()` Returns the simulation time of time domain analyses. Note that during time domain analyses the solution is computed at $t_n + h_n$, where t_n is the time instant at which the last solution was computed and h_n is the current integration time step. The current solution is computed at the $t_n + h_n$ time instant. The `PanApiGetAbsTime()` function returns the value of t_n.

`double PanApiGetTimeStep()` Returns the value of the current h_n integration time step used in time domain analyses. The current solution is computed at the $t_n + h_n$ time instant, where t_n is the time instant at which the last solution is available.

`double PanApiGetPrevTime()` Return the t_{n-1} time instant of time domain analyses.

The last available solution was computed at the t_n time instant. The next solution is computed at the $t_n + h_n$ time instant where h_n is the current value of the integration time step. The previous solution was computed at t_{n-1}.

`double PanApiSetMaxTimeStep(double Value)` Sets the maximum integration time step that can be used by `PAN` in time domain analyses.

`double PanApiRandom()` Initialises the random number generator.

8. Simulations of power systems

PAN allows simulation of power systems described through equivalent single-phase models. Single-phase models pretend that only the *positive-sequence* exists. The power netlist and instances of elements must be enclosed in a block marked at the beginning by the `begin power` keyword and at the end by the `end` keyword. PAN supports mixed analyses of for example power networks and electrical networks linked together by `powerec` pseudo-element. Its task is similar to those of the `a2d` and `d2a` pseudo-elements introduced in Section 6. Mixed power/electrical analyses can be particularly flexible and useful when simulating for example a power network and electronics equipments such as switching power converters.
A simple example of a power netlist that implements the power systems shown in Figure 8.1 is reported in Netlist 8.0.1.

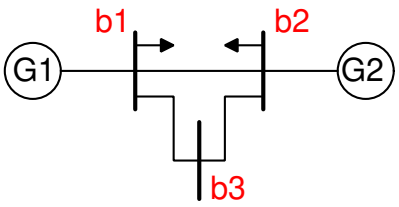

Figure 8.1: A simple power system.

Netlist 8.0.1 — Three bus two machine power network. – File: Power001.net –

```
ground electrical gnd
Dc dc nettype=1 print=1
begin power
G1 b1 powergenerator pg=0.0 vg=1.06 prating=60M vrating=69K \
        slack=1 xl=00 Ra=3.1m xd=1.05 xdp=0.185 xds=0.13 td0p=6.1 td0s=40m \
        xq=0.980 xqp=0.36 xqs=0.13 tq0p=0.3 tq0s=99m m=13.08 d=2 type=6
G2 b2 powergenerator pg=0.4*100/60 vg=1.045 prating=60M vrating=69K \
xl=0 Ra=3.1m xd=1.05 xdp=0.185 xds=0.13 td0p=6.1 td0s=40m \
        xq=0.98 xqp=0.36 xqs=0.13 tq0p=0.3 tq0s=99m m=13.08 d=2 type=6

Lo1 b1 powerload pc=0.549 qc=25.2m vrating=69k prating=100M
Lo2 b2 powerload pc=0.549 qc=25.2m vrating=69k prating=100M

Line1 b1 b2 powerline r=56.95m x=0.17388 b=34m prating=100M vrating=69k
Line2 b1 b3 powerline r=56.95m x=0.17388 b=34m prating=100M vrating=69k
Line3 b2 b3 powerline r=56.95m x=0.17388 b=34m prating=100M vrating=69k

Bus1 b1 powerbus vb=69k v0=1.06  theta0= 0.0
Bus2 b2 powerbus vb=69k v0=1.045 theta0=-0.72453
Bus3 b3 powerbus vb=69k v0=1.055 theta0=-0.41532
end
```

It implements a power system with the G1 and G2 synchronous generators, three b1, b2 and b3 busses and two constant power loads. The G1 generator instance is

```
G1 b1 powergenerator pg=0.0 vg=1.06 prating=60M vrating=69K \
        slack=1 xl=00 Ra=3.1m xd=1.05 xdp=0.185 xds=0.13 td0p=6.1 td0s=40m \
        xq=0.980 xqp=0.36 xqs=0.13 tq0p=0.3 tq0s=99m m=13.08 d=2 type=6
```

where several parameters are given. The type=6 parameter that defines the order of the synchronous generator model (some more information is given in Section 8.3.4), slack=1 that tells PAN that this is the slack generator when a Power Flow Analysis (PF) analysis is performed, that is, it will deliver as much as active and reactive power to balance the differences between those generated by the PV generators (and all the other constant sources) and loads. A PV generator keeps constant its bus voltage and active power that are specified by the pg and vg parameters. The prating parameters specifies the power rating of the synchronous generator and vrating its voltage rating. The pg and vg parameters are given in *per-unit* with respect to prating and vrating, respectively. These parameter set defines the static model of the synchronous generator, i.e., that used in the PF analysis. The other parameters defines the dynamic model of the synchronous generator that is used in transient stability analyses, i.e., in time domain analyses.
By referring to the schematic in Figure 8.0.1 the b1 and b2 buses are connected by the Line1 power line as specified by the card

```
Line1 b1 b2 powerline r=56.95m x=0.17388 b=34m prating=100M vrating=69k
```

The load instance of the

```
Lo1 b1 powerload pc=0.549 qc=25.2m vrating=69k prating=100M
```

card is connected to b1. The powerbus element as given in the

```
Bus1 b1 powerbus vb=69k v0=1.06  theta0= 0.0
```

card is an auxiliary element that tells PAN about the initial guess of d-axis and q-axis voltages of the b1 bus before starting the PF analysis.

8.1 Charles Legeyt Fortescue's transformation

$$\begin{bmatrix} v_0 \\ v_p \\ v_n \end{bmatrix} = \overbrace{\begin{bmatrix} 1 & 1 & 1 \\ 1 & \alpha^2 & \alpha \\ 1 & \alpha & \alpha^2 \end{bmatrix}}^{\mathbf{A}} \begin{bmatrix} v_a \\ v_b \\ v_c \end{bmatrix}$$

where $\alpha = e^{\mathrm{I}\frac{2}{3}\pi}$.

$$\mathbf{A}^{-1} = \frac{1}{3}\begin{bmatrix} 1 & 1 & 1 \\ 1 & \alpha & \alpha^2 \\ 1 & \alpha^2 & \alpha \end{bmatrix}$$

Power in the $pn0$ coordinate frame is

$$S = [v_a, v_b, v_c]\begin{bmatrix} i_a^* \\ i_b^* \\ i_c^* \end{bmatrix} = \left(\mathbf{A}^{-1}\begin{bmatrix} v_0 \\ v_p \\ v_n \end{bmatrix}\right)^T \left(\mathbf{A}^{-1}\begin{bmatrix} i_0 \\ i_p \\ i_n \end{bmatrix}\right)^* =$$
$$= [v_0, v_p, v_n]\,\mathbf{A}^{-1}\left(\mathbf{A}^{-1}\right)^* \begin{bmatrix} i_0^* \\ i_p^* \\ i_n^* \end{bmatrix} = \tfrac{1}{3}\left(v_0 i_0^* + v_p i_p^* + v_n i_n^*\right)$$

where the * symbol is complex conjugation.

8.2 Park's transform and differential equations

The Park's transform is a usefull and powerfull tool to describe elements of the single-phase model representation. The Park's transform is defined as

$$\mathbf{P}^{-1} = \sqrt{\frac{2}{3}}\begin{bmatrix} \cos(\omega t) & \cos(\omega t - \frac{2\pi}{3}) & \cos(\omega t + \frac{2\pi}{3}) \\ -\sin(\omega t) & -\sin(\omega t - \frac{2\pi}{3}) & -\sin(\omega t + \frac{2\pi}{3}) \\ \sqrt{1/2} & \sqrt{1/2} & \sqrt{1/2} \end{bmatrix}$$

When it is applied to the $x_a(t), x_b(t), x_c(t)$ three-phase electrical voltages/currents

$$\mathbf{P}\begin{bmatrix} x_d(t) \\ x_q(t) \\ x_0(t) \end{bmatrix} = \begin{bmatrix} x_a(t) \\ x_b(t) \\ x_c(t) \end{bmatrix}$$

we obtain the corresponding $x_d(t)$, $x_q(t)$, $x_o(t)$ electrical voltages/currents in the new DQ0 frame. On of the main important aspects of the Park's transform is that if $x_a(t)$, $x_b(t)$, $x_c(t)$ are symmetrical sinusoidal functions at ω, the corresponding $x_d(t)$, $x_q(t)$, $x_o(t)$ functions are *constant* in time. This tremendously speeds up the simulation of dynamic three-phase circuits in the time domain, since the integration time step is no longer upper limited to adequately sample the time-varying behaviour of the electrical variables along the $\frac{2\pi}{\omega}$ period but it can potentially grow without any upper limitation.

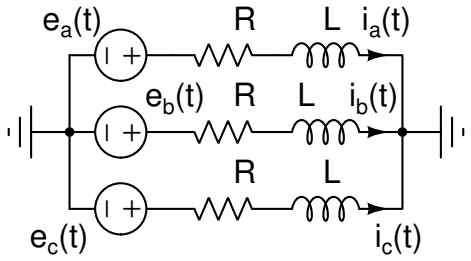

Figure 8.2: The schematic of the simple three-phase dynamic circuit.

To show this, consider the simple three-phase balanced dynamic circuit shown in Figure 8.2. It is described by the ordinary differential equation

$$L\frac{d}{dt}\begin{bmatrix} i_a \\ i_b \\ i_c \end{bmatrix} + R\begin{bmatrix} i_a \\ i_b \\ i_c \end{bmatrix} = \begin{bmatrix} e_a \\ e_b \\ e_c \end{bmatrix}$$

By using the Park's transform it can be recast as

$$L\frac{d}{dt}\left(\mathbf{P}\begin{bmatrix} i_d \\ i_q \\ i_0 \end{bmatrix}\right) + R\mathbf{P}\begin{bmatrix} i_q \\ i_q \\ i_q \end{bmatrix} = \mathbf{P}\begin{bmatrix} e_d \\ e_q \\ e_0 \end{bmatrix}$$

that can be expanded as

$$L\frac{d\mathbf{P}}{dt}\begin{bmatrix} i_d(t) \\ i_q(t) \\ i_0(t) \end{bmatrix} + L\mathbf{P}\frac{d}{dt}\begin{bmatrix} i_d \\ i_q \\ i_0 \end{bmatrix} + R\mathbf{P}\begin{bmatrix} i_q \\ i_q \\ i_q \end{bmatrix} = \mathbf{P}\begin{bmatrix} e_d \\ e_q \\ e_0 \end{bmatrix} \tag{8.1}$$

The $\dfrac{d\mathbf{P}}{dt}$ term in (8.1) expands as

$$\frac{d\mathbf{P}}{dt} = \begin{bmatrix} 0 & -\omega & 0 \\ \omega & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}\mathbf{P}$$

and that from (8.1) we obtain

$$L\mathbf{P}\begin{bmatrix} 0 & -\omega & 0 \\ \omega & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}\begin{bmatrix} i_d \\ i_q \\ i_0 \end{bmatrix} + L\mathbf{P}\frac{d}{dt}\begin{bmatrix} i_d \\ i_q \\ i_0 \end{bmatrix} + R\mathbf{P}\begin{bmatrix} i_q \\ i_q \\ i_q \end{bmatrix} = \mathbf{P}\begin{bmatrix} e_d \\ e_q \\ e_0 \end{bmatrix} \tag{8.2}$$

By left multiplying (8.2) by $\mathbf{P}^{-1}$ we finally obtain

$$\begin{bmatrix} L & 0 & 0 \\ 0 & L & 0 \\ 0 & 0 & L \end{bmatrix}\begin{bmatrix} 0 & -\omega & 0 \\ \omega & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}\begin{bmatrix} i_d \\ i_q \\ i_0 \end{bmatrix} + \underbrace{\begin{bmatrix} L & 0 & 0 \\ 0 & L & 0 \\ 0 & 0 & L \end{bmatrix}\frac{d}{dt}\begin{bmatrix} i_d \\ i_q \\ i_0 \end{bmatrix}}_{\Gamma(t)} + \begin{bmatrix} R & 0 & 0 \\ 0 & R & 0 \\ 0 & 0 & R \end{bmatrix}\begin{bmatrix} i_q \\ i_q \\ i_0 \end{bmatrix} = \begin{bmatrix} e_d \\ e_q \\ e_0 \end{bmatrix}$$

Often the $\Gamma(t)$ term is neglected in the single-phase model of power elements, since it takes into account "fast" electrical behaviours that are assumed no marginal importance with respect to the mush slow electrimechanical dynamics of mechanical elements such as for example synchronous and asynchronous machines, regulators, turbine governors, boilers .....

8.3 Power elements

In this section some of the power built-in elements implemented in PAN are described.

8.3.1 Automatic voltage regulators

PAN implements several types of automatic voltage regulators. The saturation curve used in type I and type II automatic voltage resulators is $S(v) = A_e v e^{B_e|v|}$. The A_e and B_e parameters can be directly given in the instance of the automatic voltage regulator or can be derived by two distinct point of the $S(v_f)$ characteristic in the following way

$$\begin{cases} S(v_1) = A_e e^{B_e v_1} \\ S(v_2) = A_e e^{B_e v_2} \end{cases}$$

where $S(v_1)$, $S(v_2)$ and v_1, v_2 are the given values, assumed positive. Simple algebra leads to

$$B_e = \frac{\log(S(v_1)) - \log(S(v_2))}{v_1 - v_2}$$
$$A_e = S(v_1) e^{-B_e v_1}$$

Type I

The block schematic of the type I automatic voltage regulator is shown in Figure 8.3. The

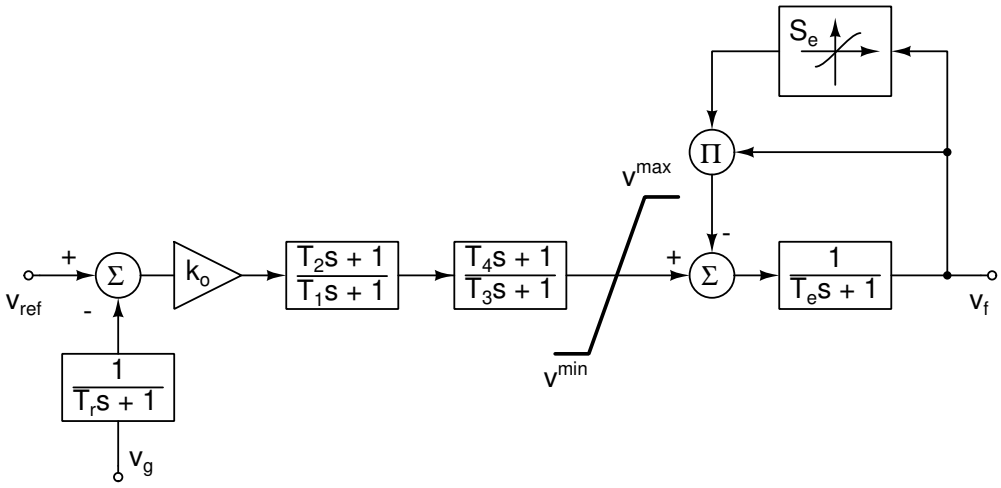

Figure 8.3: The block schematic of the type I automatic voltage regulator.

equations of the model implementing the automatic voltage regulator are

$$
\begin{aligned}
&T_r \frac{dv_m}{dt} + v_m - v_g = 0 \\
&T_1 \frac{dv_{r1}}{dt} + v_{r1} - K_0 \left(1 - \frac{T_2}{T_1}\right)(v_{\text{ref}} - v_m) = 0 \\
&T_3 \frac{dv_{r2}}{dt} + v_{r2} + \left(\frac{T_4}{T_3} - 1\right)\left(v_{r1} + K_0 \frac{T_2}{T_1}(v_{\text{ref}} - v_m)\right) = 0 \\
&v^* = v_{r2} + \frac{T_4}{T_3}\left(v_{r1} + K_0 \frac{T_2}{T_1}(v_{\text{ref}} - v_m)\right) \\
&v_r = \begin{cases} v^* & \text{if } v^{\min} \le v^* \le v^{\max} \\ v^{\max} & \text{if } v^* > v^{\max} \\ v^{\min} & \text{if } v^* < v^{\min} \end{cases} \\
&T_e \frac{dv_f}{dt} + v_f\left(1 + A_e e^{B_e |v_f|}\right) - v_r = 0
\end{aligned}
$$

Type II

The block schematic of the type II automatic voltage regulator (IEEET1) is shown in Figure 8.4. The equations of the model implementing the automatic voltage regulator are

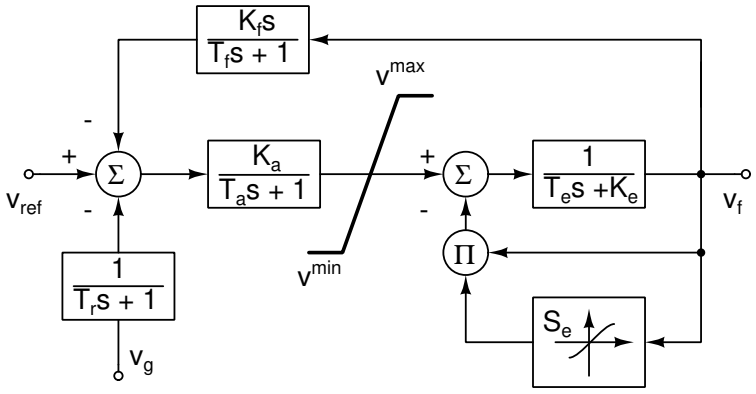

Figure 8.4: The block schematic of the type II automatic voltage regulator.

$$
\begin{aligned}
&T_r \frac{dv_m}{dt} + v_m - v_g = 0 \\
&T_a \frac{dv_{r1}}{dt} + v_{r1} - K_a\left(v_{\text{ref}} - v_m - v_{r2} - \frac{K_f}{T_f} v_f\right) = 0 \\
&v_r = \begin{cases} v_{r1} & \text{if } v^{\min} \le v_{r1} \le v^{\max} \\ v^{\max} & \text{if } v_{r1} > v^{\max} \\ v^{\min} & \text{if } v_{r1} < v^{\min} \end{cases} \\
&T_f \frac{dv_{r2}}{dt} + v_{r2} + \frac{K_f}{T_f} v_f = 0 \\
&T_e \frac{dv_f}{dt} + v_f\left(K_e + A_e e^{B_e |v_f|}\right) - v_r = 0
\end{aligned}
$$

8.3.2 Center of inertia

An instance of a center of inertia (COI) is for example

```
Coi powercoi gen="G1" gen="G2" type=1
```

where `Coi` is the name of the `powercoi` instance. There can be several synchronous generators that contribute to this center on inertia. If the angular frequency of the center of inertia is needed it can be accessed by the `coi` terminal as shown by the

```
Coi coi powercoi gen="G1" gen="G2" type=1
```

statement. The `gen` parameters link the generators to the center of inertia; there can be as many `gen` parameters as the number of synchronous generators in the power netlist. These generators contribute to the center of inertia.
The parameter `attach` links the generators to the center on inertia, but they do not contribute to the center of inertia. As before there can be as the number of synchronous generators in the power netlist.
The generators that can contribute to the center of inertia through the `gen` parameter are the synchronous ones (`powergenerator`) and the permanent magnet synchronous generators. The generators contributing to the center of inertia, determine the equivalent rotor speed of the center of inertia as

$$\omega_{\text{coi}} = \frac{\sum_{k=1}^{K} M_k P_k \omega_k}{\sum_{k=1}^{K} M_k P_k}$$

where M_m is mechanical starting time (twice the value of the inertia constant: $M = 2H$) of the m-th generator contributing to the center of inertia, P_k is its rated power and K is the total number of synchronous generators contributing to the center of inertia.
The mechanical angle of the center of inertia is

$$\delta_{\text{coi}} = \frac{\sum_{k=1}^{K} M_k P_k \delta_k}{\sum_{k=1}^{K} M_k P_k}$$

The synchronous generators contributing (gen) and simply linked (attach) to the center of inertia modify their swing equation as

$$M\frac{d\omega}{dt} = (p_m - p_e - D(\omega - \omega_{\text{coi}})) \tag{8.3}$$

where quantities are in *per-unit*, when the `type=1` or type=3 parameter of the `powercoi` instance is set.
The phase equation of the linked synchronous generators modifies as

$$\frac{d\delta}{dt} = \Omega_B(\omega - \omega_{\text{coi}}) \tag{8.4}$$

when the `type=2` or type=3 parameter of the `powercoi` instance is set.

The swing equation of the permanent magnet synchronous generator (`powerpsmg`) is

$$J\frac{d\widetilde{\omega}}{dt} = (T_m - T_e - F\widetilde{\omega} - D(P\widetilde{\omega} - \Omega_B\omega_{\text{coi}})) \tag{8.5}$$

where ω in (8.3) is in per-unit and $\widetilde{\omega}$ is in $[\text{rad}\,\text{s}^{-1}]$, that is, $\widetilde{\omega} = \Omega_B\omega$. It is recast as

$$J\Omega_B^2\frac{d\omega}{dt} = \Omega_b(T_m - T_e - F\widetilde{\omega} - D(P\widetilde{\omega} - \omega_B\omega_{\text{coi}})) = \widetilde{p}_m - \widetilde{p}_e - \Omega_B F\widetilde{\omega} + \Omega_B(P\widetilde{\omega} - \Omega_B\omega_{\text{coi}}) \tag{8.6}$$

We have that $M_{\text{pmsg}}P_{\text{rat}} = J\Omega_B^2$ and finally we obtain $M_{\text{pmsg}} = \dfrac{J\Omega_B^2}{P_{\text{rat}}}$, where P_{rat} is the rated power of the `powerpmsg`. When a `powerpmsg` is connected to a center on inertia, it contributes with the M_{pmsg} coefficient.

8.3.3 Induction machines

The induction machine instance is

```
G1 bus1 tm powerinduct type=1 prating=10M vrating=16.5k rr1=0.01 xm=1M rs=1m \
xs=2.5m xr1=2.5m hm=0.5
```

`bus1` is the but at which the induction machine is connected. The `tm` terminal is optional, if used it sets the mechanical torque applied to the machine.

Order I – `type=1`

The equations modeling this induction machine are

$$
\begin{aligned}
&2H_m\frac{d\omega_p}{dt}-\tau_m(\omega_p)+p=0\\
&p+\frac{r_{R1}\,|v|^2\,\sigma}{(r_S\sigma+r_{R1})^2+\sigma^2(x_S+x_{R1})^2}=0\\
&q+\frac{v^2}{x_m}+\frac{\sigma^2(x_S+r_{R1})v^2}{(\sigma r_S+r_{R1})^2+\sigma^2(x_S+x_{R1})^2}=0
\end{aligned}
$$

where $\sigma = 1-\omega_p$ is the slip and ω_p is the rotor angular speed (per-unit) and v is the voltage of the bus at which in induction machine is connected.
An improved version of the sleep that accounts for the inertia of the induction machine, i.e. for the stored energy, is $\sigma = \omega_{coi}-\omega_p$.
If the `tm` terminal is omitted the torque/speed characteristic

$$\tau_m = a+b\omega_p+c\omega_p^2$$

is used in the induction machine model, where a, b and c are real parameters.

Detailed dynamic model (single cage) – `type=31`

The flux equations of a single cage asynchronous machine in the DQ0 frame are

$$
\begin{bmatrix}\Phi_d(t)\\ \Phi_q(t)\\ \Phi_0(t)\\ \Phi_D(t)\\ \Phi_Q(t)\\ \Phi_Z(t)\end{bmatrix}=\begin{bmatrix}L_s&0&0&L_m&0&0\\0&L_s&0&0&L_m&0\\0&0&L_s&0&0&0\\L_m&0&0&L_r&0&0\\0&L_m&0&0&L_r&0\\0&0&0&0&0&L_r\end{bmatrix}\begin{bmatrix}i_d(t)\\ i_q(t)\\ i_0(t)\\ i_D(t)\\ i_Q(t)\\ i_Z(t)\end{bmatrix}
$$

where small case suffixes refer to the stator and upper case ones refer to the rotor, the $_Z$ suffix refers to the 0 component of the rotor, L_m is the mutual inductance, L_s is the leakage inductance of the stator and L_r is the leakage inductance of the rotor.

In the DQ0 frame by using the Park's transform the differential equations governing the electrical behaviour od the asynchronous machine are

$$
\begin{bmatrix}v_d(t)\\ v_q(t)\\ v_0(t)\\ v_D(t)\\ v_Q(t)\\ v_Z(t)\end{bmatrix}=\begin{bmatrix}R_s&0&0&0&0&0\\0&R_s&0&0&0&0\\0&0&R_s&0&0&0\\0&0&0&R_r&0&0\\0&0&0&0&R_r&0\\0&0&0&0&0&R_r\end{bmatrix}\begin{bmatrix}i_d(t)\\ i_q(t)\\ i_0(t)\\ i_D(t)\\ i_Q(t)\\ i_Z(t)\end{bmatrix}+\begin{bmatrix}-\omega\Phi_q(t)\\ \omega\Phi_d(t)\\ 0\\ (\omega_r-\omega)\Phi_Q(t)\\ (\omega-\omega_r)\Phi_D(t)\\ 0\end{bmatrix}+\frac{d}{dt}\begin{bmatrix}\Phi_d(t)\\ \Phi_q(t)\\ \Phi_0(t)\\ \Phi_D(t)\\ \Phi_Q(t)\\ \Phi_Z(t)\end{bmatrix}
\tag{8.7}
$$

where R_s is the stator parasitic resistance and R_r is the rotor parasitic resistance and $\omega_r = p\omega_m$ is the rotor electrical angular speed being ω_m is the rotor mechanical angular speed and p the numer of pole pairs. For example a machine with 2 poles leads to a 1 pole-pair.
The following mechanical equation completes the model

$$H_m \left(\frac{2P_r}{\omega^2}\right) p\frac{d\omega_m(t)}{dt} = \overbrace{\Phi_d(t)i_q(t) - \Phi_q(t)i_d(t)}^{\tau_e} - \tau_m(t) - p\frac{D}{\omega}|\omega_m(t)|$$

where $H_m = \dfrac{J_m\omega^2}{2P_r}$ is the machine inertia, J_m is the inertia moment, P_r is the machine rated power, D is damping, $\tau_m(t)$ is the external mechanical torque applied to the rotor of the machine and τ_e is the electrical torque.

Order III simplified model – `type=3`

The simplified model of the asynchronous machine that used in single-phase equivalent models of power systems is obtained by neglecting the $\frac{d\Phi_d(t)}{dt}$ and $\frac{d\Phi_q(t)}{dt}$ derivatives in (8.7), the 0 components of the electrical variables and fluxes and by considering the rotor short circuited. With these assumptions we obtain

$$\underbrace{\begin{bmatrix} R_s & 0 \\ 0 & R_s \end{bmatrix}}_{\mathbf{R_s}} \begin{bmatrix} i_d(t) \\ i_q(t) \end{bmatrix} + \underbrace{\begin{bmatrix} 0 & -\omega \\ \omega & 0 \end{bmatrix}}_{\Omega} \begin{bmatrix} \Phi_d(t) \\ \Phi_q(t) \end{bmatrix} = \begin{bmatrix} v_d(t) \\ v_q(t) \end{bmatrix} \tag{8.8}$$

and

$$\underbrace{\begin{bmatrix} R_r & 0 \\ 0 & R_r \end{bmatrix}}_{\mathbf{R_r}} \begin{bmatrix} i_D(t) \\ i_Q(t) \end{bmatrix} + \begin{bmatrix} (\omega_r - \omega)\Phi_Q(t) \\ (\omega - \omega_r)\Phi_D(t) \end{bmatrix} + \frac{d}{dt}\begin{bmatrix} \Phi_D(t) \\ \Phi_Q(t) \end{bmatrix} = 0 \tag{8.9}$$

Equation (8.8) is rewritten as

$$\begin{bmatrix} v_d(t) \\ v_q(t) \end{bmatrix} = \left(\mathbf{R_s} + \left(L_m + L_s - \frac{L_m^2}{L_m + L_r}\right)\Omega\right)\begin{bmatrix} i_d(t) \\ i_q(t) \end{bmatrix} + \frac{L_m}{L_m + L_r}\Omega\begin{bmatrix} \Phi_D(t) \\ \Phi_Q(t) \end{bmatrix} \tag{8.10}$$

Now we set

$$\begin{bmatrix} e'_d(t) \\ e'_q(t) \end{bmatrix} = \frac{L_m}{L_m + L_r}\Omega\begin{bmatrix} \Phi_D(t) \\ \Phi_Q(t) \end{bmatrix} \quad \text{and} \quad T'_0 = \frac{L_m + L_r}{R_r} \tag{8.11}$$

Equation (8.9) is rewritten as

$$\mathbf{R_r}\begin{bmatrix} i_D(t) \\ i_Q(t) \end{bmatrix} + \underbrace{\begin{bmatrix} 0 & \omega_r - \omega \\ \omega - \omega_r & 0 \end{bmatrix}}_{\Omega_s}\begin{bmatrix} \Phi_D(t) \\ \Phi_Q(t) \end{bmatrix} + \frac{L_m + L_r}{L_m}\Omega^{-1}\frac{d}{dt}\begin{bmatrix} e'_d(t) \\ e'_q(t) \end{bmatrix} = 0 \tag{8.12}$$

The rotor current vector can be expresses as a function of rotor flux and stator current as

$$\begin{bmatrix} i_D(t) \\ i_Q(t) \end{bmatrix} = \frac{1}{L_m + L_r}\begin{bmatrix} \Phi_D(t) \\ \Phi_Q(t) \end{bmatrix} - \frac{L_m}{L_m + L_r}\begin{bmatrix} i_d(t) \\ i_q(t) \end{bmatrix}$$

that substituted in Equation (8.13) leads to

$$\left(\mathbf{R_r}\frac{1}{L_m + L_r} + \Omega_s\right)\frac{L_m + L_r}{L_m}\Omega^{-1}\begin{bmatrix} e'_d \\ e'_q \end{bmatrix} - \mathbf{R_r}\frac{L_m}{L_m + L_r}\begin{bmatrix} i_d \\ i_q \end{bmatrix} + \frac{L_m + L_r}{L_m}\Omega^{-1}\frac{d}{dt}\begin{bmatrix} e'_d \\ e'_q \end{bmatrix} = 0$$

(8.13)

Equation (8.14) is a differential equation in the $e_d^{'}(t)$ and $e_q^{'}(t)$ new variables, involving only the stator currents. By substituting $T_0^{'}$ in Equation (8.14) we obtain

$$\frac{1}{T_0^{'}}\frac{L_m+L_r}{L_m}\Omega^{-1}\begin{bmatrix} e_d^{'} \\ e_q^{'} \end{bmatrix}+\Omega_s\Omega^{-1}\frac{L_m+L_r}{L_m}\begin{bmatrix} e_d^{'} \\ e_q^{'} \end{bmatrix}-\frac{1}{T_0^{'}}L_m\begin{bmatrix} i_d \\ i_q \end{bmatrix}+\frac{L_m+L_r}{L_m}\Omega^{-1}\frac{d}{dt}\begin{bmatrix} e_d^{'} \\ e_q^{'} \end{bmatrix}=0 \tag{8.14}$$

We introduce the new paramaters

$$\mathbf{x_0}=\Omega\left(L_m+L_s\right) \quad \text{and} \quad \mathbf{x}^{'}=\Omega\left(L_s+L_m-\frac{L_m^2}{L_m+L_r}\right) \quad \text{note that} \quad \mathbf{x_0}-\mathbf{x}^{'}=\Omega\frac{L_m^2}{L_m+L_r}$$

By substituting these new parameter in Equations (8.14) and (8.10) we obtain

$$\begin{aligned}\frac{d}{dt}\begin{bmatrix} e_d^{'} \\ e_q^{'} \end{bmatrix}&=-\Omega_s\begin{bmatrix} e_d^{'} \\ e_q^{'} \end{bmatrix}-\frac{1}{T_0^{'}}\begin{bmatrix} e_d^{'} \\ e_q^{'} \end{bmatrix}-\left(\mathbf{x_0}-\mathbf{x}^{'}\right)\frac{1}{T_0^{'}}\begin{bmatrix} i_d \\ i_q \end{bmatrix}\\ \begin{bmatrix} v_d \\ v_q \end{bmatrix}&=\begin{bmatrix} e_d^{'} \\ e_q^{'} \end{bmatrix}+\left(\mathbf{R_s}+\mathbf{x}^{'}\right)\begin{bmatrix} i_d \\ i_q \end{bmatrix}\end{aligned}$$

that implements the `type=3` model of the asynchronous machine.

8.3.4 Synchronous generators

Two instances of a synchronous generators are shown in the Netlist 8.0.1. The instance names are `G1` and `G2`, and the built-in model is `powergenerator`. PAN implements several built-in models of synchronous generators. In the determination of the PF solution (DC analysis) the synchronous generator behaves like a constant voltage and active power element (PV generator). It forces the modulus of voltage of the bus at which it is connected to have a fixed magnitude and generates a fixed active power, determined by the parameters of the synchronous generator. When during a PF analysis a synchronous generator is tagged as *slack* it behaves like an independent voltage source that can deliver/absorb any quantity of active power. A *slack* generator imposed the modulus of the bus voltage and sets the phase of the bus voltage. In any case during PF analysis, reactive power of synchronous generators is determined by the power grid to which it is connected.
A simplified equivalent schematic of the synchronous generator model is shown in Figure 8.5 The E_o equivalent voltage source is modelled in very different ways according to the type of

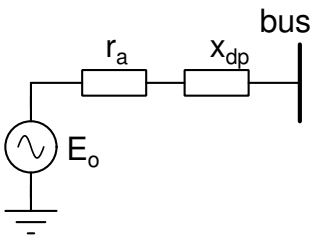

Figure 8.5: The simplified schematic of the synchronous generator.

analysis performed, i.e. PF analysis or time domain analysis and to the order of the model.

The active power generated by the synchronous generator at the connection bus is available in the "devvars" output file during several time domain analyses. The name of this output variable

is the composition of the instance name of the generator and the ":pe" suffix. For example "G1:pe" is the active power at the connection bus of the "G1" synchronous generator. Similarly the reactive power at the connection bus is made available in the "devvars" output file with the suffix ":qe". The internal active electrical power used in the swing equation of the synchronous generator, which thus takes in to account the ohmic losses, is once more available among the "devvars" output variables with the ":pei" suffix.

Order 0 – `type=0`

The order 0 model of the synchronous generator implements a PV source. It is an algebraic model with thus no dynamic.
The active power that drives the PF solution is

$$p_e = (v_q + r_a i_q)\, i_q + (v_d + r_a i_d)\, i_d$$

where v_d and v_q are the voltage components of the `bus` voltage and i_d and i_q the components of the current leaving the bus and entering the synchronous generator model. r_a and x_a are the internal resistance and reactance of stator windings. The following relationships between voltages and currents hold

$$\begin{aligned} v_q + r_a i_q - e_{qp} + x_{dp} i_d &= 0 \\ v_d + r_a i_d - x_{dp} i_q &= 0 \end{aligned}$$

where $|v_{\text{bus}}| = \sqrt{v_d^2 + v_q^2}$. The $|v_{\text{bus}}|$ and p_e are given parameters of the synchronous generator model and finally e_{qp} is the fixed voltage of $\mathtt{E_o}$.
During a DC analysis, that determines the power flow solution, the 0 order model can act as a slack synchronous generator or not. If it acts as a slack synchronous generator, it implements an independent voltage source at the but to which it is connected. If it is not a slack generator it behaves like a PV generator. In all the other analyses it always behave as a PV generator.

Order II – `type=2`

This is the classic electro-mechanical model, with constant amplitude e.m.f. e_{qp}. The state variables are δ and ω. The effects of the leakage reactance and the armature resistance can be included. The differential equations describing this synchronous generator are

$$\begin{aligned} \frac{d\delta}{dt} &= \Omega_B\,(\omega - 1) \\ M\frac{d\omega}{dt} &= (p_m - p_e - D\,(\omega - 1)) \end{aligned}$$

where ω_b is the base frequency in rad/s and the electrical power p_e is defined as

$$p_e = (v_q + r_a i_q)\, i_q + (v_d + r_a i_d)\, i_d$$

Finally, the following relationships between voltages and currents hold

$$\begin{aligned} v_q + r_a i_q - e_{qp} + x_{dp} i_d &= 0 \\ v_d + r_a i_d - x_{dp} i_q &= 0 \end{aligned}$$

An Automatic Voltage Regulator should not be connected to order II synchronous machines.

Order III – `type=3`

In this model all the q-axis electromagnetic circuits are neglected, whereas a lead-lag transfer function is used for the d-axis inductance. The state variables are δ, ω and e_{qp}. The differential equations describing this synchronous generator are

$$\begin{aligned}
&\frac{d\delta}{dt} = \Omega_B(\omega - 1)\\
&M\frac{d\omega}{dt} = (p_m - p_e - D(\omega - 1))\\
&T_{d0p}\frac{de_{qp}}{dt} + e_{qp} + \left(x_d - x_{dp}\right) i_d + v_f^* = 0
\end{aligned}$$

where ω_b is the base frequency in rad/s and the electrical power p_e is defined as

$$p_e = (v_q + r_a i_q)\, i_q + (v_d + r_a i_d)\, i_d$$

Finally, the following relationships between voltages and currents hold

$$\begin{aligned}
&v_q + r_a i_q - e_{qp} + x_{dp} i_d = 0\\
&v_d + r_a i_d - x_q i_q = 0
\end{aligned}$$

This is the simplest model of the synchronous generator that admints the connection of an Automatic Voltage Regulator.

Order V – `type=52`

This model assumes only one additional circuit of the d-axis. The resulting model has the five δ, ω, e_{qp}, e_{qs}, and e_{ds} state variales. The differential equations describing this synchronous generator are

$$\begin{aligned}
&\frac{d\delta}{dt} = \Omega_B(\omega - 1)\\
&M\frac{d\omega}{dt} = (p_m - p_e - D(\omega - 1))\\
&T_{d0p}\frac{de_{qp}}{dt} + e_{qp} + \left(x_d - x_{dp} - \frac{T_{d0s}}{T_{d0p}}\frac{x_{ds}}{x_{dp}}\left(x_d - x_{dp}\right)\right) i_d + \left(\frac{T_{aa}}{T_{d0p}} - 1\right) v_f^* = 0\\
&T_{d0s}\frac{de_{qs}}{dt} + e_{qs} - e_{qp} + \left(x_{dp} - x_{ds} + \frac{T_{d0s}}{T_{d0p}}\frac{x_{ds}}{x_{dp}}\left(x_d - x_{dp}\right)\right) i_d - \frac{T_{aa}}{T_{d0p}} v_f^* = 0\\
&T_{q0s}\frac{de_{ds}}{dt} + e_{ds} - (x_q - x_{qs})\, i_q = 0
\end{aligned}$$

where Ω_B is the base frequency in rad/s and the electrical power p_e is defined as follows

$$p_e = (v_q + r_a i_q)\, i_q + (v_d + r_a i_d)\, i_d$$

Finally, the following relationships between voltages and currents hold

$$\begin{aligned}
&v_q + r_a i_q - e_{qs} + x_{ds} i_d = 0\\
&v_d + r_a i_d - e_{ds} - x_{qs} i_q = 0
\end{aligned}$$

Order V – `type=53`

This is the basic model that considers the electromagnetic behaviour of the synchronous machine in a detailed way. The effects of rotor speed on fluxes that thus voltages are considered along

with field flux dynamics. The state variables are δ, ω, e_{qp}, ϕ_d and ϕ_q. The differential equations describing this synchronous generator are

$$\begin{aligned}
&\frac{d\delta}{dt} = \Omega_B(\omega - 1)\\
&M\frac{d\omega}{dt} = (p_m - p_e - D(\omega - 1))\\
&T_{d0p}\frac{de_{qp}}{dt} + \frac{x_d}{x_{dp}}\left(e_{qp} - v_f^* + \left(1 - \frac{x_{dp}}{x_d}\right)T_{d0p}\Omega(v_d + r_a i_d + \omega\phi_q)\right) = 0\\
&\frac{d\phi_d}{dt} - \Omega_B(v_d + r_a i_d + \omega\phi_q)\\
&\frac{d\phi_q}{dt} - \Omega_B(v_q + r_a i_q - \omega\phi_d)
\end{aligned}$$

where ω_b is the base frequency in rad/s and the electrical power p_e is defined as

$$p_e = \omega(\phi_d i_q - \phi_q i_d)$$

The following algebraic equations complete the model

$$\begin{aligned}
&\phi_d + x_d i_d - e_{qp} = 0\\
&\phi_q + x_q i_q = 0
\end{aligned}$$

Order VI – `type=6`

The sixth order model is obtained assuming the presence of a field circuit and an additional circuit along the d-axis and two additional circuits along the q-axis. The system has six state variables δ, ω, e_{dp}, e_{ds}, e_{qp} and e_{qs}. The model is described as

$$\begin{aligned}
&\frac{d\delta}{dt} = \Omega_B(\omega - 1)\\
&M\frac{d\omega}{dt} = (p_m - p_e - D(\omega - 1))\\
&T_{d0p}\frac{de_{qp}}{dt} + e_{qp} + \left(x_d - x_{dp} - \frac{T_{d0s}}{T_{d0p}}\frac{x_{ds}}{x_{dp}}(x_d - x_{dp})\right)i_d + \left(\frac{T_{aa}}{T_{d0p}} - 1\right)v_f^* = 0\\
&T_{q0p}\frac{de_{dp}}{dt} + e_{dp} - \left(x_q - x_{qp} - \frac{T_{q0s}}{T_{q0p}}\frac{x_{qs}}{x_{qp}}(x_q - x_{qp})\right)i_q = 0\\
&T_{d0s}\frac{de_{qs}}{dt} + e_{qs} - e_{qp} + \left(x_{dp} - x_{ds} + \frac{T_{d0s}}{T_{d0p}}\frac{x_{ds}}{x_{dp}}(x_d - x_{dp})\right)i_d - \frac{T_{aa}}{T_{d0p}}v_f^* = 0\\
&T_{q0s}\frac{de_{ds}}{dt} + e_{ds} - e_{dp} - \left(x_{qp} - x_{qs} + \frac{T_{q0s}}{T_{q0p}}\frac{x_{qs}}{x_{qp}}(x_q - x_{qp})\right)i_q = 0
\end{aligned}$$

where ω_b is the base frequency in rad/s and the electrical power p_e is defined as follows

$$p_e = (v_q + r_a i_q)i_q + (v_d + r_a i_d)i_d$$

Finally, the following relationships between voltages and currents hold

$$\begin{aligned}
&v_q + r_a i_q - e_{qs} + x_{ds} i_d = 0\\
&v_d + r_a i_d - e_{ds} + x_{qs} i_q = 0
\end{aligned}$$

This model is similar to the VIII (`type=8`) one, but with the assumptions $\frac{d\phi_d}{dt} = \frac{d\phi_q}{dt} = 0$ and $\omega\phi_d \simeq \phi_d$, $\omega\phi_q \simeq \phi_q$, that is, the electromagnetic dynamic behaviour of the generator is not considered.

8.3.5 Permanent magnet synchronous generator

The permanent magnet synchronous machine (`powerpmsg`) operates in either generator or motor mode. The mode of operation is dictated by the sign of the mechanical torque (negative for motor mode, positive for generator mode) at the *tm* terminal. The electrical and mechanical parts of the machine are each represented by the

$$\frac{dx}{dt} = \frac{x}{\sqrt{x^2+y^2}} - (1+\alpha)x - (P\omega - \Omega_B)\,y$$
$$\frac{dy}{dt} = \frac{y}{\sqrt{x^2+y^2}} - (1+\alpha)y + (P\omega - \Omega_B)\,x$$
$$\sqrt{x^2+y^2} = 1$$
$$J\frac{d\omega}{dt} = T_m - T_e - F\omega - D\,(P\omega - \Omega_B)$$
$$v_d = R_s i_d + L_d \frac{di_d}{dt} - L_q P\omega i_q + \lambda P\omega x$$
$$v_q = R_s i_q + L_q \frac{di_q}{dt} + L_d P\omega i_d + \lambda P\omega y$$
$$T_e = -\frac{3}{2} P\lambda\,(x i_d + y i_q)$$

model where v_d and v_q are the 'd-axis' and 'q-axis' components of the voltage of bus to which the permanent magnet synchronous generator is conneced to, i_d and i_q are the 'd-axis' and 'q-axis' components of the stator current, which flows positive from the bus to the machine (normal convention), R_s is the stator resistance, L_d and L_q are the d-axis and q-axis inductances, Ω_B is the reference system angular frequency (in general $2\pi\,50\,\text{Hz}$ or $2\pi\,60\,\text{Hz}$), λ is the magnitude of the flux induced by the permanent magnets of the rotor in the stator phases, P is the number of poles pairs, T_e is the electrical torque, T_m is the mechanical rorque, H is the inertia coefficient, D is the damping coefficient, F is the friction coefficient,
The first three equations compute the position of the rotor in the Chartesian reference frame, that is, if we assume δ is the angular position of the rotor, we have $x = \cos(\delta)$ and $y = \sin(\delta)$. The sinusoidal model assumes that the (λ) flux established by the permanent magnets in the stator is sinusoidal, which implies that the electromotive forces are constant in the dq-frame at the synchronous (Ω_B) system angular frequency. The `powerpmsg` has different types of initialisation determined by the `init` parameter.

- `[init=0]` takes as input the mechanical torque read from the `tm` terminal and the `omegab` parameter, which sets the rotor angular speed. The model determines the λ flux such that the electrical torque matches the mechanical one.
- `[init=1]` takes as input the mechanical torque and the λ flux and determine the angular rotor speed such that the electrical torque matched the mechanical one. The rotor speed is available as output at the `omega` terminal.
- `[init=2]` sets the `powerpmsg` to work as a 'slack' generator that forces the `vg` voltage at its electrical bus, active and reactive power are derived accordingly.
- `[init=3]` sets the `powerpmsg` to generate the `pg` active power and the `vg` modulus at the electrical bus. The reactive power is derived accordingly.

8.3.6 Polar/Cartesian representation of the phase of the synchronous generator

The representation of the differential equation that governs the δ phase of the synchronous generators does not allow to use methods that compute the steady-state solution since, the value of δ diverges when the electrical solution lays on an orbit not at the reference frequency. To

avoid this problem the δ differential equation of each of the synchronous generator models described in the previous subsections is reformulated in the Cartesian reference frame. The user can select which kind of representation is used in each model of synchronous generator.
More specifically the equation governing δ in Carterian coordinates is

$$\begin{aligned} \frac{d\rho}{dt} &= k\,(\rho_o - (1+\alpha)\rho) \\ \frac{d\delta}{dt} &= \Omega_B\,(\omega - 1) \quad , \end{aligned} \tag{8.15}$$

which describes a closed orbit in the xy-plane that should describe a circle of ρ_o radius. By introducing the $x = \rho_o \cos(\delta)$ and $y = \rho_o \sin(\delta)$ transformations, after some algebraic handling we obtain

$$\begin{aligned} \frac{dx}{dt} &= \frac{k\rho_o}{\sqrt{x^2+y^2}}x - k(1+\alpha)x - \Omega_B\,(\omega-1)\,y \\ \frac{dy}{dt} &= \frac{k\rho_o}{\sqrt{x^2+y^2}}y - k(1+\alpha)y + \Omega_B\,(\omega-1)\,x \quad . \end{aligned} \tag{8.16}$$

The x, y solutions of (8.16) are periodic functions of period $\Omega_B\,(\omega-1)$. The $1/k$ parameter in (8.15) is the time constant that governs how a point not laying on the steady state orbit falls on it. The ρ_o parameter defines the radius of the circle in the xy-plane that represents the steady state orbit. We chose $\rho_0 = 1$.
The α term should be chosen equal to 0 in order to have $\rho = \rho_o$ at steady state. However the numerical integration of (8.16) introduces "warping" of the solution and especially a "dissipative" term that dumps the solution. This means that the close orbit does not lay on the unit circle and for example may collapse close to or even in the $x = 0$, $y = 0$ origin. We thus add an extra algebraic equation to (8.16), which ensures the steady state orbit must lay on the circle with ρ_o radius. The complete equation system is

$$\begin{aligned} \frac{dx}{dt} &= \frac{k\rho_o}{\sqrt{x^2+y^2}}x - k(1+\alpha)x - \Omega_B\,(\omega-1)\,y \\ \frac{dy}{dt} &= \frac{k\rho_o}{\sqrt{x^2+y^2}}y - k(1+\alpha)y + \Omega_B\,(\omega-1)\,x \\ \sqrt{x^2+y^2} &= \rho_o \quad . \end{aligned}$$

Since we have added a new equation we need a new unknown to have a well posed problem. The α parameter is now an *unknown* or better said an unknown time domain function. It will assume values so that to compensate the "warping" by the integration method and to keep the orbit on the unit circle in the xy-plane.
To have an insight in the collapsing mechanism, assume to solve (8.16) with the implicit Euler integration method and set $\alpha = 0$. We obtain the nonlinear algebraic equation

$$\begin{aligned} \frac{x_{n+1}-x_n}{h} &= \frac{k\rho_o}{\sqrt{x_{n+1}^2+y_{n+1}^2}}x_{n+1} - kx_{n+1} - \Omega_B\,(\omega-1)\,y_{n+1} \\ \frac{y_{n+1}-y_n}{h} &= \frac{k\rho_o}{\sqrt{x_{n+1}^2+y_{n+1}^2}}y_{n+1} - ky_{n+1} + \Omega_B\,(\omega-1)\,x_{n+1} \quad , \end{aligned} \tag{8.17}$$

from which we can derive the x_{n+1}, y_{n+1} values from the x_n, y_n ones. Since we want the orbit stays on the unit circuit, we require $\left(x_{n+1}^2 + y_{n+1}^2 = x_n^2 + y_n^2\right)$. Algebraic handling of Eq. (8.17)

leads to

$$\overbrace{\left((1+hk)^2+h^2\Omega_B^2(\omega-1)^2\right)\chi^2-2hk\rho_o(1+hk)\chi+h^2k^2\rho_o^2}^{\zeta}-\left(x_n^2+y_n^2\right)=0\ , \qquad (8.18)$$

where $\chi=\sqrt{x_{n+1}^2+y_{n+1}^2}$. We require that $\chi=\sqrt{x_n^2+y_n^2}$. From Eq. (8.18) we see that

$$\lim_{h\to 0}\zeta=\chi^2$$

and thus $\chi^2=\left(x_n^2+y_n^2\right)$. This tells that if we were able to advance in time by setting $h\to 0$, the orbit would be "preserved".
Now consider the limit

$$\lim_{h\to\infty}\zeta=\lim_{h\to\infty}h^2\left[\left(k^2+\Omega_B^2(\omega-1)^2\right)\chi^2-2k^2\rho_o^2\chi+k^2\rho_o^2\right]$$

To have the limit result be a finite number, we pretend that

$$\left(k^2+\Omega_B^2(\omega-1)^2\right)\chi^2-2k^2\rho_o^2\chi+k^2\rho_o^2=0 \qquad (8.19)$$

The following simple algebraic handling

$$\left(k^2+\Omega_B^2(\omega-1)^2\right)\rho_o^2-2k^2\rho_o^2\rho_o+k^2\rho_o^2=0$$
$$\left(k^2+\Omega_B^2(\omega-1)^2\right)-2k^2\rho_o+k^2=0$$

shows that ρ_o must equal $\left(1+\frac{\Omega_B^2(\omega-1)^2}{2k^2}\right)$ when $h\to\infty$. This is true only when $\omega=1$. For sufficiently large values of h and with $\omega\neq 1$, we can see that the solution does not remain on the unit circle ($\rho_o=1$).
As already said to avoid this, we add the algebraic constraint in Eq. (8.16) (last equation), which forces α to assume values such that the $\chi=\left(x_n^2+y_n^2\right)$ constraint is satisfied for any value of the h time integration step during the computation of the time domain solution.

8.3.7 Line

The schematic of the power line model is shown in Figure 8.7. An example of a power line

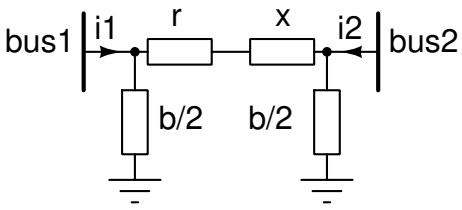

Figure 8.6: The schematic of the power line.

instance is

```
Line1 bus1 bus2 powerline r=56.95m x=0.17388 b=34m prating=100M vrating=69k
```

that connects the bus1 and bus2 buses; r is the resistance of the line, x is the inductive impedance (positive) and b/2 is the capacitive admittance (positive). All units are in PU.
The operating point of the power line can be printed by setting true the print=true parameter. The information reported in the operating point are the i1 and i2 currents, i1 can be different from i2 due to the contributions by the shunt b/2 capacitors, the power at bus1 (positive active means that active power enters the line) and the power at bus2. Power at bus2 can be different from that at bus1 due to the conrtribution by line parasitics (r, x and b/2). For example the operating point

```
P1:  36.331 MW Q1:  3.2362 MVAR Id1:  158.71  A Iq1:  1.4427  A
P2: -34.865 MW Q2:  11.422 MVAR Id2: -158.71  A Iq2: -1.4427  A
Pline:  1.4658 MW   Qline:  14.658 MVAR
```

shows that at bus1 the active power (entering the line) is 36.331 MW and the reactive power 3.2362 MVAR (inductive); the active power at bus2 (entering the line) is −34.865 MW; the negative sign means that the measured active power flows in the opposite direction with respect to the meaure reference direction. The reactive power at bus2 (entering the line) is 11.422 MVAR (inductive); this measured reactive power effectively enters the line since the reactive power of the line is 14.658 MVAR (Qline). Finally the power dissipated by the line is 1.4658 MW. In the operating point printing also the Id1/Iq1 current at bus1 and the Id2/Iq2 at bus2 are reported (positive entering).
The open=yes parameter open the line, i.e., the line becomes an open circuit.

Dynamic line

The dynamic=yes parameter makes dynamic the line model. The conventional line model is algebraic and the line inductance and parasitic capacitances are considered an "impedances". When the dynamic is set 'true', the algebraic series reactance of the line is now an inductance modelled by a differential equation in the time domain. Also the algebraic shunt susceptance is now modelled in the time domain by a capacitor. Note that the value of the inductance and capacitance of the dynamic line model are derived from the corresponding values of reactances at the given frequency. Thus it is important to correctly specify the f0 parameter that sets the working frequency of the line. The initial conditions of the inductance and shut capacitance of the dynamic line model are computed through a power-flow analysis where they are considered as as impedances (see Section 8.2 for details).
When the devvars option of time domain analyses is set true, the I1 current flowing into the line from bus1 to bus2 is saved in a dedicated output file (labelled with the "devvars" suffix).

8.3.8 Load

The constant power load (type=1 argument of the powerload instance card) is modelled by the

$$p_{\mathrm{L}}(t) = P_{\mathrm{L0}}\,(1+\Delta\omega_p)^{\beta_p}\left(\frac{|v(t)|}{V_0}\right)^{\gamma_p}$$
$$q_{\mathrm{L}}(t) = Q_{\mathrm{L0}}\,(1+\Delta\omega_q)^{\beta_q}\left(\frac{|v(t)|}{V_0}\right)^{\gamma_q}$$

equations. $P_{\mathrm{LO}} = P_0\alpha_p$ is the constant load active power, where P_0 is the power rating (prating parameter of the instance) of the load and α_p is the per-unit scaling factor (pc argument). $Q_{\mathrm{LO}} = P_0\alpha_q$ is the constant load reactive power, α_q is the per-unit scaling factor (qc argument of the instance). $v(t)$ is the bus voltage at which the load is connected, V_0 is the voltage rating of the load (vrating argument), $\Delta\omega_p$ and $\Delta\omega_q$ are optional driving signals, i.e., two additional terminals of the instance, that govern how the nominal P_{LO} and Q_{LO} active and reactive power can be controlled, γ_p (ap parameter of the instance) and γ_q (ap parameter of the instance)

are exponents that define how the load active and reactive powers vary with bus voltage. If $\gamma_p = \gamma_q = 0$ we have a constant power load (if $\Delta\omega_p = 0$ and $\Delta\omega_q = 0$), if $\gamma_p = \gamma_q = 1$ we have a constant current load and if $\gamma_p = \gamma_q = 2$ we have a constant impedance load.
Examples of `powerload` instances are given in the Netlist 8.0.1 and specifically by the following two statements

```
Lo1 b1 cntp cntq powerload pc=0.549 qc=25.2m vrating=69k prating=100M
Lo2 b2 powerload pc=0.549 qc=25.2m vrating=69k prating=100M type=0 dynamic=yes
```

The `Lo1` powerload is connected to the `b1` bus, the `cntp` and `cntq` (optional) pins allow to control the active and reactive powers of the load, respectively. They implement the $\Delta\omega_p$ and $\Delta\omega_p$ variables of the power load characteristic. Note that, when the power flow is computed, the values at the `cntp` and `cntq` pins, that is, $\Delta\omega_p$ and $\Delta\omega_p$, are considered.

When we have a costant power load, i.e., $\gamma_p = 0$ and $\gamma_q = 0$, it is modeled as a non-linear voltage-controller current source described by

$$i_d = \frac{P_{\mathrm{LO}} v_d + Q_{\mathrm{LO}} v_q}{v_d^2 + v_q^2} \qquad i_q = \frac{P_{\mathrm{LO}} v_q - Q_{\mathrm{LO}} v_d}{v_d^2 + v_q^2}$$

where v_d, v_q are the 'd' and 'q' voltage components of the bus and i_d, i_q are the corresponding components of the bus current (normal convention). The derivatives of these currents with respect to the bus voltage componenets are:

$$\frac{di_d}{dv_d} = -\frac{P\left(v_d^2 - v_q^2\right) + 2Q v_d v_q}{\left(v_d^2 + v_q^2\right)^2} \qquad \frac{di_d}{dv_q} = \frac{Q_{\mathrm{LO}}\left(v_d^2 - v_q^2\right) - 2P_{\mathrm{LO}} v_d v_q}{\left(v_d^2 + v_q^2\right)^2}$$

$$\frac{di_q}{dv_d} = \frac{Q_{\mathrm{LO}}\left(v_d^2 - v_q^2\right) - 2P_{\mathrm{LO}} v_q v_d}{\left(v_d^2 + v_q^2\right)^2} \qquad \frac{di_q}{dv_q} = \frac{P\left(v_d^2 - v_q^2\right) + 2Q v_d v_q}{\left(v_d^2 + v_q^2\right)^2}$$

Dynamic load

The `dynamic=yes` parameter of the `Lo2` powerload instance makes dynamic the model of the load. This option is supported only if the load is a *constant impedance* type (`type=0` in the instance card). This means that the load is now modelled as a series connection of a resistor and an inductor or as a parallel connection of a conductance and a capacitor according to the load active and reactive involved power. The dynamic model is implemented by a differential equation in the time domain. The value of the capacitance/inductance is derived from that of the load impedance thus it is important to correctly specify the `f0` parameter that sets the working frequency of the load at the power-flow. The initial conditions of the dynamic elements are computed at the power-flow; capacitors and inductors are considered as admittances/impedances, respectively (see Section 8.2 for details).

8.3.9 Shunt

The `powershunt` contributes mainly reactive power to the bus at which it is connected. It can behave both as a constant impedance or as a constant power element. If the `q` argument is specified it behaves as a constant power load element. Positive values of `q` means capacitive load; negative means inductive load. If the `b` and possibly `g` arguments are given the `powershunt` behaves as a constant impedance loading the bus. The active power is given by $P = g\left|v_b^2\right|$ (absorbed with `g` positive) and the reactive power by $Q = b\left|v_b^2\right|$, where v_b is the bus voltage. Positive `b` refers to capacitive power.

Dynamic shunt

The `dynamic=yes` parameter makes dynamic the model of the `powershunt`. The value of the capacitance/inductance is derived from the values of the given impedance thus it is important

to correctly specify the f0 parameter that sets the working frequency of the `powershunt` at the power-flow. The initial conditions is determined at the power-flow. The capacitor is considered as an admittance (see Section 8.2 for details).

8.3.10 Transformer

The block schematic of the power transformer is shown in Figure 8.7. The power transformer

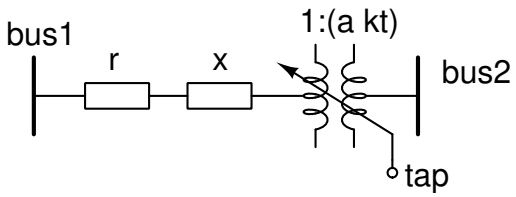

Figure 8.7: The schematic of the power transformer.

is tied between the `bus1` and `bus2` buses. The transformer impedance is described by the `r` resistance and the `x` reactance. This impedance is put in series between `bus1` and the ideal power transferrer that implements the $(a \cdot \mathrm{kt})$ tap ratio. The $(a \cdot \mathrm{kt})$ tap ratio refers to `bus2` voltage with respect to `bus1`. The voltage rating refers to `bus1`. The `tap` terminal is optional and if used it allows to vary the kt voltage ratio during an analysis. For example by assuming $a = 1$ and $\mathrm{kt} = 2$ we have that the voltage at `bus2` is twice that at `bus1` (assuming null the impedance of the transformer, i.e. $r = 0$ and $x = 0$).

Dynamic transformer

The `dynamic=yes` parameter makes dynamic the model of the transformer. This means that the algebraic impedance of the transformer is now modelled as a series connection of a resistor and an inductor. The dynamic model is implemented by a differential equation in the time domain. The value of the inductance is derived from that of the series impedance of the transformer thus it is important to correctly specify the `f0` parameter that sets the working frequency of the transformer. The initial conditions are determined by a power-flow analysis, where the dynamic part of the transformer model are discarded being the solution constant if the Park's dq-frame (see Section 8.2 for details).

8.3.11 Turbine governors

`PAN` implements several types of turbine governors.

Type I

The block schematic of the type I turbine governor is shown in Figure 8.8. Its characteristic is

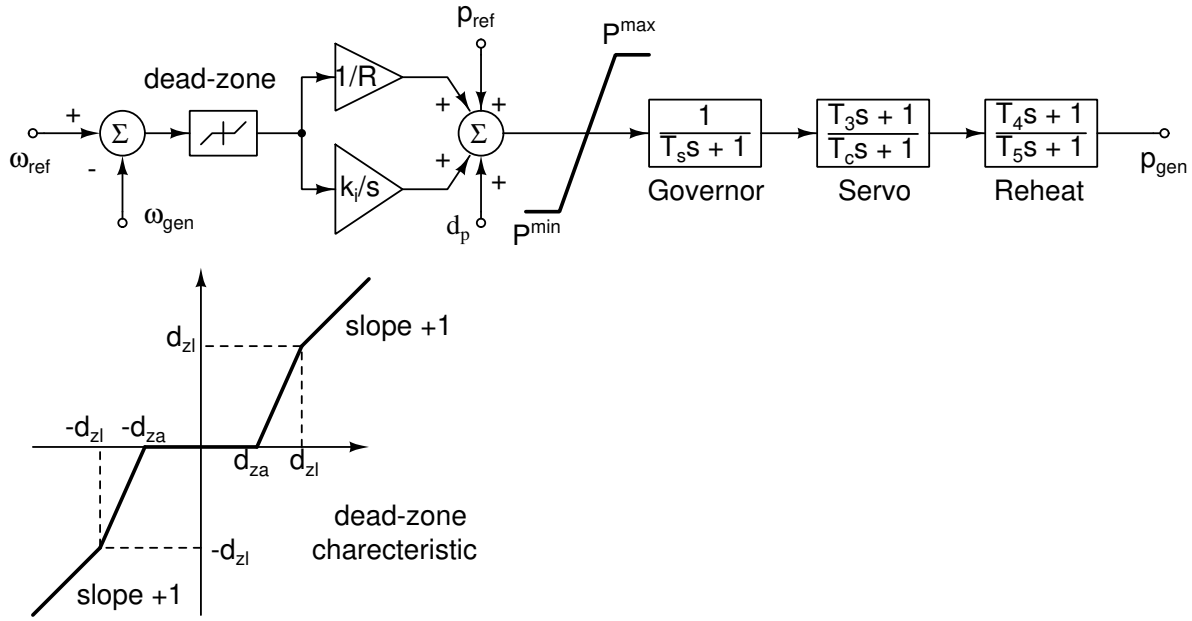

Figure 8.8: The block schematic of the type I turbine governor.

$$
\begin{aligned}
p_{in}^{*} &= p_{\text{ref}} + \frac{1}{R} f\left(\omega_{\text{ref}} - \omega_{\text{gen}}\right) + d_p \\
p_{in} &= \begin{cases} p_{in}^{*} & \text{if } p^{\min} \le p_{in}^{*} \le p^{\max} \\ p^{\max} & \text{if } p_{in}^{*} > p^{\max} \\ p^{\min} & \text{if } p_{in}^{*} < p^{\min} \end{cases} \\
T_s \frac{dx_1}{dt} &+ x_1 - p_{in} = 0 \\
T_c \frac{dx_2}{dt} &+ x_2 + \left(\frac{T_3}{T_c} - 1\right) x_1 = 0 \\
T_5 \frac{dx_3}{dt} &+ x_3 + \left(\frac{T_4}{T_5} - 1\right)\left(x_2 + \frac{T_3}{T_c} x_1\right) = 0 \\
p_m &= x_3 + \frac{T_4}{T_5}\left(x_2 + \frac{T_3}{T_c} x_1\right)
\end{aligned}
\tag{8.20}
$$

The $f\left(\omega_{\text{ref}} - \omega_{\text{gen}}\right)$ function assumes different forms according to how the dead-zone is defined.

Type II

The block schematic of the type II turbine governor is shown in Figure 8.9. Its characteristic is

$$
\begin{aligned}
\delta_\omega &= f\left(\omega_{\text{ref}} - \omega_{\text{gen}}\right) \\
T_2 \frac{dx_1}{dt} &+ x_1 - \left(1 - \frac{T_1}{T_2}\right)\left(k_i \int_0^t \delta_\omega dt + \frac{1}{R}\delta_\omega\right) = 0 \\
p_m^{*} &= x_1 + \frac{T_1}{RT_2}\delta_\omega + p_{\text{ref}} + d_p \\
p_{\text{gen}} &= \begin{cases} p_m^{*} & \text{if } p^{\min} \le p_m^{*} \le p^{\max} \\ p^{\max} & \text{if } p_m^{*} > p^{\max} \\ p^{\min} & \text{if } p_m^{*} < p^{\min} \end{cases}
\end{aligned}
\tag{8.21}
$$

The $f\left(\omega_{\text{ref}} - \omega_{\text{gen}}\right)$ function assumes different forms according to how the dead-zone is defined.

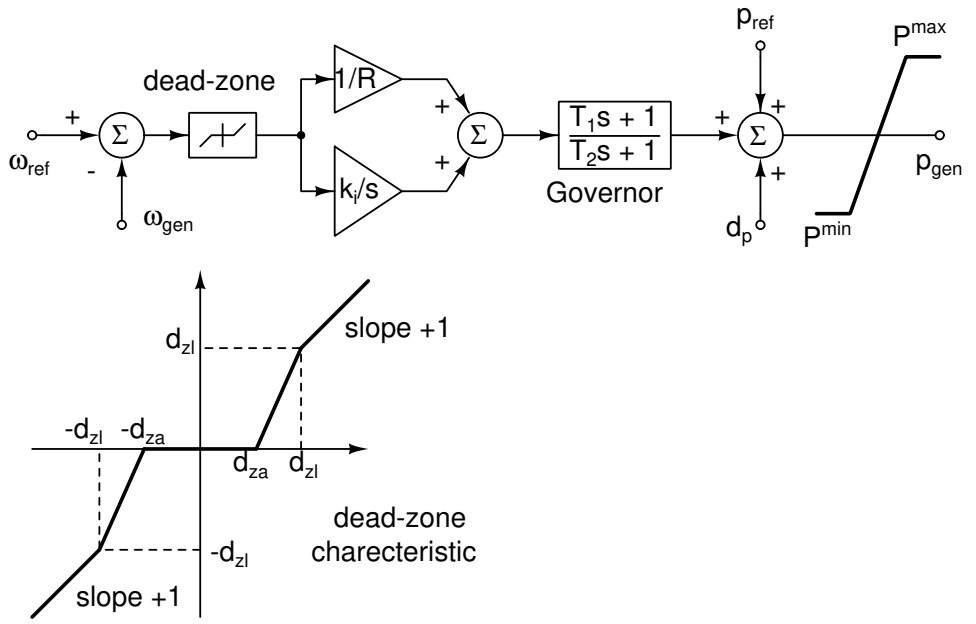

Figure 8.9: The block schematic of the type II turbine governor.

Type IEEEG1

The block schematic of the IEEEG1 type turbine governor is shown in Figure 8.10. The equations

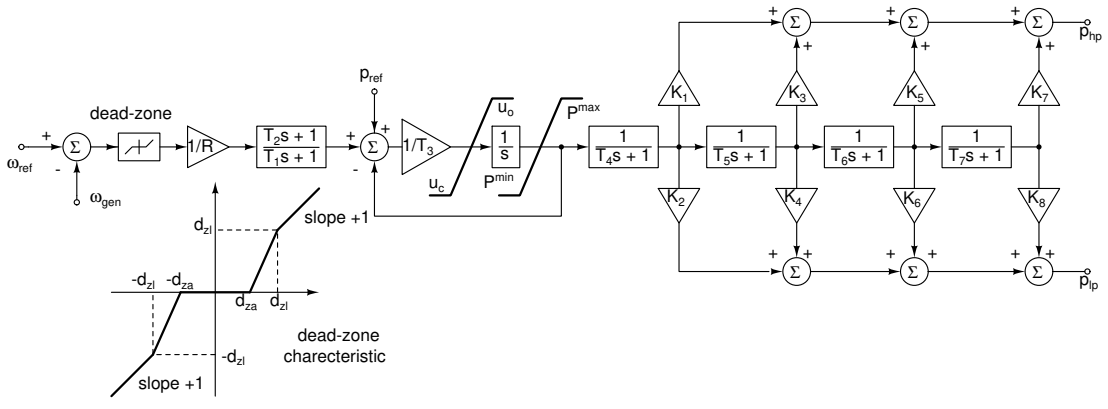

Figure 8.10: The block schematic of the IEEEG1 type turbine governor.

of the model of the IEEEG1 turbine governor are

$$T_1\frac{dx_1}{dt}+x_1-\frac{1}{R}f\left(\omega_{\text{ref}}-\omega_{\text{gen}}\right)=0$$

$$p_1=p_2-\frac{T_2}{T_1R}f\left(\omega_{\text{ref}}-\omega_{\text{gen}}\right)+\left(\frac{T_2}{T_1}-1\right)x_1-p_{\text{ref}}$$

$$u_1=\begin{cases} p_1 & \text{if} \quad T_3u_c\le p_1\le T_3u_o \\ u_c & \text{if} \quad p_1<T_3u_c \\ u_o & \text{if} \quad p_1>T_3u_o \end{cases}$$

$$T_3\frac{dx_2}{dt}+u_1=0$$

$$p_2=\begin{cases} x_2 & \text{if} \quad p_{\min}\le x_2\le p_{\max} \\ p_{\min} & \text{if} \quad x_2<p_{\min} \\ p_{\max} & \text{if} \quad x_2>p_{\max} \end{cases}$$

$$T_4\frac{dx_3}{dt}+x_3-p_2=0$$

$$T_5\frac{dx_4}{dt}+x_4-x_3=0$$

$$T_6\frac{dx_5}{dt}+x_5-x_4=0$$

$$T_7\frac{dx_6}{dt}+x_6-x_5=0$$

$$p_{\text{hp}}=K_1x_3+K_3x_4+K_5x_5+K_7x_6$$

$$p_{\text{lp}}=K_2x_3+K_4x_4+K_6x_6+K_8x_6$$

Type IEEEG3 (hydro)

The block schematic of the IEEEG3 type hydro turbine governor is shown in Figure 8.11. The

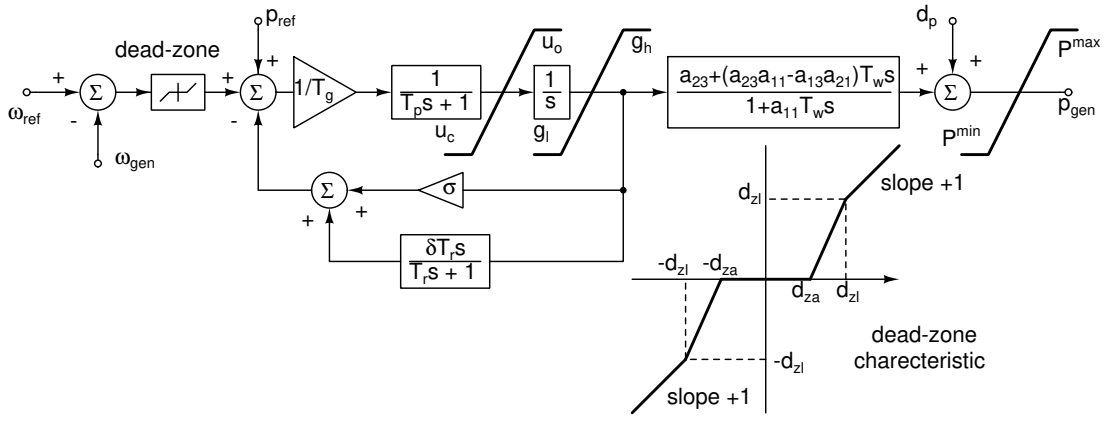

Figure 8.11: The block schematic of the IEEEG3 type hydro turbine governor.

equations of the model of the IEEEG3 turbine governor are

$$\delta_\omega = f\left(\omega_{\text{ref}} - \omega_{\text{gen}}\right)$$
$$T_g\left(T_p\frac{dx_1}{dt} + x_1\right) - \left[\delta_\omega + p_{\text{ref}} - (\sigma+\delta)\,p_1 + \delta x_3\right] = 0$$
$$u_1 = \begin{cases} x_1 & \text{if} \quad u_c \le x_1 \le u_o \\ u_c & \text{if} \quad x_1 < u_c \\ u_o & \text{if} \quad x_1 > u_o \end{cases}$$
$$\frac{dx_2}{dt} - u_1 = 0$$
$$p_1 = \begin{cases} x_2 & \text{if} \quad g_1 \le x_2 \le g_{\text{h}} \\ g_1 & \text{if} \quad x_2 < g_1 \\ g_{\text{h}} & \text{if} \quad x_2 > g_{\text{h}} \end{cases}$$
$$T_r\frac{dx_3}{dt} + x_3 - p_1 = 0$$
$$a_{11}T_w\frac{dx_4}{dt} + x_4 - p_1 = 0$$
$$p_2 = \frac{k}{a_{11}}p_1 + \left(a_{23} - \frac{k}{a_{11}}\right)x_4 + d_{\text{p}}$$
$$p_{\text{gen}} = \begin{cases} p_2 & \text{if} \quad p_{\min} \le p_2 \le p_{\max} \\ p_{\min} & \text{if} \quad p_2 < p_{\min} \\ p_{\max} & \text{if} \quad p_2 > p_{\max} \end{cases}$$

where $k = (a_{23}a_{11} - a_{13}a_{21})$.

8.3.12 Power electric couplers

The `powerec` implements the following functions:

- phase/angle meter of the bus voltage;
- angular frequency meter of the bus voltage;
- power meter;
- current meter;
- passthrough coupler between the power sub-network and the electrical sub-network;
- power sub-network (DQ) to electrical sub-network (ABC) coupler.

Its main target of the `powerec` is to connect the power sub-network to the electrical sub-network. The `powerec` is always connected to a `bus` of the power sub-network and may have a different number of terminals connected to the electrical sub-network according to its task. The `powerec` instance must be in the power sub-network, in the electrical sub-network or indifferently in one of these twos, according to the type of task performed. Better said the type of instance pin/node, i.e. power or electrical types, imposes the element instance to be in the power sub-network, in the electrical sub-network or indifferently in one of these.

Phase meter

An instance example of the phase meter of the bus voltage is

```
Ec1 bus1 phase gnd powerec type=1
```

where `bus1` is the bus to which the meter is connected and the `phase gnd` terminal pair is that of the electric output port whose voltage is numerically equivalent to the phase (radiant) of the bus voltage. The instance must be in the power sub-network.

Angular frequency meter

The block schematic of the meter that estimates the angular frequency of a bus voltage is shown in Fig. 8.12. The input of the meter is the $\theta(t)$ instantaneous phase of the bus voltage. The

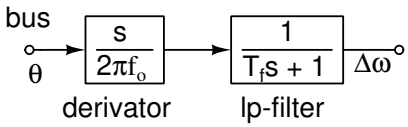

Figure 8.12: The block schematic of the angular frequency meter.

time derivative of $\theta(t)$ is used to estimate the instantaneous frequency of the bus voltage. Since numerical differentiation is prone to round-off errors, the result is low-pass filtered by the `lp-filter` block. The output of this block is the $\Delta\omega$ frequency variation of the bus voltage in p.u., since it is scaled by the $2\pi f_0$ factor, where f_0 is the reference frequency of the bus (typically 50 Hz or 60 Hz).
The transfer function of the meter is implemented as

$$H(s) = \frac{1}{2\pi f_o}\frac{1}{T_f}\left(1 - \frac{1}{1+T_f s}\right) \tag{8.22}$$

The differential equation that implements the (8.22) transfer function is

$$T_f\frac{du}{dt} + u = \theta$$
$$\Delta\omega = \frac{1}{2\pi f_o T_f}(\theta - u)$$

We suggest to low-pass filter once more the output of the frequency meter. An instance example of the frequency meter is

```
Ec1 bus1 freq gnd powerec type=2
```

where `bus1` is the bus to which the meter is connected and the `freq gnd` terminal pair is that of the electric output port whose voltage is numerically equivalent to the $\Delta\omega$ frequency (p.u.) of the bus voltage.

Power meter

The `powerec` of `type=4` is connected between two buses and measures positive power when it flows across it from `b0` to `b1`. An instance example is

```
Ec1 b0 pa gnd pq gnd b1 powerec type=4
```

The `pa,gnd` terminal pair forms a port whose voltage is numerically equal to the active power flow. The `pq,gnd` terminal pair forms a port whose voltage is numerically equal to the reactive power flow. Assume to measure the power of the `G1` synchronous generator in Figure 8.1. The `powerec` must be inserted in series to the `G1` generator as shown in Figure 8.13. The extra-bus **b0** was created and the `Pw` power meter was inserted between bus **b0** and **b1**.
This element allows to compute and use the complex power flow during a simulation, for example during a time domain (tran) simulation. It makes accessible the complex power through its pairs of output ports at *run-time*. If the complex power is needed only for post-elaboration purposes, i.e., it is needed after the end of a simulation, consider the possibility to access power of several power elements, such as active power of synchronous generators, by activating the `devvars` option of the time domain analyses, that saves those in a specific output file. This may make useless the introduction of this element.

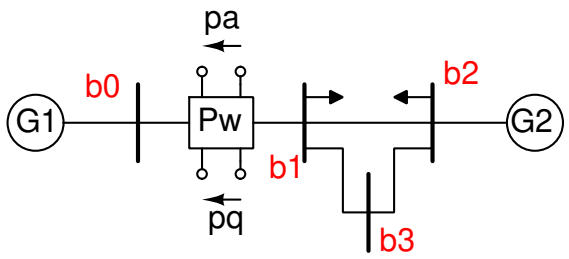

Figure 8.13: Power meter insertion.

Current meter

The `powerec` of `type=5` is connected between two buses and measures positive i_d and i_q components of the current, when it flows from `bus1` to `bus2`. An instance example is

```
Ec1 bus1 id gnd iq gnd bus2 powerec type=5
```

The `id gnd` terminal pair forms a port whose voltage is numerically equal to the i_d current. The `iq gnd` terminal pair forms a port whose voltage is numerically equal to the i_q current. It must be inserted as the power meter (see Figure 8.13).
This element allows to compute and use the complex current flowing through it during a simulation, for example during a time domain (tran) simulation. It makes accessible the complex current through its pair of output ports at *run-time*. If the complex current is needed only for post-elaboration purposes, i.e., it is needed after the end of a simulation, some of the electrical quantities of the power elements, such as current of synchronous generators, can be saved in a specific output file by activating the `devvars` option of the time domain analyses. This may make useless the introduction of this element.

Passthrough: DQ splitter

The `powerec` behaves as a virtual element that splits the vectorial dq components of the bus in two pairs of terminals. For example the instance

```
Ec1 bus1 dp dn qp qn powerec type=0
```

implements a `powerec` that connects the d-component to the (dp,dn) electrical port and the q-component to the electrical (qp,qn) port. The splitter behaves as an ideal power transfer element, i.e., it mirrors currents and voltages between the power bus and the two electrical ports. The instance of the `powerec` must be in the power sub-network.

DQ-ABC coupler

When it is used as a "dq-abc" converter (`type=3`), there must be three electrical terminals corresponding to the "a", "b" and "c" phases in the electrical sub-network. The "a","b" and "c" terminals are *star connected* in the electrical subsystem and the star connection is not grounded. The schematic of the "dq-abc" converter is shown in Figure 8.14. The three controlled voltage source generate three-phase balanced voltages, whose magnitude is governed by the Park transform detailed in the following. The currents of the current controlled sources are governed by the magnitude of those crossing the three controlled voltage sources. An instance example is

```
Ec1 bus1 a b c powerec type=3
```

This type of `powerec` behaves in different ways according to the fact that the analysis uses both the power and the electrical sub-networks or only the power sub-network or only the

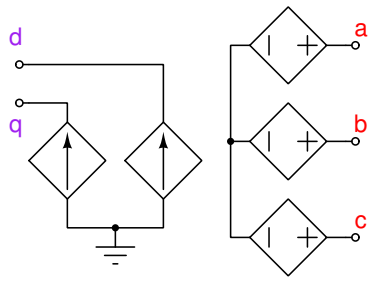

Figure 8.14: The schematic of the dq-abc converter.

electrical sub-network (see the `nettype` parameter of the analyses). The transformation of the dq quantities to the corresponding "a", "b" and "c" ones is performed by the Park transform. More precisely for voltages we have

$$\begin{bmatrix} v_a \\ v_b \\ v_c \end{bmatrix} = \kappa_v \begin{bmatrix} \cos(2\pi f_o t) & -\sin(2\pi f_o t) \\ \cos(2\pi f_o t - \frac{2}{3}\pi) & -\sin(2\pi f_o t - \frac{2}{3}\pi) \\ \cos(2\pi f_o t + \frac{2}{3}\pi) & -\sin(2\pi f_o t + \frac{2}{3}\pi) \end{bmatrix} \begin{bmatrix} v_d \\ v_q \end{bmatrix}$$

where $\kappa_v \in \mathbb{R}$ is an arbitrary constant. In the conversion the *zero* component is omitted since it is not present in the single-phase equivalent model of the power sub-network. The f_o frequency is set by the `f0` parameter of the `powerec`. Assume that the $v_d \in \mathbb{R}$ component is constant and the $v_q = 0$, i.e., there is only the positive sequence with a suitable phase. In this simple case we have

$$\begin{aligned} v_a(t) &= v_d \kappa_v \cos(2\pi f_o t) \\ v_b(t) &= v_d \kappa_v \cos(2\pi f_o t - \tfrac{2}{3}\pi) \\ v_c(t) &= v_d \kappa_v \cos(2\pi f_o t + \tfrac{2}{3}\pi) \end{aligned} \tag{8.23}$$

Due to the *star connection* of the "a", "b" and "c" terminals of the 'dq-abc' coupler, the zero component of current is forced to zero. The currents injected in the dq-bus are

$$\begin{bmatrix} i_d \\ i_q \end{bmatrix} = \kappa_i \begin{bmatrix} \cos(2\pi f_o t) & \cos(2\pi f_o t - \frac{2}{3}\pi) & \cos(2\pi f_o t + \frac{2}{3}\pi) \\ -\sin(2\pi f_o t) & -\sin(2\pi f_o t - \frac{2}{3}\pi) & -\sin(2\pi f_o t + \frac{2}{3}\pi) \end{bmatrix} \begin{bmatrix} i_a \\ i_b \\ i_c \end{bmatrix}$$

where again κ_i is an arbitrary (till now) conversion constant.
Assume $i_a(t) = \frac{1}{R} v_a(t)$, $i_b(t) = \frac{1}{R} v_b(t)$, $i_c(t) = \frac{1}{R} v_c(t)$, where voltages are those in (8.23). We have

$$\begin{bmatrix} i_d \\ i_q \end{bmatrix} = \frac{\kappa_i \kappa_v v_d}{R} \begin{bmatrix} \cos(2\pi f_o t) & \cos(2\pi f_o t - \frac{2}{3}\pi) & \cos(2\pi f_o t + \frac{2}{3}\pi) \\ -\sin(2\pi f_o t) & -\sin(2\pi f_o t - \frac{2}{3}\pi) & -\sin(2\pi f_o t + \frac{2}{3}\pi) \end{bmatrix} \begin{bmatrix} \cos(2\pi f_o t) \\ \cos(2\pi f_o t - \frac{2}{3}\pi) \\ \cos(2\pi f_o t + \frac{2}{3}\pi) \end{bmatrix}$$

that simplifies as

$$\begin{bmatrix} i_d \\ i_q \end{bmatrix} = \frac{\kappa_i \kappa_v}{R} \frac{3}{2} \begin{bmatrix} v_d \\ 0 \end{bmatrix}$$

In the dq-frame active power is expressed as $P_{dq} = v_d i_d + v_q i_q$, thus in our case we obtain $P_{dq} = \frac{\kappa_i \kappa_v}{R} \frac{3}{2} v_d^2$. The corresponding active power in the "abc" frame is expressed as $P_{abc} =$

$v_a i_a + v_b i_b + v_c i_c$, in our case $P_{abc} = \frac{3}{2}\left(\frac{1}{R}k_v v_d\right)(k_v v_d) = \frac{3}{2}\frac{\kappa_v^2}{R}v_d^2$. We pretend that the "dq-abc" converter *does not contribute or dissipate power*. We thus pretend $P_{dq} = P_{abc}$, which leads to $\frac{\kappa_i \kappa_v}{R}\frac{3}{2}v_d^2 = \frac{\kappa_v^2}{R}\frac{3}{2}v_d^2$, that in turn simplifies $\kappa_i = \kappa_v$.
Since v_d is the RMS value of $v_{a,b,c}$ in (8.23), we choose $\kappa_v = \sqrt{2}$, which implies $\kappa_i = \sqrt{2}$. With this choice we have $P_{dq} = \frac{3}{R}v_d^2$. A more generic Z_{abc} impedance in the abc-frame is transformed by the "dq-abc" coupler in the $Z_{dq} = \frac{1}{3}Z_{abc}$ equivalent in the dq-frame (and vice versa).
Another "popular" choice of the κ_v and κ_i coefficients, which is not used in this "dq-abc" converter, is $\kappa_v = \kappa_i = \sqrt{\frac{2}{3}}$. It leads to $P_{dq} = P_{abc} = \frac{3}{2}\frac{\sqrt{\frac{2}{3}}\sqrt{\frac{2}{3}}}{R}v_d^2 = \frac{1}{R}v_d^2$. The single phase Z_{abc} in the "abc" reference frame is preserved in the "dq" refernce frame. Obviously the RMS values of the v_a, v_b and v_c voltages do no longer equal the modulus of the voltage in the "dq" frame. In this case the "dq-abc" converter would act as an ideal YΔ transformer with a turn ratio equal to 1, that thus it would lower RMS phase voltage by the $\sqrt{3}$ factor.

When only the power sub-network is used (`nettype=2`), the `powercec` is an open circuit for what concerns the bus terminal in the dq-frame, i.e., it does not link the power sub-network to the electrical (ignored) sub-network.

When only the electrical sub-network is used it is an open circuit for what concerns the electrical terminals if the `v0` parameter is not given, otherwise it behaves like an independent three-phase voltage source connected to the electrical sub-network generating sinusoidal waveforms at the `f0` frequency. This means that if the `v0` parameter is not given the `powerec` behaves like an open circuit. If the `v0` parameter is given it behaves just like an independent three-phase voltage source.

When the `devvar=true` argument of analyses is given, the active and reactive power flowing through the "dq-abc" converter is saved in the output file. The active power is assumed positive when it flows from the abc-frame to the dq-frame. The (inductive, i.e. positive) reactive power is assumed positive when it flows from the abc-frame to the dq-frame.

Scalar power-electrical coupler

Scalar terminals of the instances in the power sub-network, such as for example the mechanical output terminal of a turbine governor is of type "power". To access this scalar connection from the electrical sub-network a `powerec` instance is needed. This instance must be in the electrical sub-network. For example the instance

```
Ec1 ex1 ep en powerec type=0
```

implements a `powerec` that connects the `ex1` scalar component of the power sub-network to the (ep,en) electrical port in the electrical sub-network. Recall that most but not all scalar power terminals can be directly accessed by the electrical sub-network without the insertion of any `powerec`. If `PAN` complains about a wrong connection, use a `powerec` for coupling scalar terminals in the power sub-network and electrical terminals in the electrical sub-network. Note that quantities at nodes in the power sub-network are always referred to ground.

8.3.13 Power system stabiliser

The block schematic of the built-in TypeII power system stabiliser is shown in Figure 8.15. The

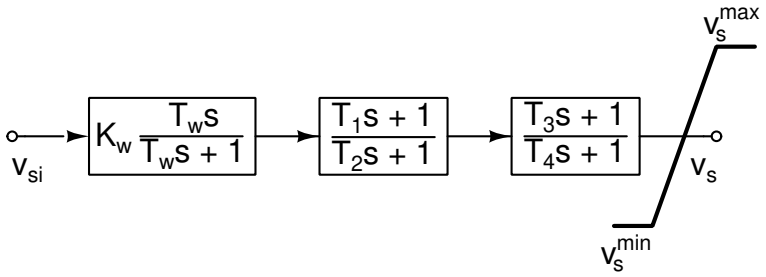

Figure 8.15: The block schematic up of the TypeII power system stabiliser.

equation of implementing its model are:

$$\begin{aligned}
&T_w\frac{dv_1}{dt}+K_w v_{si}+v_1=0\\
&T_2\frac{dv_2}{dt}+\left(\frac{T_1}{T_2}-1\right)(K_w v_{si}+v_1)+v_2=0\\
&T_4\frac{dv_3}{dt}+\left(\frac{T_3}{T_4}-1\right)\left(v_2+\frac{T_1}{T_2}(K_w v_{si}+v_1)\right)+v_3=0\\
&v_s=v_3+\frac{T_3}{T_4}\left(v_2+\frac{T_1}{T_2}(K_w v_{si}+v_1)\right)
\end{aligned} \tag{8.24}$$

8.3.14 Wind turbine

The $C_p(\beta,\lambda)$ wind power curve depends on the β blade pitch angle (degree) and on the λ parameter defined as

$$\lambda = G_r R\frac{\omega_g}{v_w}$$

where G_r is the gear ratio, i.e., the ratio between the angular speed of the shaft connected to the electrical generator and the angular speed of the blades shaft, R is the blade radius, ω_g is the angular speed of the generator shaft, v_w is the wind speed.

The $C_p(\beta,\lambda)$ function has two implementation that can be selected through the `cptype` argument of the `powerwt` instance The `cptype=1` function is defined by the

$$C_p(\beta,\lambda)=\max\left(0,\sum_{k=0}^{4}\sum_{j=0}^{4}C_{k+1,j+1}\beta^k\min(25,\lambda)^j\right)$$

polynomial, where the $C_{k+1,j+1}$ matrix is

$$C=\begin{bmatrix}
-4.1909e-1 & 2.1808e-1 & -1.2406e-2 & -1.3365e-4 & 8.1524e-6\\
-6.7606e-2 & 6.0405e-2 & -1.3934e-2 & 1.0683e-3 & -2.3895e-5\\
1.5727e-2 & -1.0996e-2 & 2.1495e-3 & -1.4855e-4 & 2.7937e-6\\
-8.6018e-4 & 5.7051e-4 & -1.0479e-4 & 5.9924e-6 & -8.9194e-8\\
1.4787e-5 & -9.4839e-6 & 1.1667e-6 & -7.1535e-8 & 4.9686e-10
\end{bmatrix}$$

λ is upper limited and the resulting $C_p(\beta,\lambda)$ is lower limited. It was used in the GENERAL ELECTRIC model of a 3 MW wind turbine model. The author stated that it well fits the physical behaviour when λ is in the $[2,20]$ range and β in the $[0^o,15^o]$ range. When $\lambda<2$ the $C_p(\beta,\lambda)$ values can drop to 0.

The second `cptype=2` expression of the $C_p(\beta,\lambda)$ wind power curve is that found in MATLAB/SIMULINK,

$$C_p(\beta,\lambda) = c_1\,(c_2\gamma - c_3\beta - c_4)\,e^{-c_5\gamma} \qquad \gamma = \frac{1}{\lambda + 0.08\beta} - \frac{0.035}{\beta^3+1}$$

where $c_1 = 0.5176$, $c_2 = 116$, $c_3 = 0.4$, $c_4 = 5$, $c_5 = 21$, $c_6 = 0.0068$. The $c_5\gamma$ argument is limited at the maximum allowed exponential value by the ALU (about 270), which leads to a very small exponential result.

The T_m mechanical torque that drives the electrical generator shaft is

$$T_m = \frac{1}{2}\rho\pi R^2 N_b C_p(\beta,\lambda)\frac{v_w^3}{\omega_g}$$

where ρ is the air density, N_b is the number of blades. Assuming a constant v_w speed, the T_m torque lowers, if β increases; the minimum allowed value of β is 0.

The block schematic of the built-in blade pitch angle controller is shown in Figure 8.16.

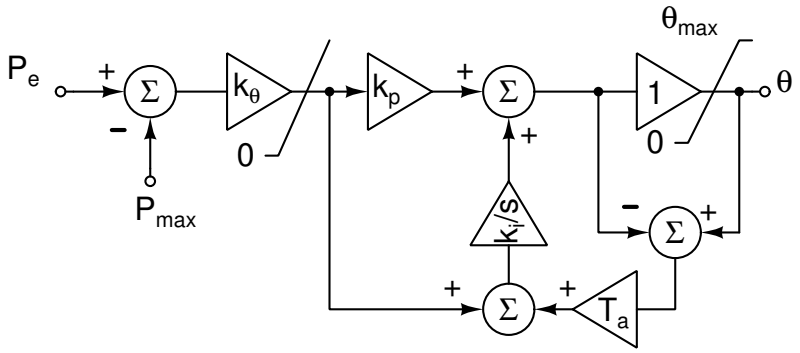

Figure 8.16: The PI controller with anti wind-up of the blade pitch angle.

8.4 Power flow analysis

The target of PF analysis is the determination of an equilibrium point of the power system. The dynamic part is ignored, power and Kirchhoff voltage and current laws are solved. `PAN` used the MNA formulation to model the power system. The PF solution is computed by an extended DC analysis that acts also on the power networks according to the value of the `nettype` parameter. The `nettype=1` parameter tells the DC analysis to consider also the power sub-network in addition to the electrical sub-network in computing the operating point of the full circuit.
Consider the Netlist 8.0.1 and the

```
Dc dc nettype=1 print=1
```

statement; it performs the `Dc` DC analysis and the `nettype=1` argument tells that it must consider the power network that is described in the `begin power/end` section. When a solution is determined the dynamic elements of the circuit (power network) are initialised. The `print=1` parameter shows the solution and the initialisation of the dynamic elements such as for example the those of synchronous generators.
The logfile of the simulation of the simple power system shown in 8.0.1 is reported Logfile 8.4.1.

Logfile 8.4.1 — Logfile of simple power netlist. –

```
Power balance of the power-sub-network:

                    P             Q
Generators:     109.91 MW     -5.9572 MVAR
Lines:         -106.28 kW      10.997 MVAR
Loads:         -109.80 MW     -5.0400 MVAR

DC analysis <Dc> successfully terminated in <2> iterations.

Current solution of <Dc> DC analysis:

|V(b1)| = 73.140 kV D:  73.140 kV Q:  0.0000  V Ph:  0.0000  Rad  0.0000  Deg
|V(b2)| = 72.105 kV D:  72.099 kV Q: -911.77  V Ph: -12.645 mRad -724.53 mDeg
|V(b3)| = 72.836 kV D:  72.834 kV Q: -527.96  V Ph: -7.2487 mRad -415.32 mDeg
|I(G1)| = 958.94  A D: -955.79  A Q:  77.659  A Ph:  3.0605  Rad  175.35  Deg
|I(G2)| = 577.75  A D: -556.74  A Q: -154.36  A Ph: -2.8711  Rad -164.50  Deg

Generator <G2>
    Pm   :  40.082 MW       668.04 mPu
    Pe   :  40.000 MW       666.67 mPu
    Qe   : -11.637 MVAR    -193.95 mPu
    Delta:  613.59 mRad
    Omega:  1.0000  Rad/sec
    Eqp  :  61.471 kV       890.88 mPu
    Eqs  :  60.561 kV       877.70 mPu
    Edp  :  23.577 kV       341.69 mPu
    Eds  :  36.696 kV       531.83 mPu
    Vf   :  74.751 kV       1.0833  Pu

Generator <G1> (slack)
    Pm   :  70.132 MW       1.1689  Pu
    Pe   :  69.906 MW       1.1651  Pu
    Qe   :  5.6800 MVAR     94.666 mPu
    Delta:  752.18 mRad
    Omega:  1.0000  Rad/sec
    Eqp  :  64.209 kV       930.56 mPu
    Eqs  :  60.887 kV       882.42 mPu
    Edp  :  27.945 kV       404.99 mPu
    Eds  :  43.494 kV       630.35 mPu
    Vf   :  112.70 kV       1.6333  Pu
```

8.5 Time domain analyses

The transient stability analysis is based on the TRAN analysis as the PF analysis described in Section 8.4 is based on the DC analysis. A proper selection of the `nettype` parameter of the TRAN analysis turns on its acting also on the power network. A use example is

```
Tr tran stop=10 nettype=2
```

During a tran analysis the active and reactive powers of the synchronous generators measured at the buses to which they are connected (this does not take into account the active and reactive internal powers of the generators) can be saved in the output files, if the `devvars` option of the tran analysis is set true.

9. Simulations of three-phase systems

We refer to three-phase systems and their elements as *"ph3"*. In general the `ph3` elements can be intantiated preferably in a `begin power`, `end` sub-netlist; however when it is not ambiguous they can be instantiated also outside thus sub-netlist block and their model is the `ph3` type.
When they are instantiated in the `begin electrical`, `end` electrical sub-netlist block, their model is automatically switched from `ph3` to `electrical`; thus the `ph3` elements are modeled as `electrical` ones and allow for example a conventional `tran` time domain analysis. Said from the power system standpoint, they are modeled by *electro-magnetic-tranient* (EMT) models.

9.1 Ph3 elements

In this section some of the three-phase built-in elements implemented in `PAN` are described.

9.1.1 The ph3dq three phase and power sub-circuit coupler

The `ph3dq` coupler links the ph3 sub-circuit and the power sub-circuit.
The $v_{d,q}$ voltage of the bus in the dq-frame is converted to the $v_{ar,ai}$, $v_{br,bi}$ and $v_{cr,ci}$ voltages in the ph3-frame by the

$$\begin{bmatrix} v_{ar} \\ v_{ai} \\ v_{br} \\ v_{bi} \\ v_{cr} \\ v_{ci} \end{bmatrix} = \begin{bmatrix} \cos(\phi) & \sin(\phi) \\ -\sin(\phi) & \cos(\phi) \\ \cos\left(\frac{2}{3}\pi+\phi\right) & \sin\left(\frac{2}{3}\pi+\phi\right) \\ -\sin\left(\frac{2}{3}\pi+\phi\right) & \cos\left(\frac{2}{3}\pi+\phi\right) \\ \cos\left(-\frac{2}{3}\pi+\phi\right) & \sin\left(-\frac{2}{3}\pi+\phi\right) \\ -\sin\left(-\frac{2}{3}\pi+\phi\right) & \cos\left(-\frac{2}{3}\pi+\phi\right) \end{bmatrix} \begin{bmatrix} v_d \\ v_q \end{bmatrix}$$

linear time invariant transformation. The $v_{ar,ai}$, $v_{br,bi}$, $v_{cr,ci}$ voltage pairs drive the ph3 sub-circuit bus at which the `ph3dq` coupler is connected.
The $i_{ar,ai}$, $i_{br,bi}$, $i_{cr,ci}$ currents of bus at which the `ph3dq` coupler is connected are converted to the

corresponding i_a, i_b and i_c currents in the abc-frame by the transformation

$$\begin{bmatrix} i_a \\ i_b \\ i_c \end{bmatrix} = \sqrt{2}\left(\mathbf{1}_3 \otimes \begin{bmatrix} \cos(\omega t) & -\sin(\omega t) \end{bmatrix}\right) \begin{bmatrix} i_{ar} \\ i_{ai} \\ i_{br} \\ i_{bi} \\ i_{cr} \\ i_{ci} \end{bmatrix}$$

where $\mathbf{1}_3$ is the identity matrix of order 3 and $\otimes$ is the Kronecker product. The corresponding $i_{,q}$ currents in the dq-frame are given by the Park's transformation

$$\begin{bmatrix} i_d \\ i_q \end{bmatrix} = \sqrt{2} \begin{bmatrix} \cos(\omega t) & \cos\left(\omega t - \frac{2}{3}\pi\right) & \cos\left(\omega t + \frac{2}{3}\pi\right) \\ -\sin(\omega t) & -\sin\left(\omega t - \frac{2}{3}\pi\right) & -\sin\left(\omega t + \frac{2}{3}\pi\right) \end{bmatrix} \begin{bmatrix} i_a \\ i_b \\ i_c \end{bmatrix}$$

The zero component is omitted since it is dropped by the Y connection of the sources. The transformation of these current from the ph3-frame to the dq-frame is thus

$$\begin{bmatrix} i_d \\ i_q \end{bmatrix} = 2 \begin{bmatrix} \cos(\omega t) & \cos\left(\omega t - \frac{2}{3}\pi\right) & \cos\left(\omega t + \frac{2}{3}\pi\right) \\ -\sin(\omega t) & -\sin\left(\omega t - \frac{2}{3}\pi\right) & -\sin\left(\omega t + \frac{2}{3}\pi\right) \end{bmatrix} \times$$

$$\times \left(\mathbf{1}_3 \otimes \begin{bmatrix} \cos(\omega t) & -\sin(\omega t) \end{bmatrix}\right) \begin{bmatrix} i_{ar} \\ i_{ai} \\ i_{br} \\ i_{bi} \\ i_{cr} \\ i_{ci} \end{bmatrix} =$$

$$= 2 \begin{bmatrix} \cos(\omega t) & \cos\left(\omega t - \frac{2}{3}\pi\right) & \cos\left(\omega t + \frac{2}{3}\pi\right) \\ -\sin(\omega t) & -\sin\left(\omega t - \frac{2}{3}\pi\right) & -\sin\left(\omega t + \frac{2}{3}\pi\right) \end{bmatrix} \begin{bmatrix} \cos(\omega t)\, i_{ar} - \sin(\omega t)\, i_{ai} \\ \cos(\omega t)\, i_{br} - \sin(\omega t)\, i_{bi} \\ \cos(\omega t)\, i_{cr} - \sin(\omega t)\, i_{ci} \end{bmatrix}$$

A further expansion leads to

$$\begin{aligned} \tfrac{1}{2} i_d = \;& \cos(\omega t)^2\, i_{ar} - \cos(\omega t)\sin(\omega t)\, i_{ai} + \\ & \cos\left(\omega t - \tfrac{2}{3}\pi\right)\cos(\omega t)\, i_{br} - \cos\left(\omega t - \tfrac{2}{3}\pi\right)\sin(\omega t)\, i_{bi} + \\ & \cos\left(\omega t + \tfrac{2}{3}\pi\right)\cos(\omega t)\, i_{cr} - \cos\left(\omega t + \tfrac{2}{3}\pi\right)\sin(\omega t)\, i_{ci} \\ \tfrac{1}{2} i_q = \;& -\sin(\omega t)\cos(\omega t) i_{ar} + \sin(\omega t)^2 - \\ & \sin\left(\omega t - \tfrac{2}{3}\pi\right)\cos(\omega t)\, i_{br} + \sin\left(\omega t - \tfrac{2}{3}\pi\right)\sin(\omega t)\, i_{bi} - \\ & \sin\left(\omega t + \tfrac{2}{3}\pi\right)\cos(\omega t)\, i_{cr} + \sin\left(\omega t + \tfrac{2}{3}\pi\right)\sin(\omega t)\, i_{ci} \end{aligned}$$

To avoid the injection of (hightly oscillating) negative sequence and harmonics in the dq sub-circuit in case of an unbalances and/or non-linear ph3 sub-circuit, the `ph3dq` coupler can rebalance the $i_{a,b,c}$ currents. Note that balancing keeps the `ph3dq` coupler an *inert* element, i.e., is does not contribute any instantaneous power. Current ballancing is done in this way

$$\begin{aligned} P_b &= v_{ar} i_{ar} + v_{ai} i_{ai} + v_{br} i_{br} + v_{bi} i_{bi} + v_{cr} i_{cr} + v_{ci} i_{ci} \\ Q_b &= v_{ai} i_{ar} - v_{ar} i_{ai} + v_{bi} i_{br} - v_{br} i_{bi} + v_{ci} i_{cr} - v_{cr} i_{ci} \\ i_d &= \frac{P_b v_d + Q_b v_q}{v_d^2 + v_q^2} \\ i_q &= \frac{P_b v_q - Q_b v_d}{v_d^2 + v_q^2} \end{aligned}$$

9.1.2 Statcom

There are three versions of the `ph3statcom`: (i) the I_o current, exchanged at the bus where the statcom is connected, is controlled by the bus voltage itself; (ii) the current is externally controlled through a terminal of the statcom; (iii) the current is controlled by the reactive power flowing through a meter

voltage controlled statcom

The `ph3statcom` exchanges reactive power in order to keep each phase voltage at the given v_r voltage set-point. A three-phase statcom is Δ connected, in this case the `phase` instance argument is equal to `phases="abc"`. When the statcom in connected to only two phase, for example 'b' and 'c', the `phase` instance argument is equal to `phases="bc"`. If it is connected to a single phase and neutral, the `phase` instance argument is a single character identifing the phase, for example `phases="b"`.

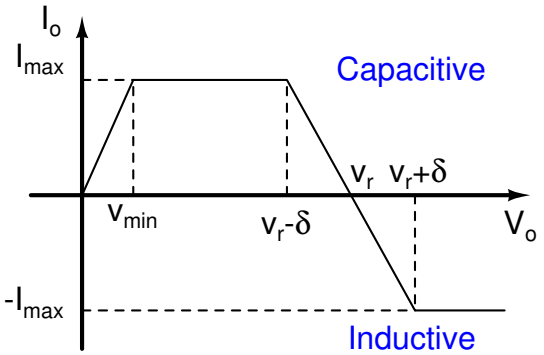

Figure 9.1: The single phase current versus single phase voltage characteristic of the statcom.

The characteristic of the `ph3statcom` is

$$
\begin{aligned}
&T_d\frac{dV_o}{dt}+V_o-\sqrt{v_d^2+v_q^2}=0\\
&I_o=\begin{cases}
\dfrac{I_\mathrm{max}}{v_\mathrm{min}}V_o & V_o<v_\mathrm{min}\\
I_\mathrm{max} & v_\mathrm{min}\le V_o\le v_\mathrm{r}-\delta\\
\dfrac{I_\mathrm{max}}{\delta}(v_\mathrm{r}-V_o) & v_\mathrm{r}-\delta<V_o<v_\mathrm{r}+\delta\\
-I_\mathrm{max} & V_o\ge v_\mathrm{r}+\delta
\end{cases}
\end{aligned}
\tag{9.1}
$$

where I_o is the current flowing through each phase of the `ph3statcom` (normal convention); v_r is the given voltage set-point; I_max is the maximum absolute value of the `ph3statcom` phase current, the phase current saturates at this value; v_min is the minimum modulus of the phase voltage below which the `ph3statcom` starts to lower the phase current toward 0; 2δ is the voltage interval inside which the drop controller of the `ph3statcom` regulates the phase current.

$$
\mathrm{I}\left(\frac{v_d}{\sqrt{v_d^2+v_q^2}}+\mathrm{I}\frac{v_q}{\sqrt{v_d^2+v_q^2}}\right)=\frac{i_d}{I_o}+\mathrm{I}\frac{i_q}{I_o}\Longrightarrow i_d=-\frac{I_o v_q}{\sqrt{v_d^2+v_q^2}}\ \text{ and }\ i_q=\frac{I_o v_d}{\sqrt{v_d^2+v_q^2}} \tag{9.2}
$$

where i_d and i_q are the direct and quadrature components of the `ph3statcom` phase current, that is $|I_o|=\sqrt{i_d^2+i_q^2}$ and $V_o=\sqrt{v_d^2+v_q^2}$.

At steady state by substituting the expression of I_o of (9.1) in the expression of i_d in (9.2) we have

$$i_d = \begin{cases} -\dfrac{I_{\max} v_q}{v_{\min}} & V_o < v_{\min} \\ -\dfrac{I_{\max} v_q}{V_o} & v_{\min} \leq V_o \leq v_{\mathrm{r}} - \delta \\ \dfrac{I_{\max}}{\delta} v_q \left(1 - \dfrac{v_r}{V_o}\right) & v_{\mathrm{r}} - \delta < V_o < v_{\mathrm{r}} + \delta \\ \dfrac{I_{\max} v_q}{V_o} & V_o \geq v_{\mathrm{r}} + \delta \end{cases}$$

and

$$i_q = \begin{cases} \dfrac{I_{\max} v_d}{v_{\min}} & V_o < v_{\min} \\ \dfrac{I_{\max} v_d}{V_o} & v_{\min} \leq V_o \leq v_{\mathrm{r}} - \delta \\ \dfrac{I_{\max}}{\delta} v_d \left(\dfrac{v_r}{V_o} - 1\right) & v_{\mathrm{r}} - \delta < V_o < v_{\mathrm{r}} + \delta \\ -\dfrac{I_{\max} v_d}{V_o} & V_o \geq v_{\mathrm{r}} + \delta \end{cases}$$

where here $V_o = \sqrt{v_d^2 + v_q^2}$.

10. Simulations of FMI modules

PAN allows to import fmi modules and insert multiple instances of modules in the netlist. A usage examples is shown by 10.0.1,

Netlist 10.0.1 — VanDerPol oscillator. –

```
ground electrical gnd
Tr tran stop=10 tmax=10m
I1  (x0 gnd) (x1 gnd) VDP vout=[p1,"x0",p2,"x1"]
I2  (y0 gnd) (y1 gnd) VDP vout=[p1,"x0",p2,"x1"] mu=0.1
model VDP fmi path="VanDerPol.fmu"
```

where we have the two I1 and I2 instances of the VDP object. VDP model is of fmi type and it is loaded by the simulator from the "VanDerPol.fmu" file specified by the path parameter of the model card. Each instance has two ports: (x0 gnd) and (x1 gnd); these ports are connects to the pints of the fmi module as described by the vout parameter of the instance card. Specifically the first port, which is identified by the p1 statement, where 'p' is the prefix of the port number, followed by the name of the corresponding output terminal of the fmi module. The list is thus composed of statement pairs describing how the instance ports are connected to the fmi module terminals. There is a similar instance parameter for the fmi module input terminals. The vout parameter specifies that the instance port behaves like a controlled voltage source, where its control is the corresponding output terminal of the fmi module. Parameters of the fmi module can be modified by the instance card; by observing the I2 instance the fmi module mu parameter is set at 0.1. The corresponding value of the mu parameter is left at its default values in the I1 instance.

10.1 Stability issues of co-simulation

The co-simulation of fmi modules is affected by several numerical problems and among these, by stability issues. In the sequel we give a simple insight on what can happen.

Consider the simple scalar differential algebraic equation (DAE)

$$\begin{cases} \dfrac{dx}{dt} + \lambda x(t) + u(t) = 0 \\ u(t) = kx(t) \end{cases} \tag{10.1}$$

where the algebraic equation introduces a loop and $\lambda \in \mathbb{C}$, $k \in \mathbb{R}$. We assume to solve it by the implicit Euler method and to check stability against values of k. We get

$$\begin{cases} \dfrac{x_{n+1}-x_n}{h} + \lambda x_{n+1} + u_{n+1} = 0 \\ u_{n+1} = k x_{n+1} \end{cases}$$

which leads to the recursion expression $x_{n+1} = \dfrac{1}{1+h(\lambda+k)} x_n$ and to $x_{n+m} = \dfrac{1}{(1+h(\lambda+k))^m} x_n$. We have stability if $|1+h(\lambda+k)| \geq 1$ or equivalently

$$|1+h(\lambda+k)|^2 \geq 1 \tag{10.2}$$

We show some algebraic handling of (10.2) to get the condition on k that ensure stability. By assuming that $\lambda = \rho e^{\mathrm{I}\theta}$ with $-\frac{\pi}{2} \leq \theta \leq \frac{\pi}{2}$ and $\mathrm{I} = \sqrt{-1}$, we have $(1+h(\rho\cos(\theta)+k))^2 + h^2\rho^2\sin(\theta)^2 \geq 1$ which simplifies as

$$hk^2 + 2(h\rho\cos(\theta)+1)k + h\rho^2 + 2\rho\cos(\theta) \geq 0 \tag{10.3}$$

From (10.3) we derive the two roots

$$k_{1,2} = \frac{-2(h\rho\cos(\theta)+1) \pm \sqrt{(h\rho\cos(\theta)+1)^2 - h\rho(2\cos(\theta)+h)}}{2h}$$

Since $\cos(\theta) \geq 0$ we have that both $k_{1,2}$ roots have *negative* real parts. Since k is a positive real constant, the application of the implicit Euler method to the (10.1) stable DAE always leads to a stable solution.

Now we introduce a delay due to the coupling of the internal solver of the fmi module and the simulator solver (co-simulation).
Consider the simple scalar DAE

$$\begin{cases} \dfrac{dx}{dt} + \lambda x(t) + u(t) = 0 \\ u(t) = k x(t-h) \end{cases} \tag{10.4}$$

The $u(t)$ in the second equation of (10.4) is now the $h \in \mathbb{R}^+$ delayed version of $x(t)$ in (10.1). Assume to solve (10.4) by the implicit Euler method, exactly as before, and to check stability versus variation of k with $\Re(\lambda) \geq 0$. We obtain

$$\begin{cases} \dfrac{x_{n+1}-x_n}{h} + \lambda x_{n+1} + u_{n+1} = 0 \\ u_{n+1} = k x_n \end{cases}$$

and the recursion expression $x_{n+1} = \dfrac{1-hk}{1+h\lambda} x_n$ from which we thus have $x_{n+m} = \left(\dfrac{1-hk}{1+h\lambda}\right)^m x_n$. We have stability when $\left|\dfrac{1-hk}{1+h\lambda}\right| \leq 1$, which implies

$$|1-hk|^2 \leq |1+h\lambda|^2 \tag{10.5}$$

We now show some algebraic handling of (10.5) to express k as a function of h. From (10.5) we have

$$1+h^2k^2-2hk \leq (1+h\rho\cos(\theta))^2 + h^2k^2\sin(\theta)^2 = 1+h^2\rho^2+2h\rho\cos(\theta)$$

Some simplifications leads to $k\,(hk-2)-\rho\,(h\rho+2\cos(\theta))\leq 0$, which is solved with respect to k obtaining

$$k_1=\frac{1-\sqrt{1+h\rho\,(h\rho+2\cos(\theta))}}{h}<0,\qquad k_2=\frac{1+\sqrt{1+h\rho\,(h\rho+2\cos(\theta))}}{h}>0$$

We have *stability if* $0\leq k\leq k_2$, which clearly shows that the delay can transform the originally *stable* DAE in an *unstable* one when the implicit Euler integration method is applied. This depending on the k loop gain. By observing k_2 we see that $\lim_{h\to 0}k_2=+\infty$, which suggest that *stability* can be gained also with a delay if a sufficiently short integration time step h is chosen (basically a sufficiently short delay is introduced). On the other hand $\lim_{h\to infty}k_2=\rho$, which suggest that having $k<\rho$ can be mandatory to pursue stability.

11. Reference

11.1 Where and How: downloading and installing

The PAN circuit simulator binary can be downloaded from `http://brambilla.ws.dei.polimi.it`. The PAN binary requires a standard linux installation. PAN developers use Debian based systems (most recently Debian, Mint and Ubuntu), but PAN has been used without any pain with other Linux flavours, including RedHat, Gentoo, Sabayon and several others.
Using PAN is relatively simple, and only requires knowledge of basic Linux terminal commands and the use of any ASCII editor (such as Vi).

11.2 Compatibility

Netlists are entered in ASCII format in the native language of PAN , but other languages and SPICE dialects are recognised. Standard SPICE cards are known together with those of ELDO and PSPICE. The language implemented in SPECTRE is also known. Note that compatibility is not always 100%. The various card languages can be freely mixed without any indication to PAN that automatically recognises them. For example

Netlist 11.2.1 — Mixed languages. —

```
V1   n10    0   10
R1   n10  n20   resistor r=1
R2   n10    0   1
.op
```

that mixes PAN and SPICE card is allowed and correctly parsed.

SPICE single and multi-ports controlled sourced can be described by the `poly` statemenet; its

expression is:

$$
\begin{aligned}
y = \quad & p_0 + \\
& p_1 x_1 + p_2 x_2 + \cdots + p_n x_n \\
& p_{n+1} x_1 x_1 + p_{n+2} x_1 x_2 + \cdots + c_{n+n} x_1 x_n \\
& p_{2n+1} x_2 x_2 + p_{2n+2} x_2 x_3 + \cdots + c_{2n+n} x_2 x_n \\
& \vdots \\
& p_{2n+{}^{n!}/_{2(2n-2)!}} x_n x_n + \\
& p_{2n+1+{}^{n!}/_{2(2n-2)!}} x_1^2 x_1 + p_{2n+2+{}^{n!}/_{2(2n-2)!}} x_1^2 x_2 + \\
& \vdots
\end{aligned}
$$

where p_n are the coefficients of the polynomial expression and x_n are the inputs of the polynomial element, for example port voltages or currents.

Bibliography

Books
Articles
Manuals
Technical Reports